

Advanced Algorithms & Data Structures



Department of CSE

ADVANCED ALGORITHMS AND DATA STRUCTURES 23CS03HF

Topic:

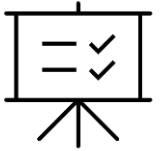
Divide and Conquer, Merge Sort

AIM OF THE SESSION



To familiarize students with the concept of Divide and Conquer and Merge sort

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

1. Demonstrate :- Divide and conquer and merge sort.

Describe :- Recurrence relation and time complexity of merge sort

LEARNING OUTCOMES



At the end of this session, you should be able to:

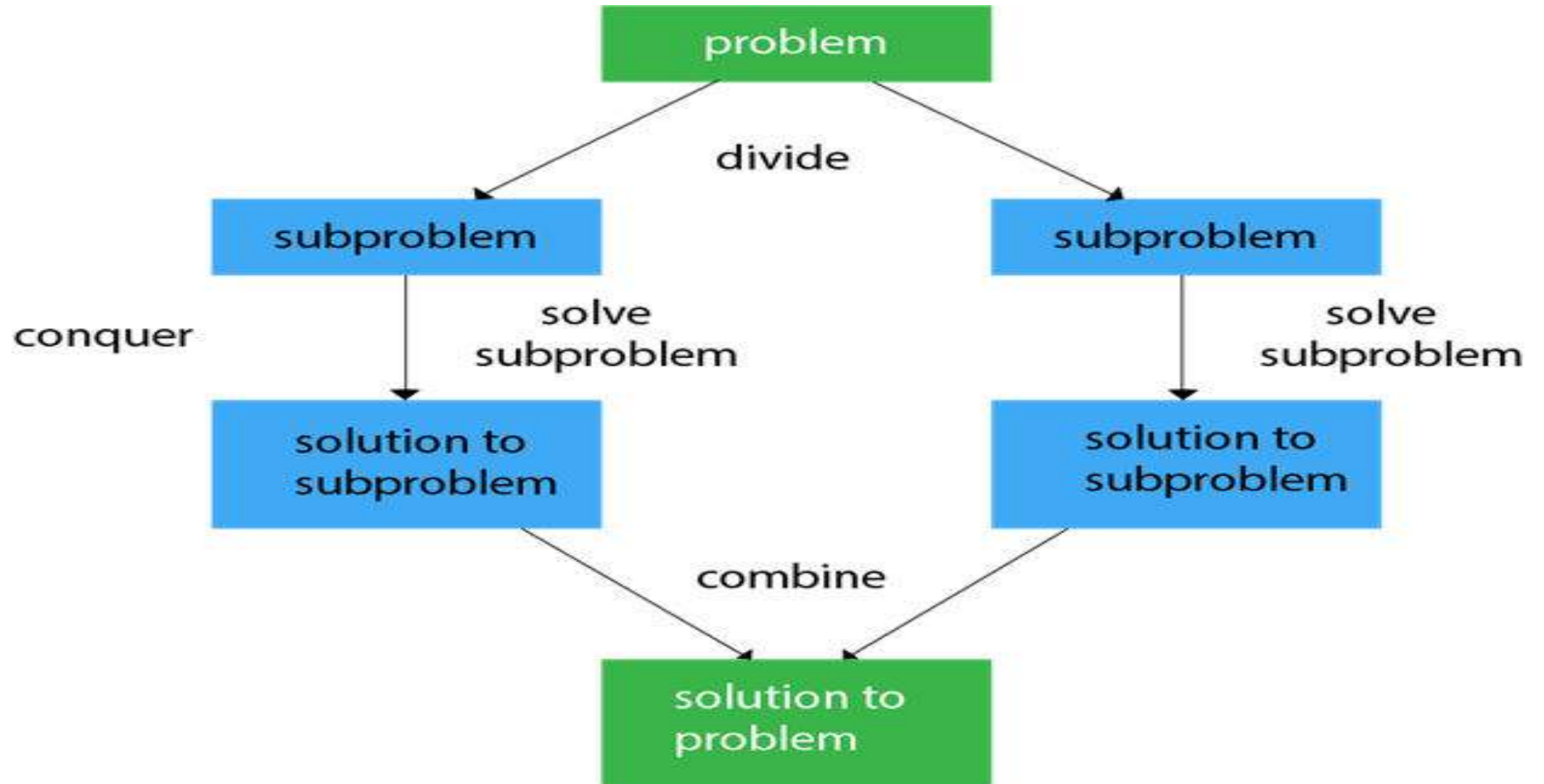
1. Define :- Merge sort .

2. Describe :- Recurrence relation and time complexity of merge sort

3. Summarize:- It gives the description about the merge sort and time complexity of merge sort

Divide and Conquer

- It is a top-down approach. The algorithms which follow the divide & conquer techniques involve three steps:
 - i. Divide the original problem into a set of subproblems.
 - ii. Solve every subproblem individually, recursively.
 - iii. Combine the solution of the subproblems (top level) into a solution of the whole original problem.



- Broadly, we can understand approach in a three-step process.

Divide/ Break

This step involves breaking the problem into smaller sub-problems. Sub-problems should represent a part of the original problem.

- This step generally takes a recursive approach to divide the problem until no sub-problem is further divisible.

Conquer/Solve

This step receives a lot of smaller sub-problems to be solved..

Merge/Combine

When the smaller sub-problems are solved, this stage recursively combines them until they formulate a solution of the original problem.

- This algorithmic approach works recursively and conquer & merge steps works so close that they appear as one

DIVIDE AND CONQUER ALGORITHM

Algorithm D and C(P)

```
{  
if small(P)  
then return S(P)  
else  
{ divide P into smaller instances P1 ,P2 .....Pk  
Apply D and C to each sub problem  
Return combine (D and C(P1)+ D and  
C(P2)+.....+D and C(Pk))}  
}
```

Examples

- 1) Binary Search
- 2) Quicksort
- 3) Merge Sort
- 4) Strassen's Algorithm
- 8) Calculating power
- 9) Factorial problem
- 10) Towers of Hanoi
- 11) Tree traversals :
- 12) Fibonacci number
- 13) Finding min,max etc

Merge Sort

- Merge sort is a divide-and-conquer algorithm based on the idea of breaking down a list into several sub-lists until each sub list consists of a single element and merging those sub lists in a manner that results into a sorted list.
- Idea:
 - Divide the unsorted list into N sub lists, each containing 1 element.
 - Take adjacent pairs of two singleton lists and merge them to form a list of 2 elements. will
now convert into $N/2$ lists of size 2.
 - Repeat the process till a single sorted list of obtained.

Suppose we had to sort an array A . A subproblem would be to sort a sub-section of this array starting at index p and ending at index r , denoted as $A[p..r]$.

Divide

If q is the half-way point between p and r , then we can split the subarray $A[p..r]$ into two arrays : $A[p..q]$ and $A[q+1, r]$

Conquer

In the conquer step, we try to sort both the subarrays $A[p..q]$ and $A[q+1, r]$. If we haven't yet reached the base case, we again divide both these subarrays and try to sort them

Combine

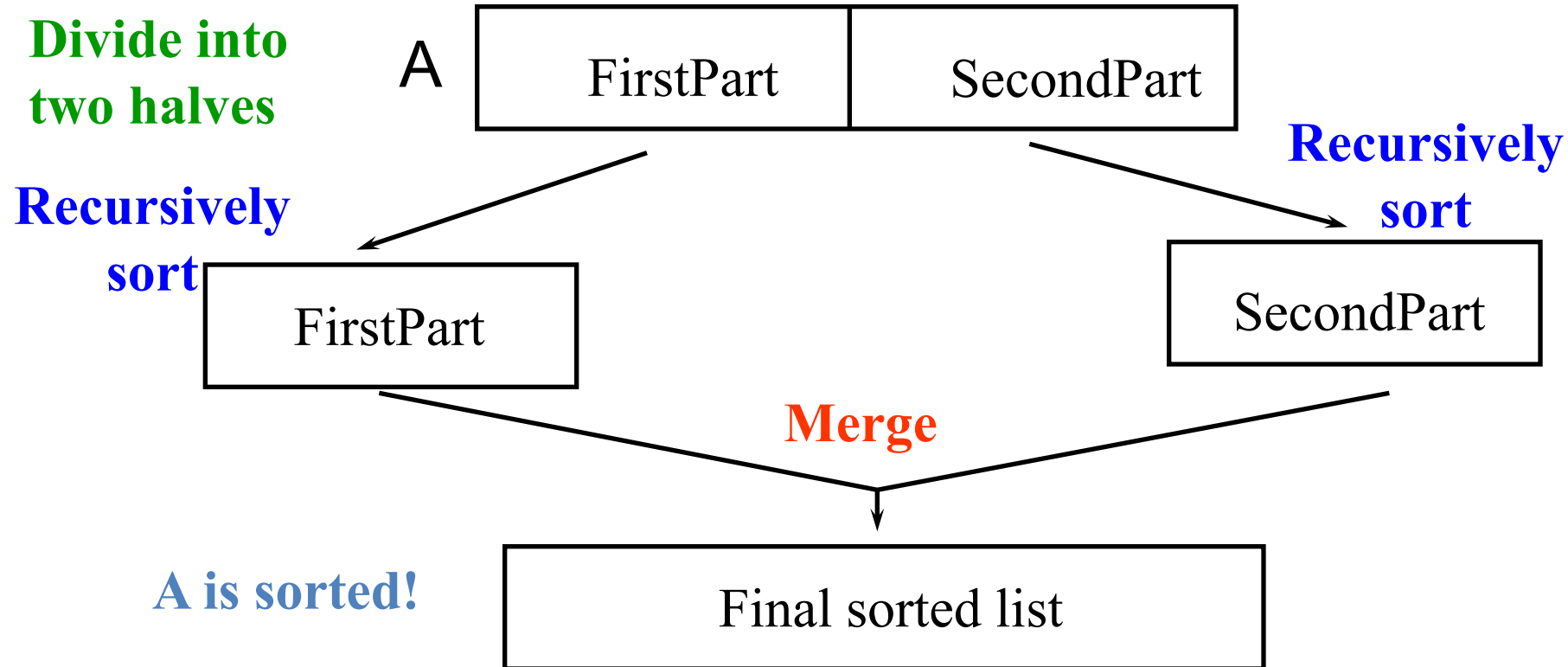
When the conquer step reaches the base step and we get two sorted subarrays $A[p..q]$ and $A[q+1, r]$ for array $A[p..r]$, we combine the results by creating a sorted array $A[p..r]$ from two sorted subarrays $A[p..q]$ and $A[q+1, r]$

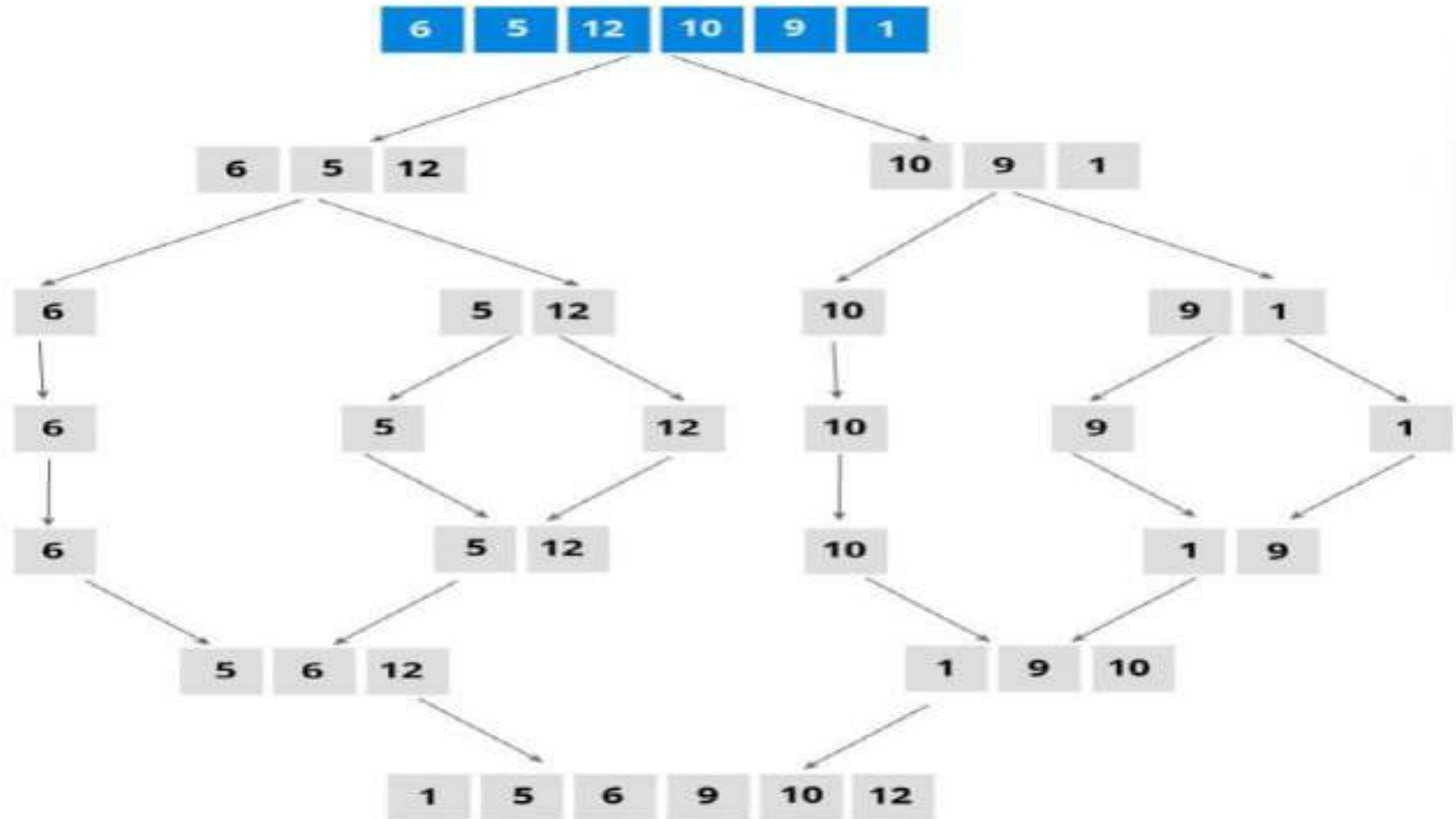
While comparing two sublists for merging, the first element of both lists is taken into consideration.

While sorting in ascending order, the element that is of a lesser value becomes a new element of the sorted list. This procedure is repeated until both the smaller sublists are empty and the new combined sublist comprises all the elements of both the sublists

MERGE SORT:

IDEA





Merge Sort: Algorithm

Algorithm **MergeSort** (low, high)

// a[low: high] is a global array to be sorted.

// Small(P) is true if there is only one element to sort. In this case the list is already sorted.

{ **if (low < high) then** // if there are more than one element

{

Recursive Calls

// Divide P into sub problems.

// Find where to split the set.

mid := [(low+high)/2];

//solve the sub problems.

MergeSort(low,mid);

MergeSort(mid+1, high);

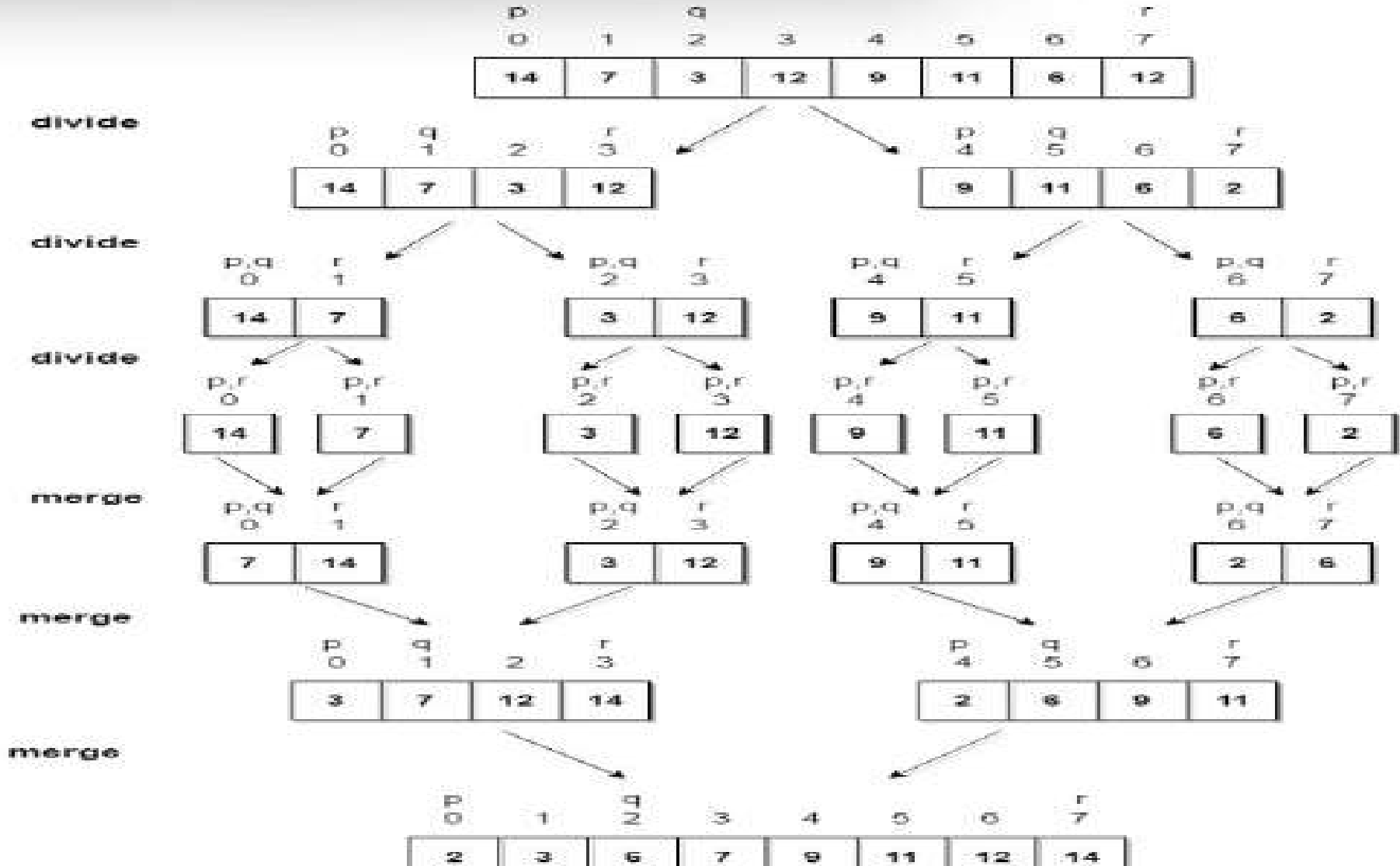
// Combine the solutions.

Merge(low, mid, high);

}

}

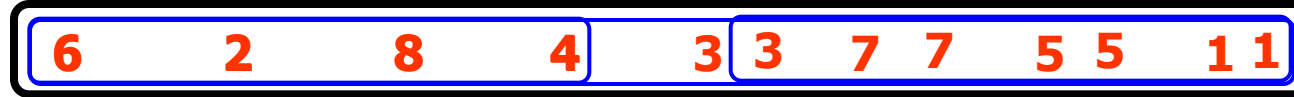
- The merge function works as follows:
 1. Create copies of the subarrays $L \leftarrow A[p..q]$ and $R \leftarrow A[q+1..r]$.
 2. Create three pointers i, j and k
 1. i maintains current index of L , starting at 1
 2. j maintains current index of R , starting at 1
 3. k maintains current index of $A[p..q]$, starting at p
 3. Until we reach the end of either L or R , pick the larger among the elements from L and R and place them in the correct position at $A[p..q]$
 4. When we run out of elements in either L or R , pick up the remaining elements and put in $A[p..q]$



Merge-Sort(A, 0, 7)

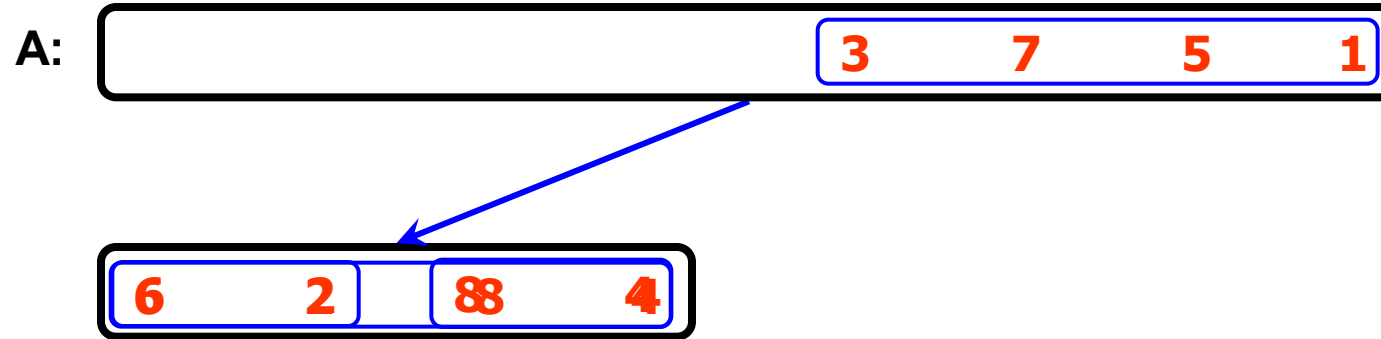
Divide

A:



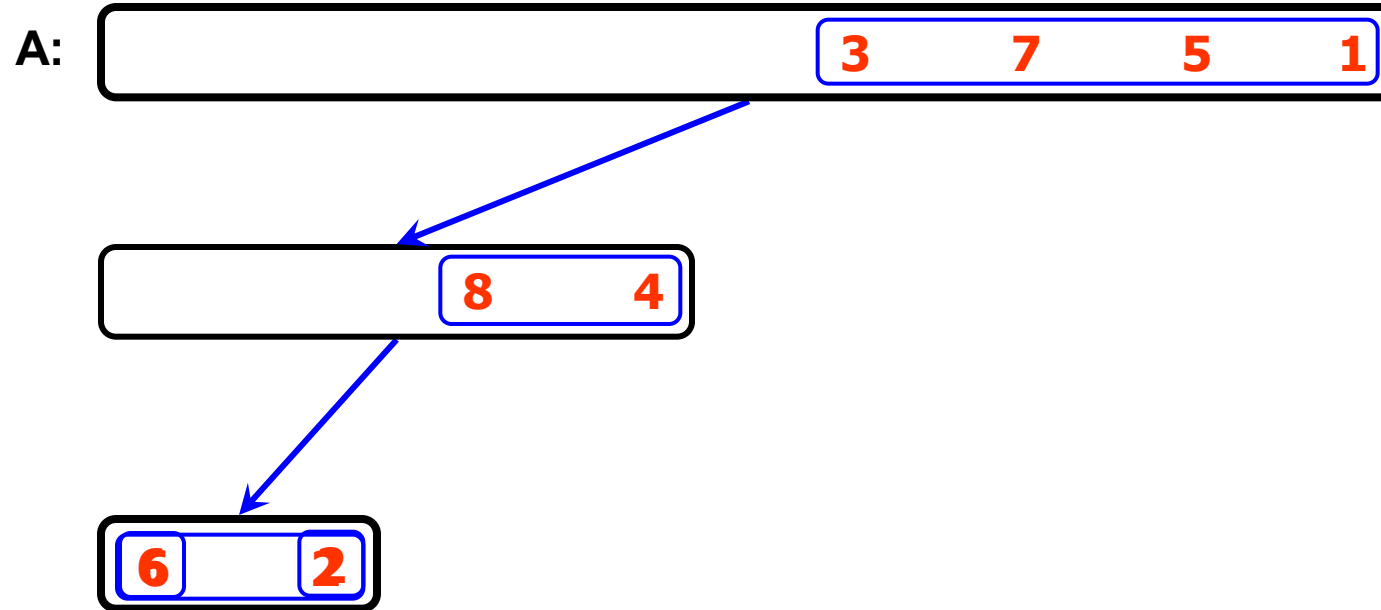
Merge-Sort(A, 0, 7)

Merge-Sort(A, 0, 3) , divide



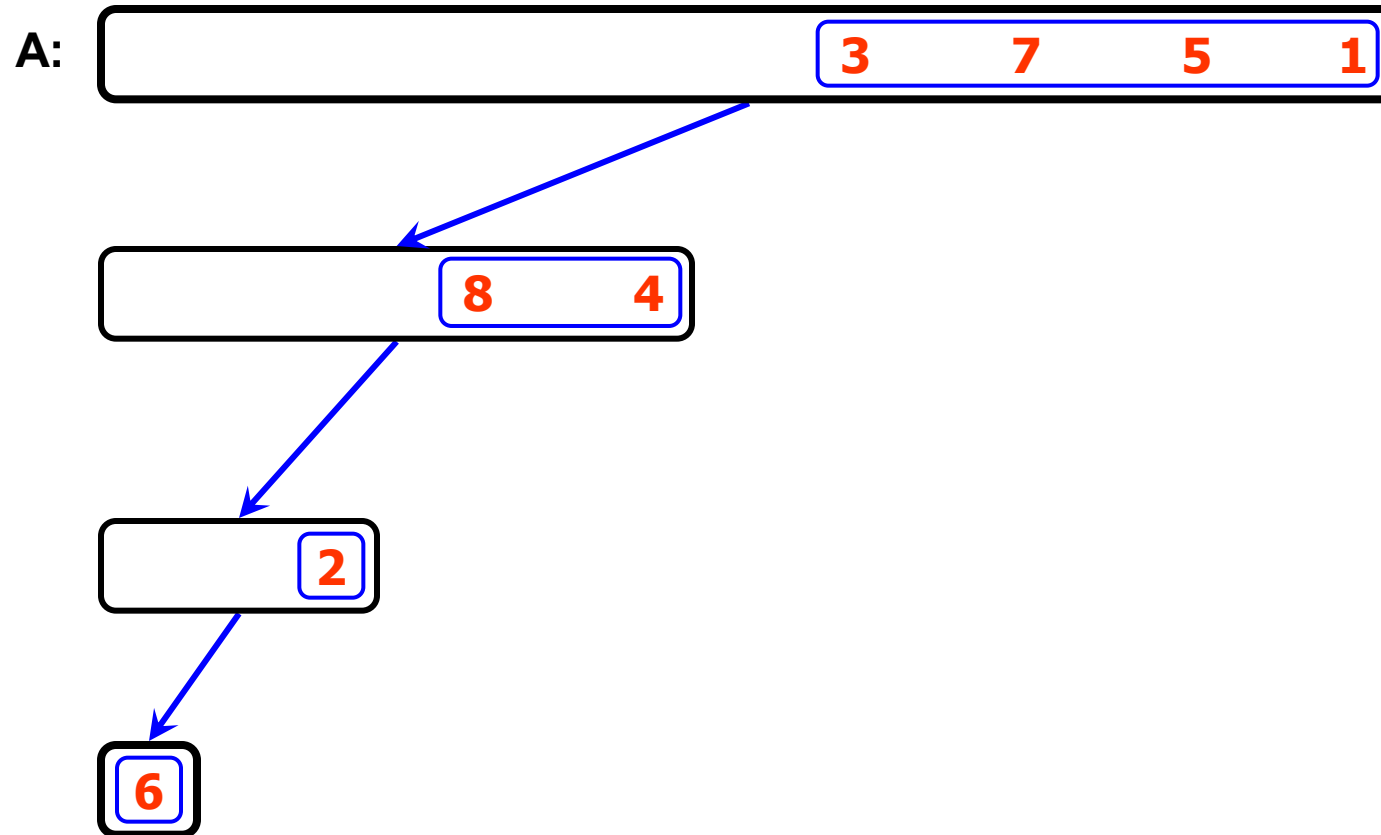
Merge-Sort(A, 0, 7)

Merge-Sort(A, 0, 1) , divide



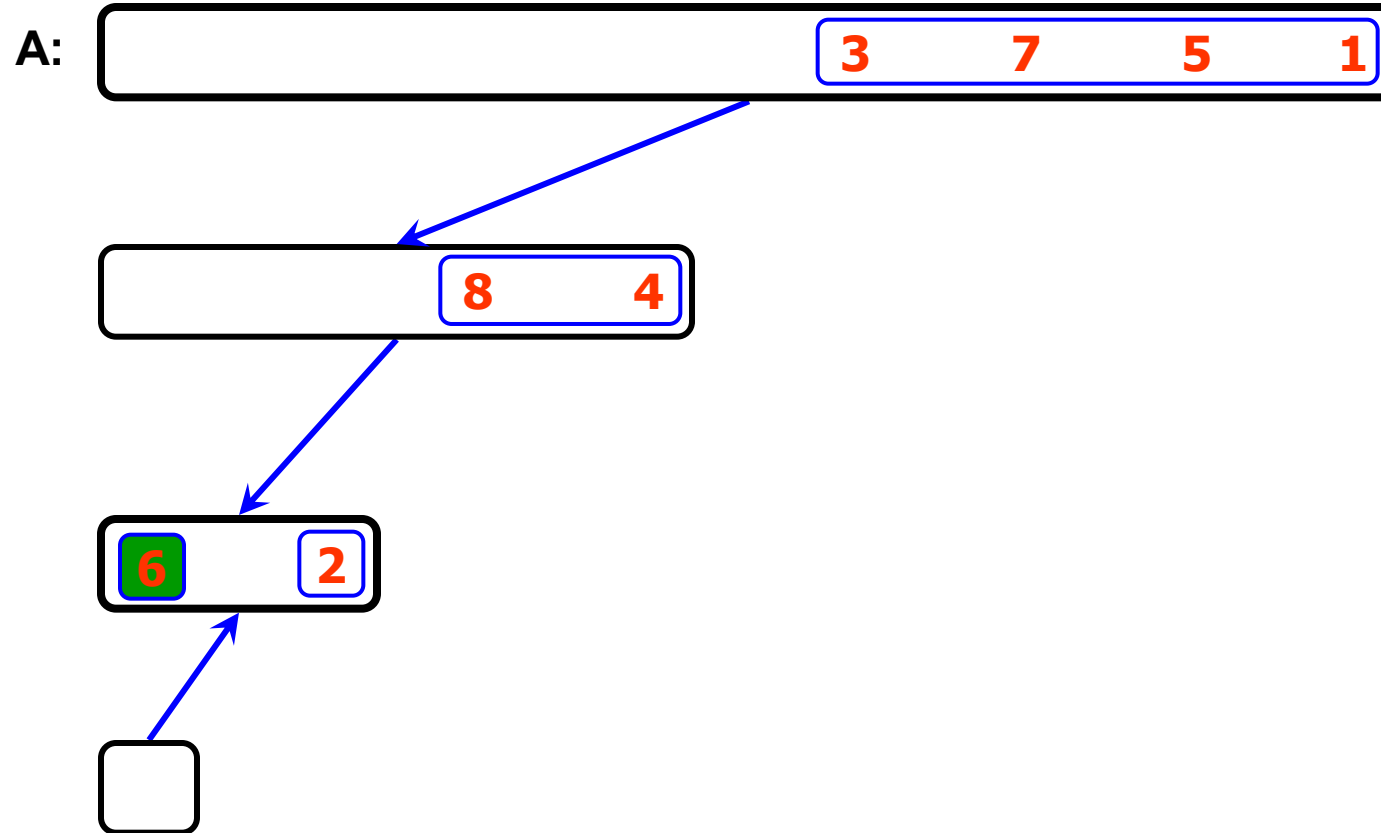
Merge-Sort(A, 0, 7)

Merge-Sort(A, 0, 0) , base case



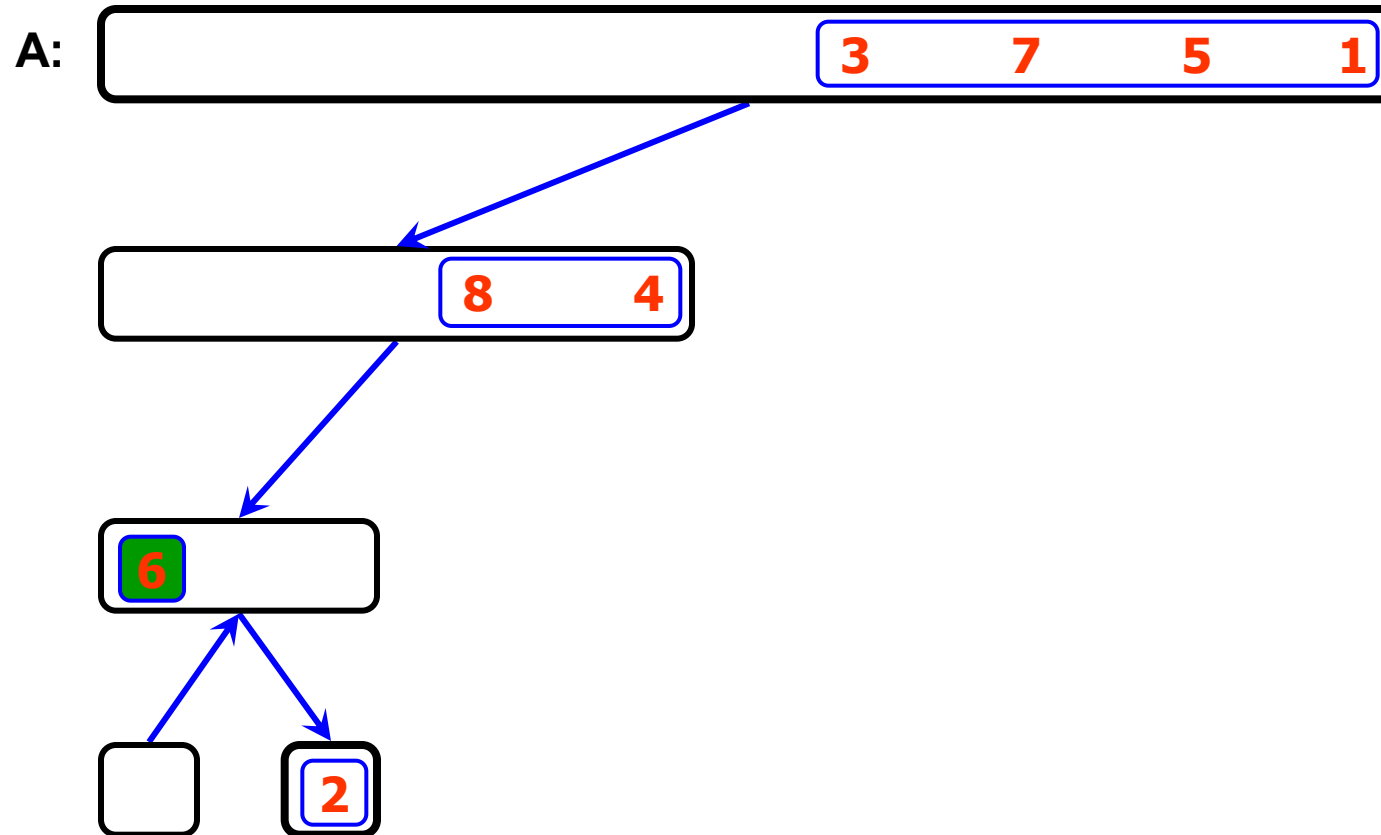
Merge-Sort(A, 0, 7)

Merge-Sort(A, 0, 0), return



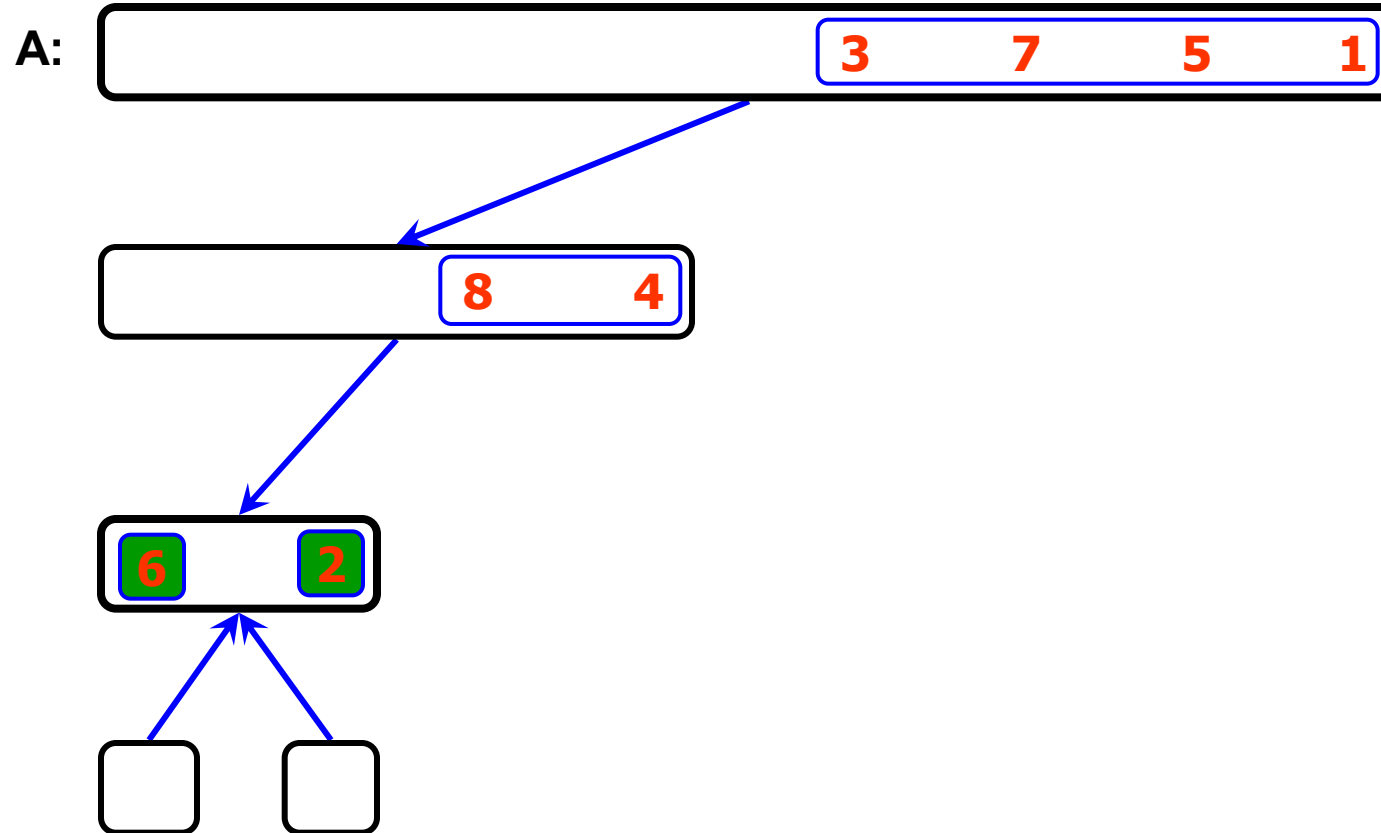
Merge-Sort(A, 0, 7)

Merge-Sort(A, 1, 1) , base case



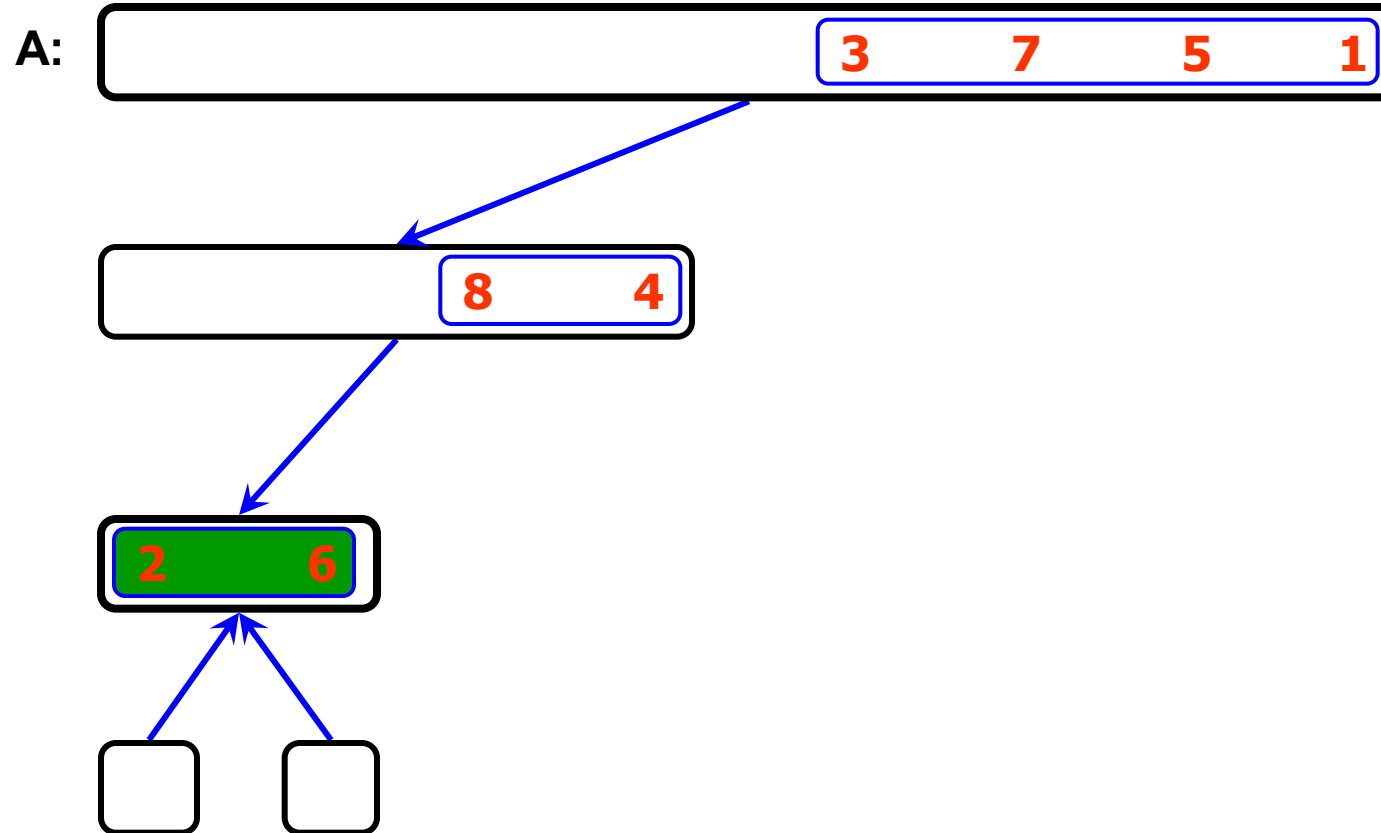
Merge-Sort(A, 0, 7)

Merge-Sort(A, 1, 1), return



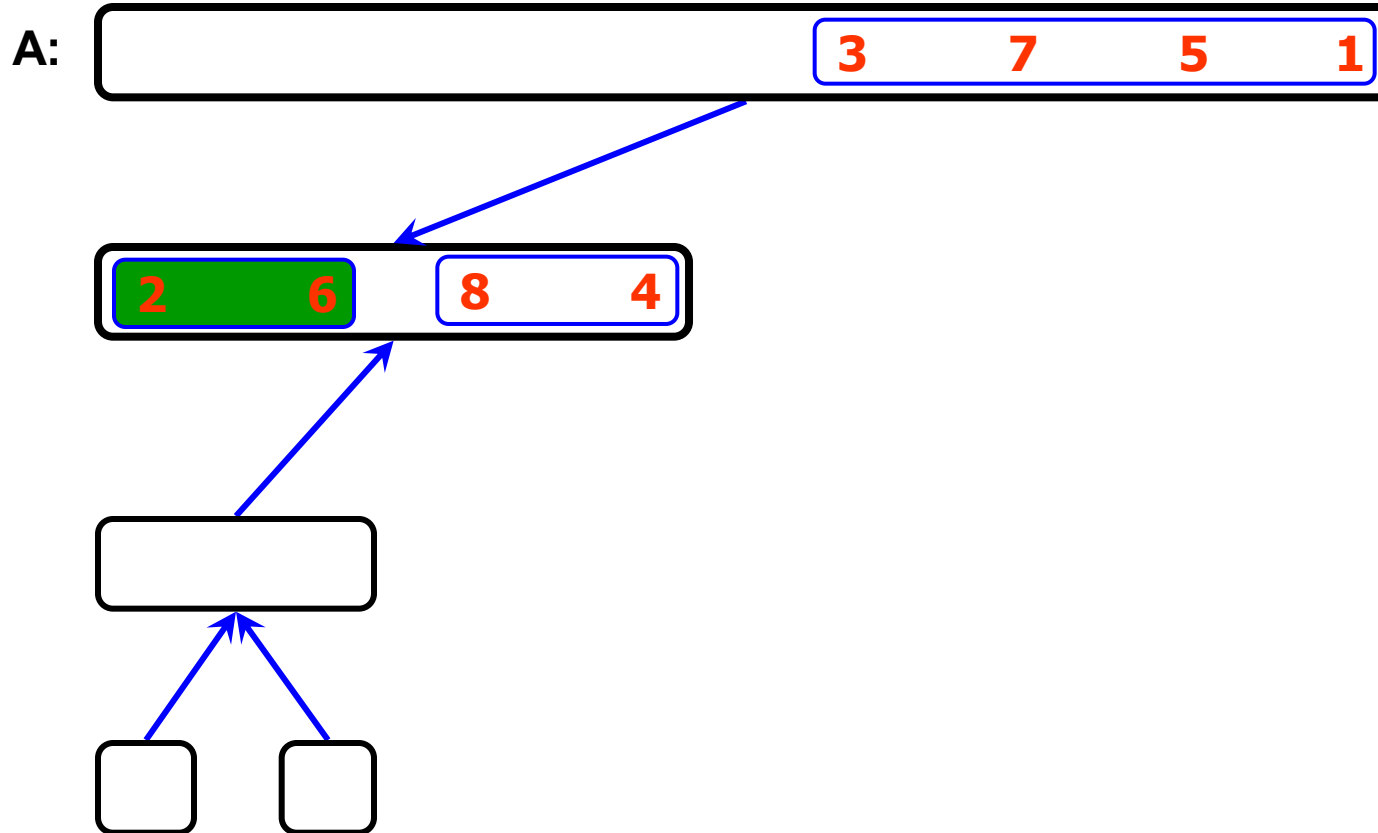
Merge-Sort(A, 0, 7)

Merge(A, 0, 0, 1)



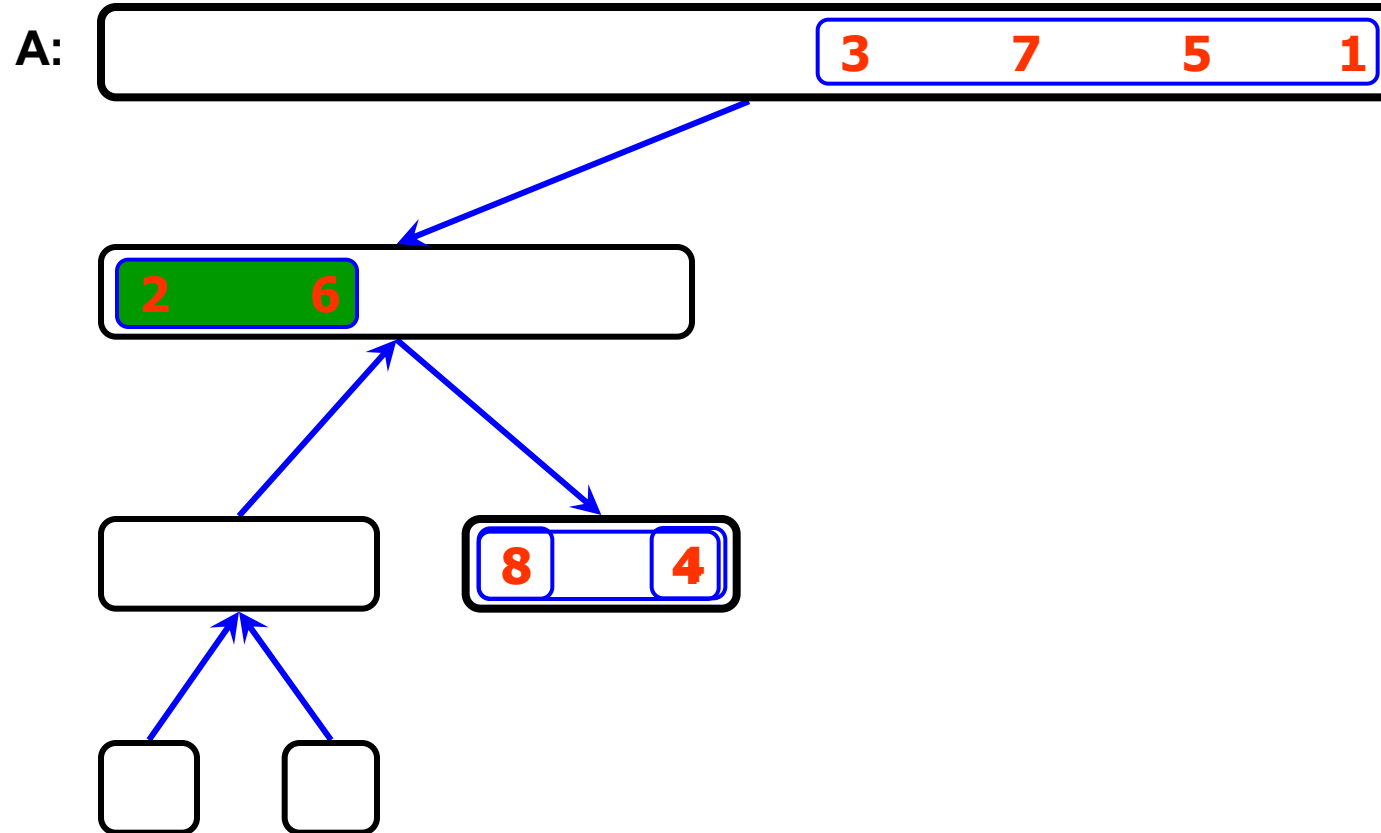
Merge-Sort(A, 0, 7)

Merge-Sort(A, 0, 1), return



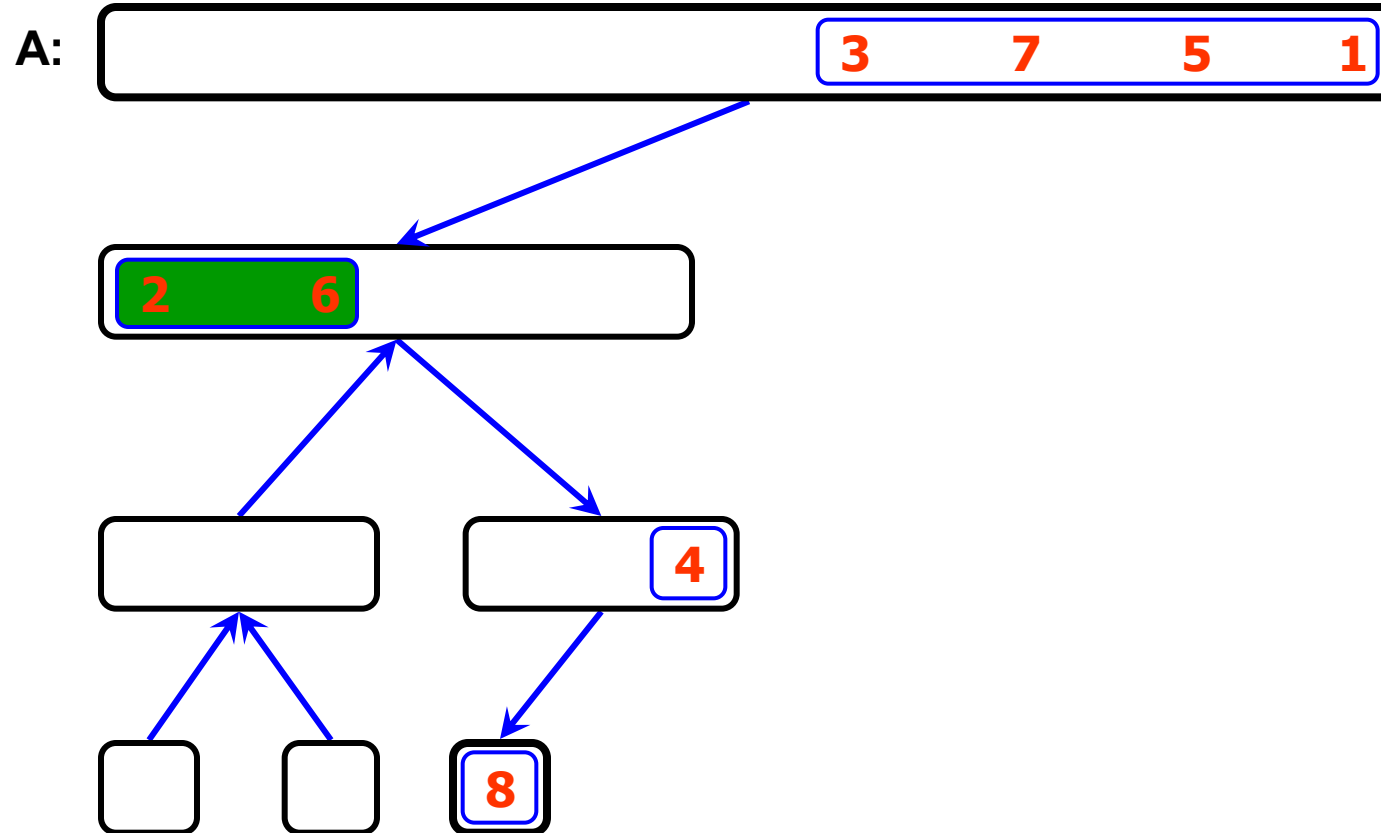
Merge-Sort(A, 0, 7)

Merge-Sort(A, 2, 3) , divide



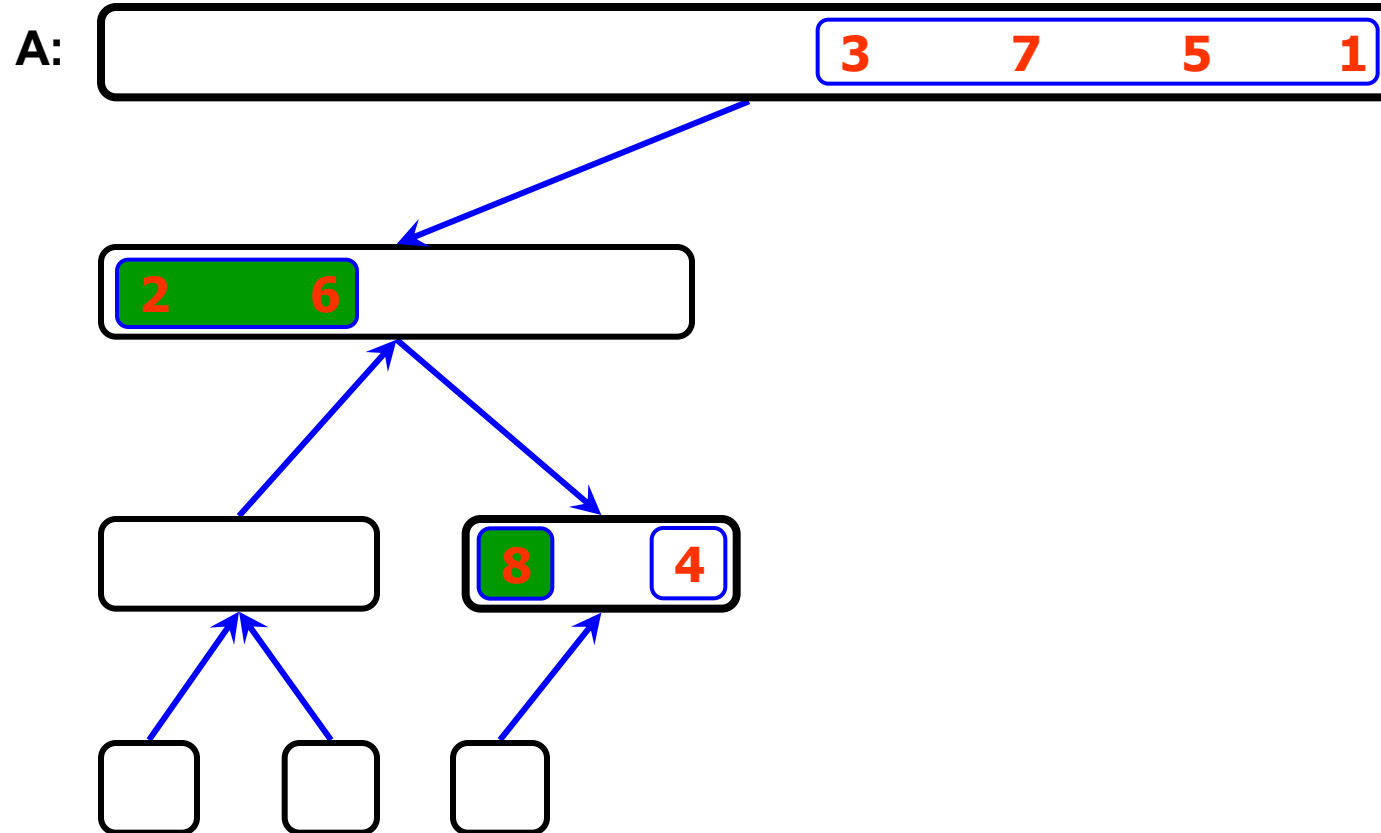
Merge-Sort(A, 0, 7)

Merge-Sort(A, 2, 2), base case



Merge-Sort(A, 0, 7)

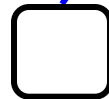
Merge-Sort(A, 2, 2), return



Merge-Sort(A, 0, 7)

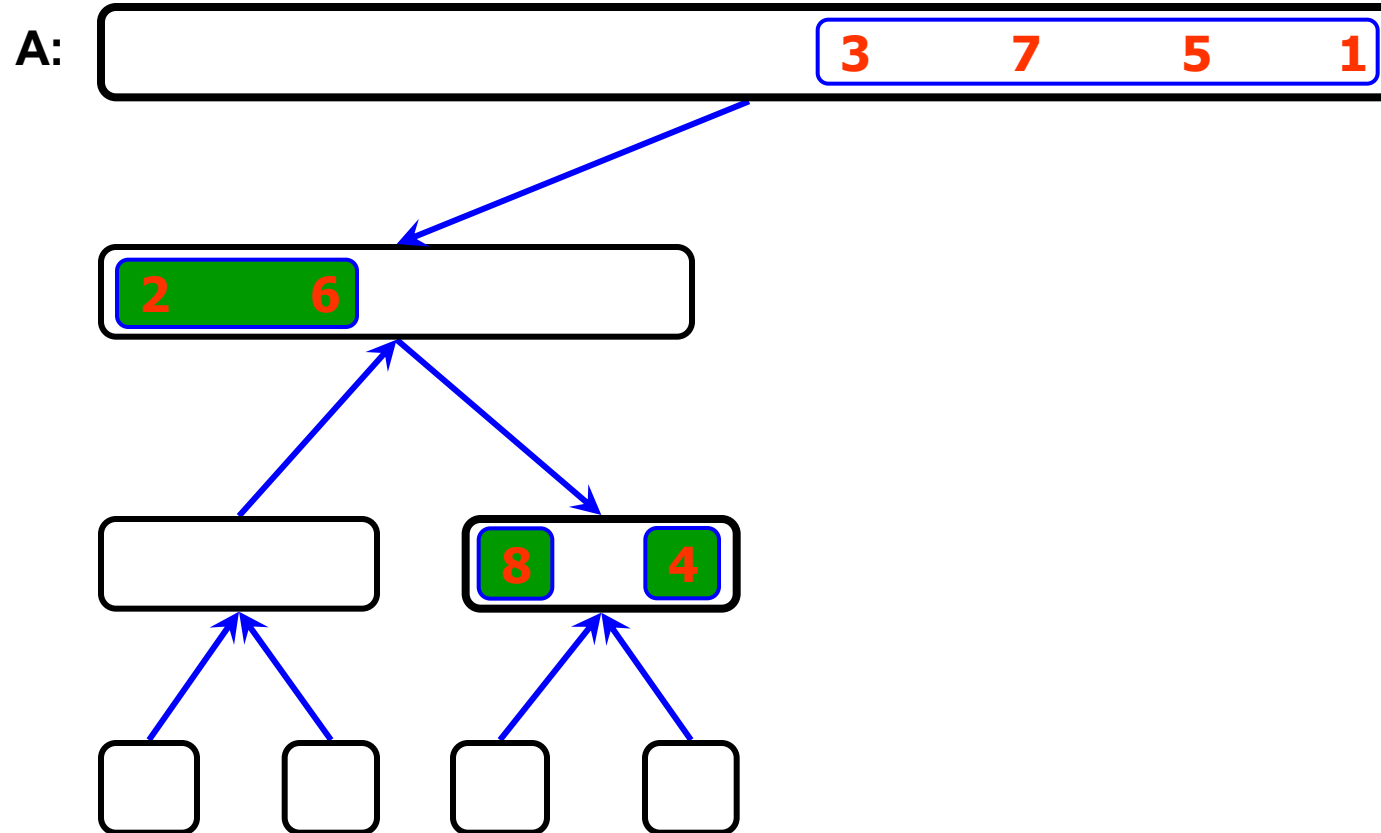
Merge-Sort(A, 3, 3), base case

A:



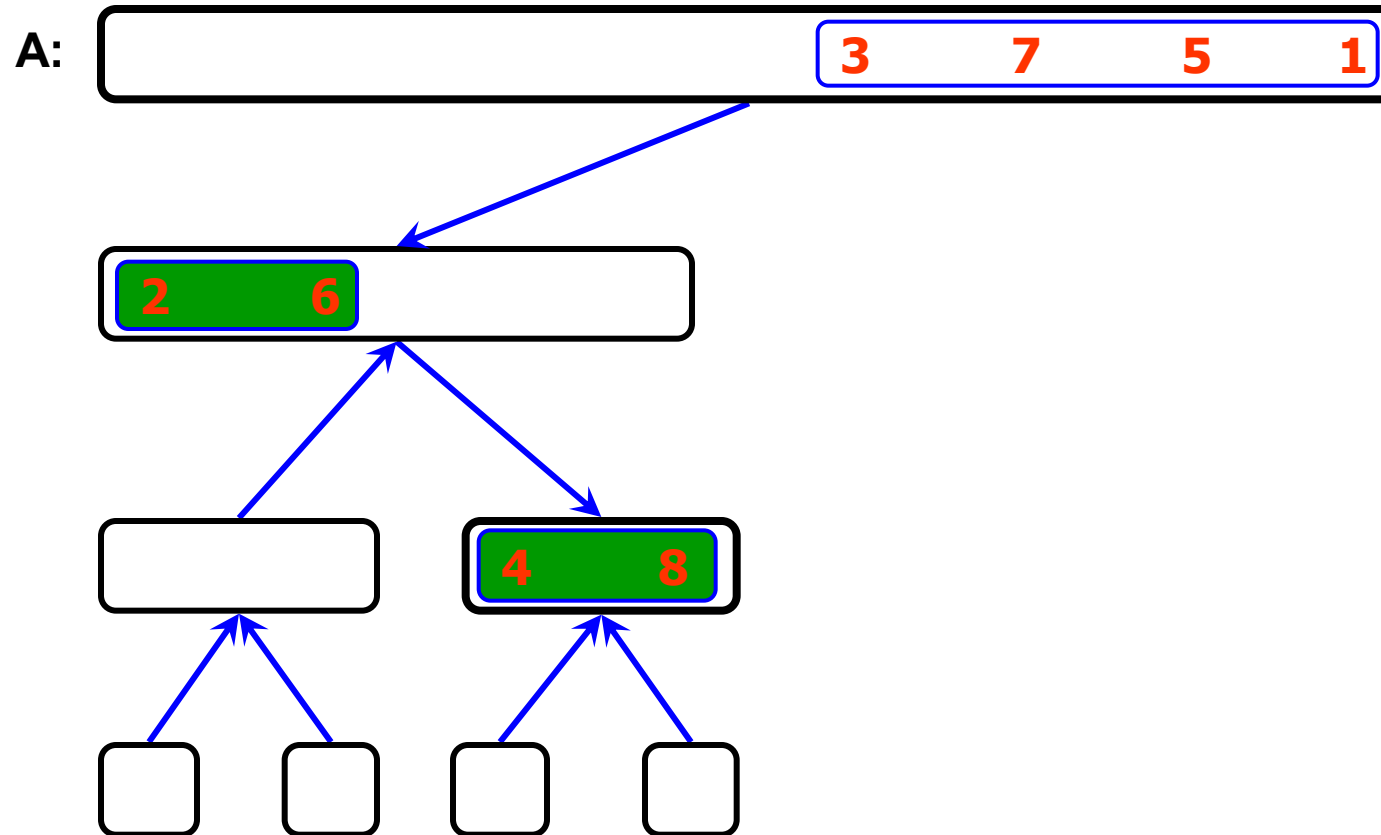
Merge-Sort(A, 0, 7)

Merge-Sort(A, 3, 3), return



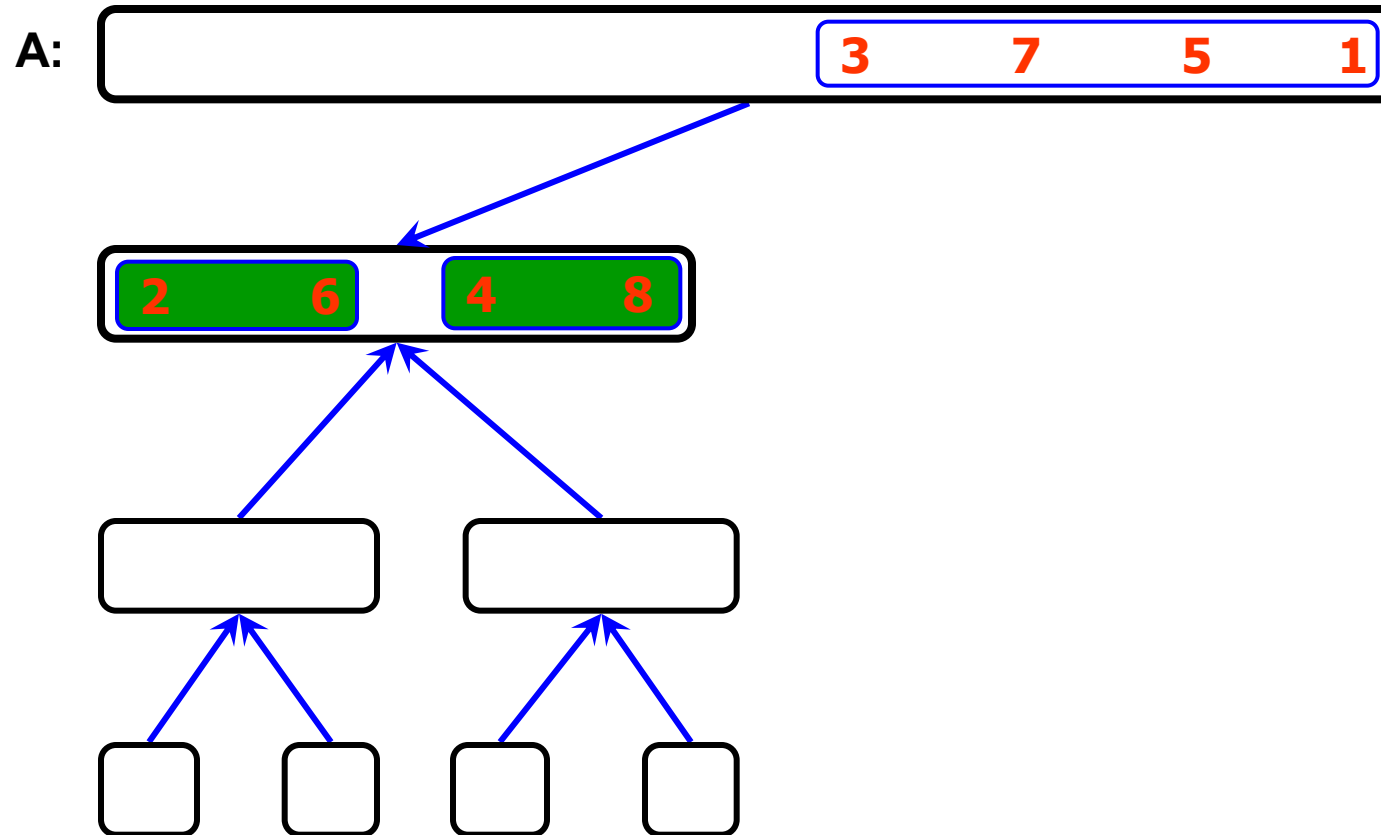
Merge-Sort(A, 0, 7)

Merge(A, 2, 2, 3)



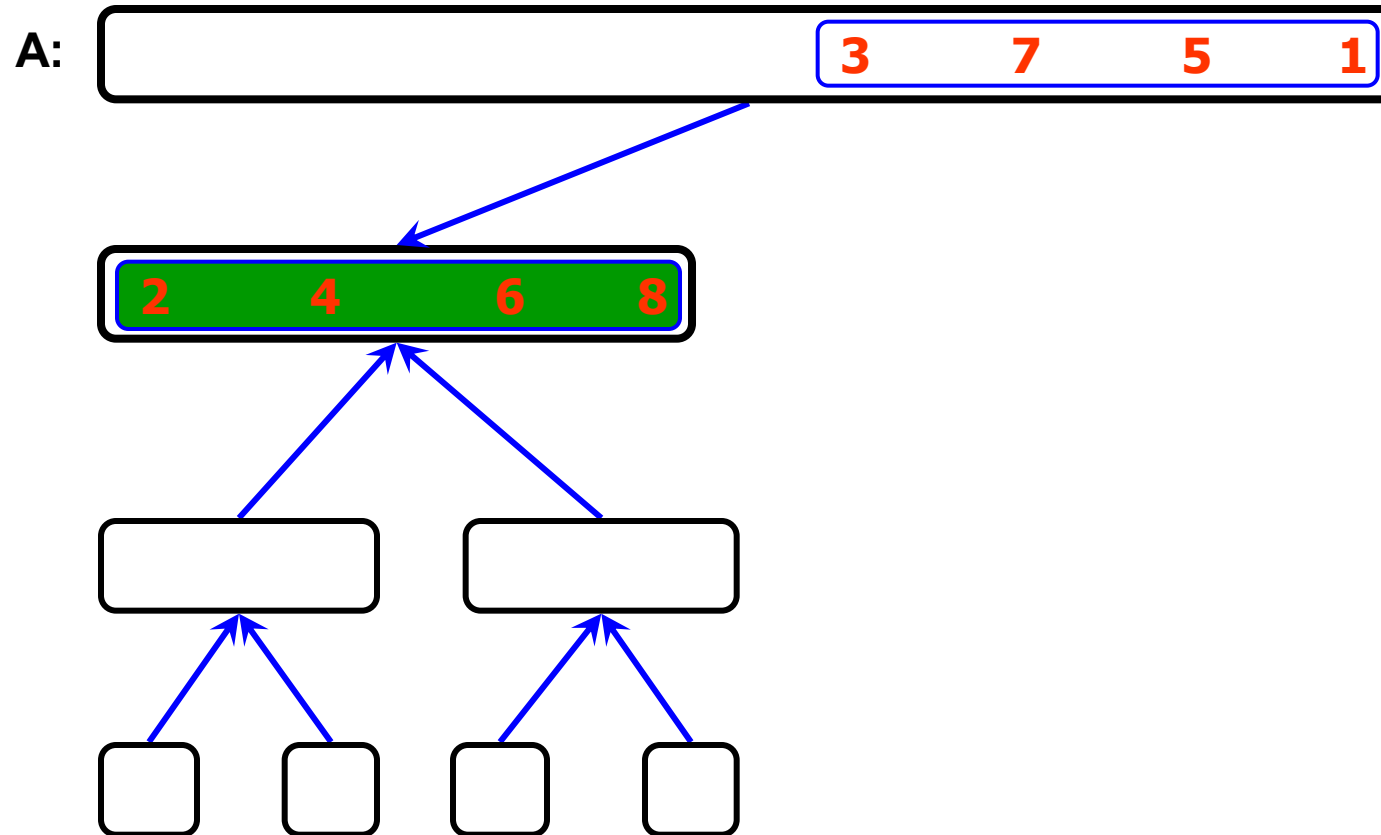
Merge-Sort(A, 0, 7)

Merge-Sort(A, 2, 3), return



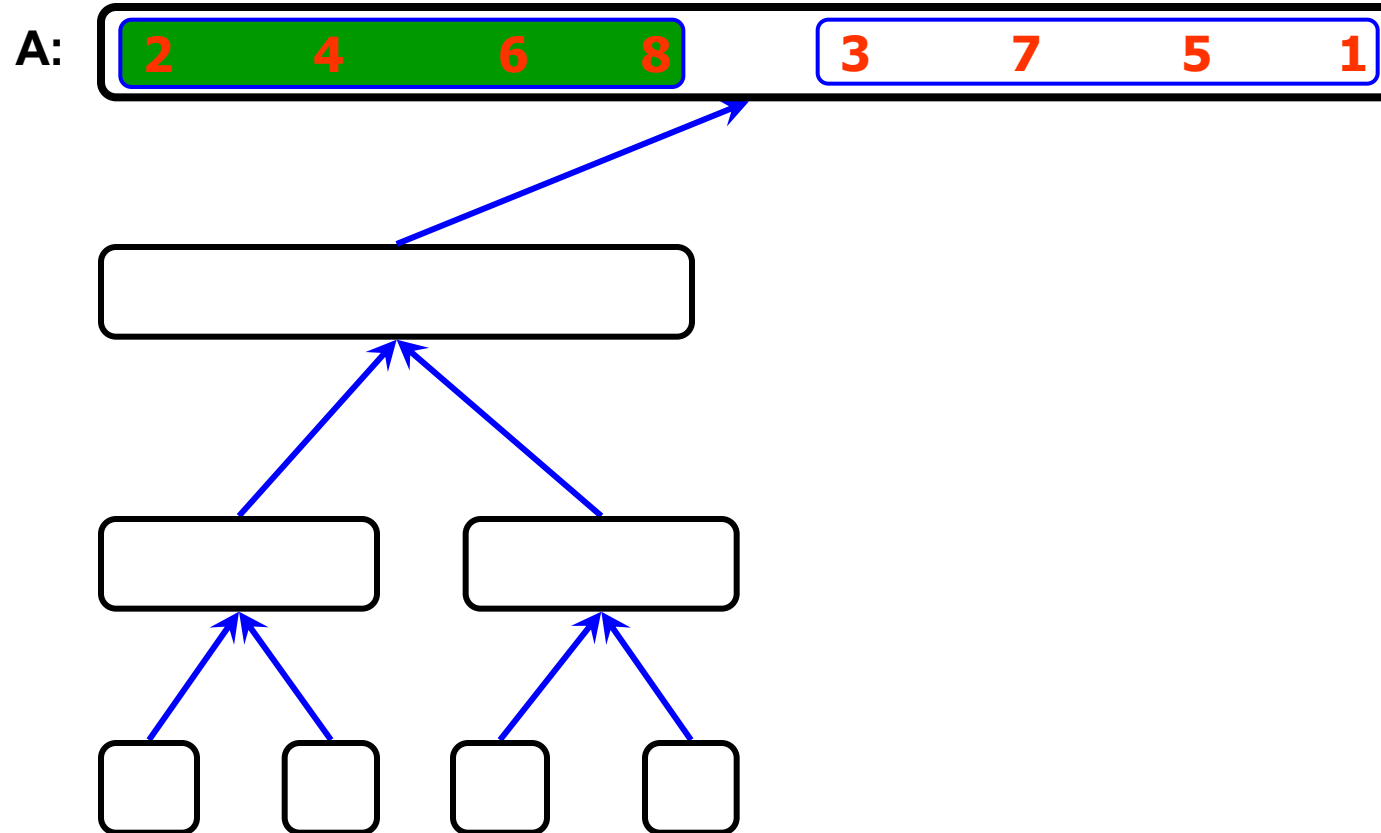
Merge-Sort(A, 0, 7)

Merge(A, 0, 1, 3)



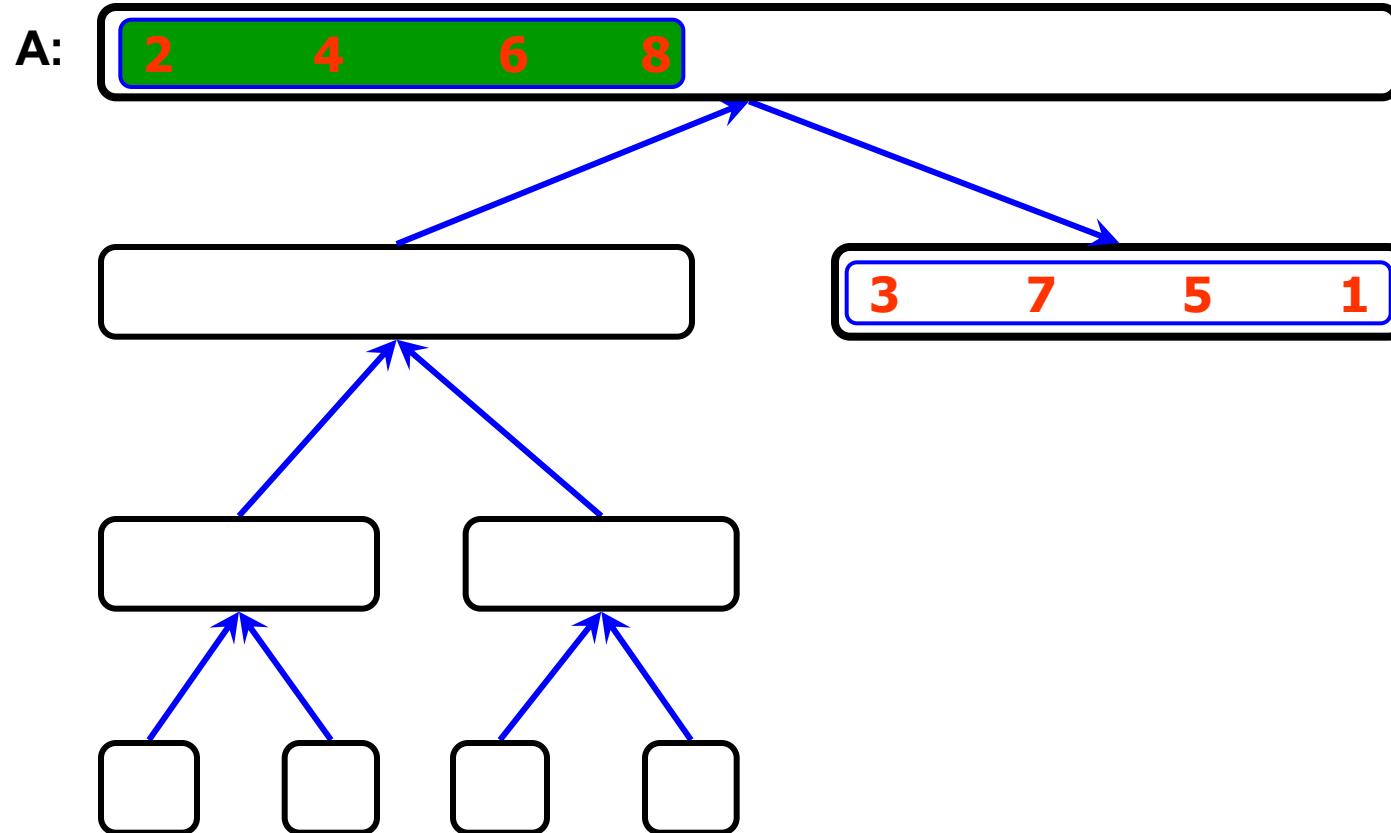
Merge-Sort(A, 0, 7)

Merge-Sort(A, 0, 3), return



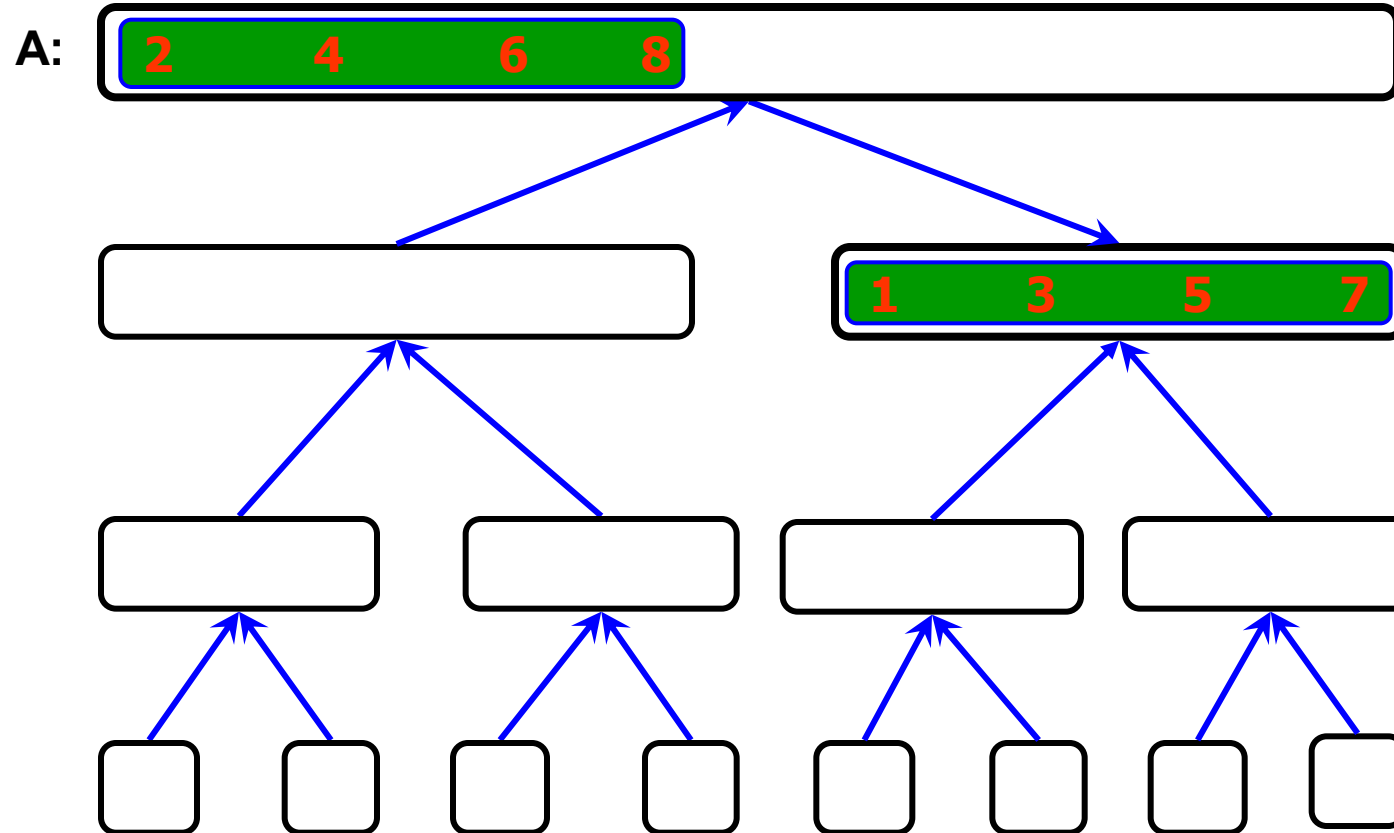
Merge-Sort(A, 0, 7)

Merge-Sort(A, 4, 7)



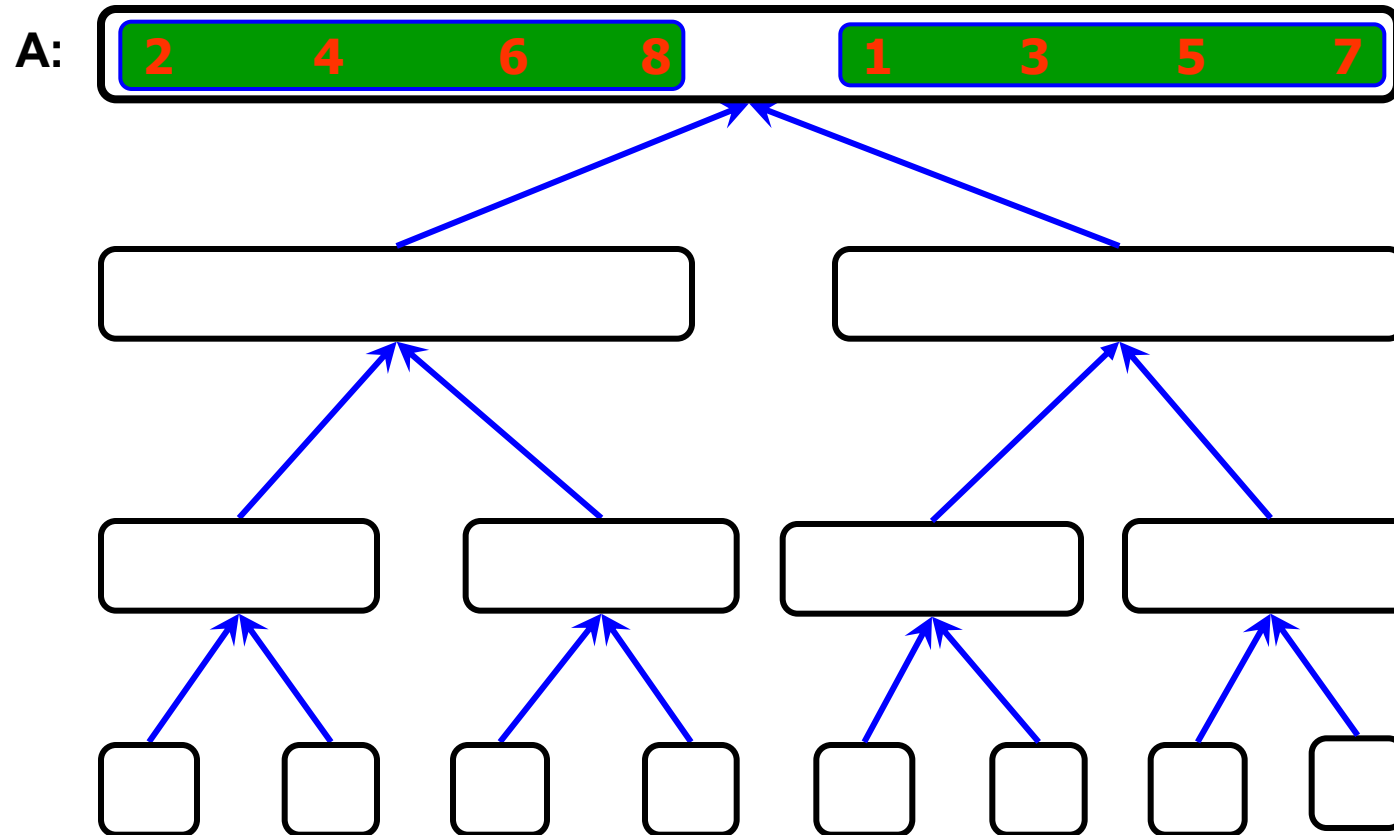
Merge-Sort(A, 0, 7)

Merge (A, 4, 5, 7)



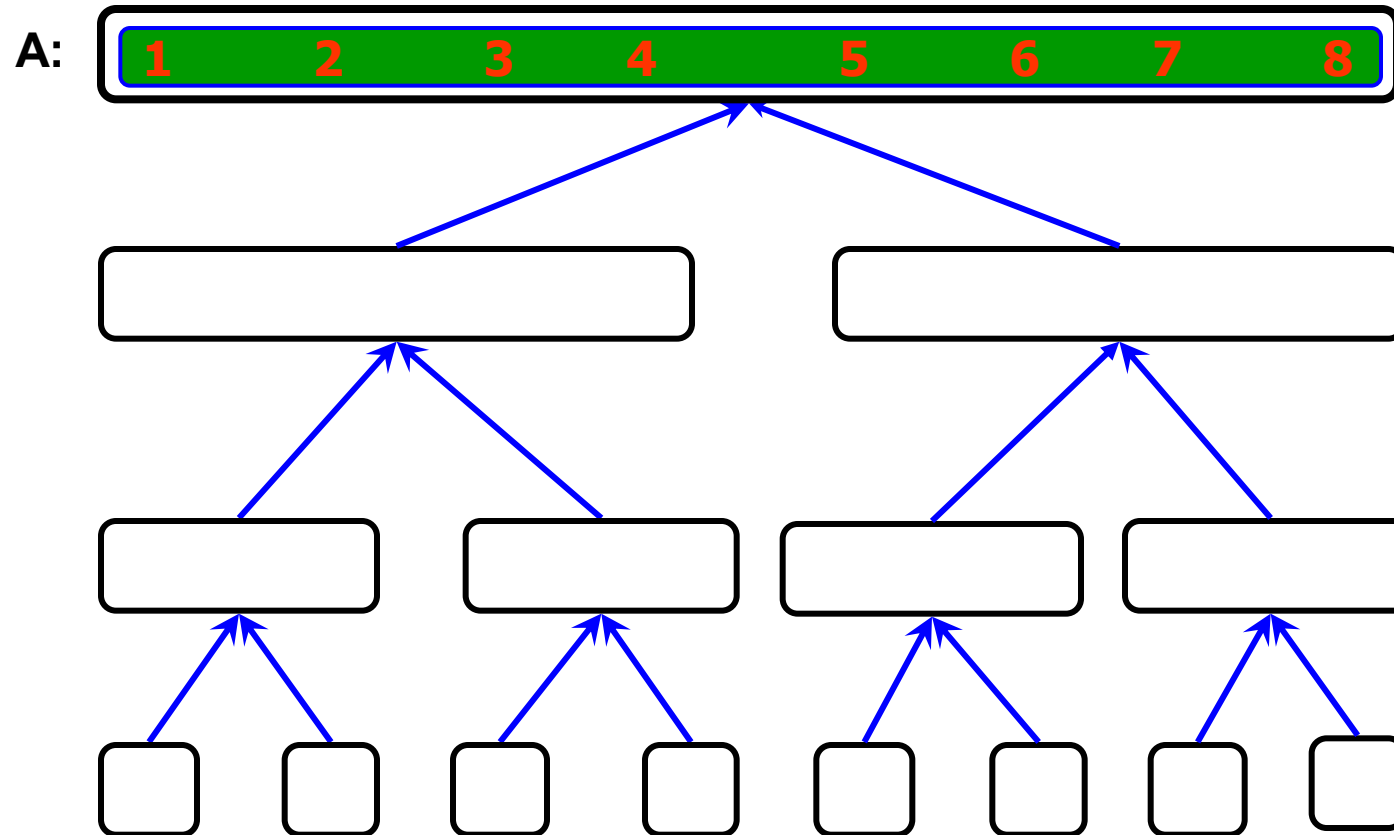
Merge-Sort(A, 0, 7)

Merge-Sort(A, 4, 7), return



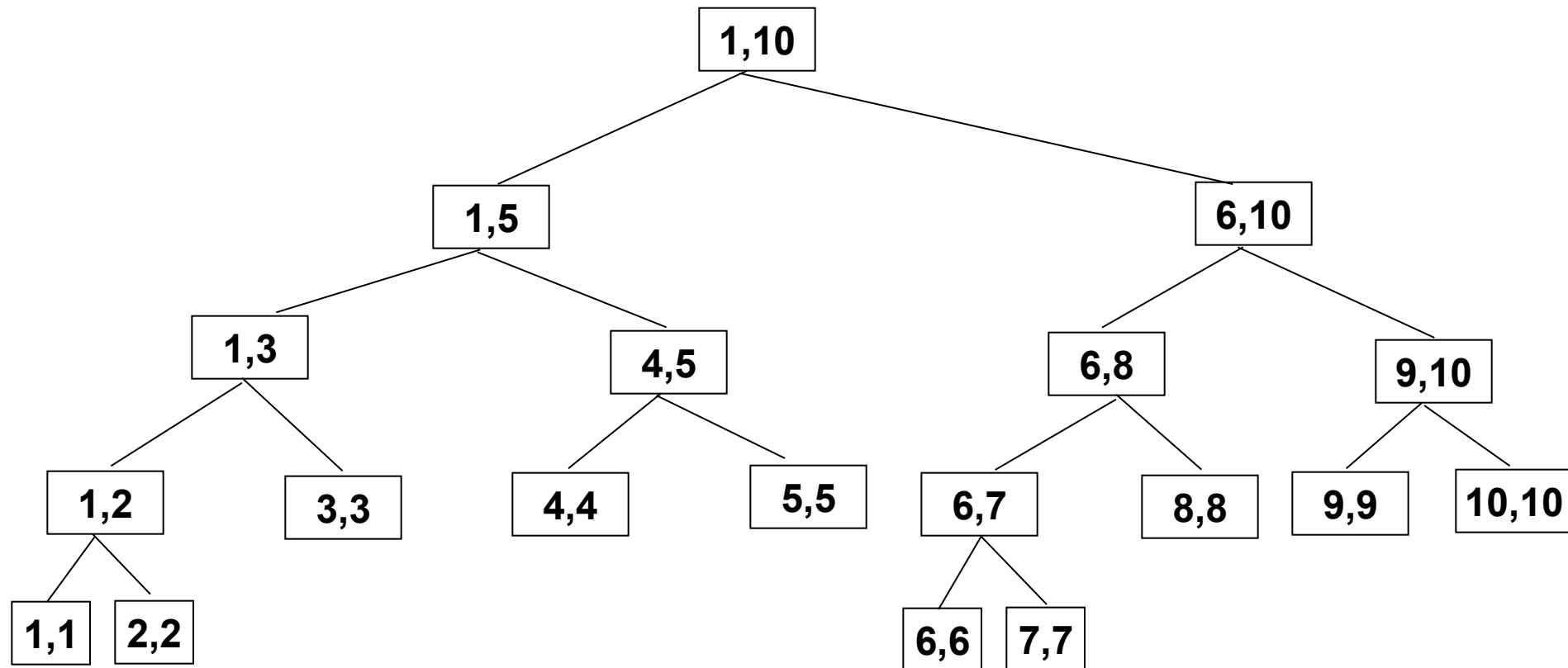
Merge-Sort(A, 0, 7)

Merge-Sort(A, 0, 7), done!



Ex:- [179, 254, 285, 310, 351, 423, 450, 520, 652, 861]

Tree of calls of merge sort



Merge-Sort: Merge Example

	low			mid		high		
A:	2	3	7	8	1	4	5	6

B:							
-----------	--	--	--	--	--	--	--

L:				
-----------	--	--	--	--

R:				
-----------	--	--	--	--

Merge-Sort: Merge Example

A:

--	--	--	--	--	--	--	--

B:

1							
---	--	--	--	--	--	--	--


k=low

L:

2	3	7	8
---	---	---	---


i=low

R:

1	4	5	6
---	---	---	---


j=mid+1

Merge-Sort: Merge Example

A:

--	--	--	--	--	--	--	--

B:

1	2						
---	---	--	--	--	--	--	--

↑
k

L:

2	3	7	8
---	---	---	---

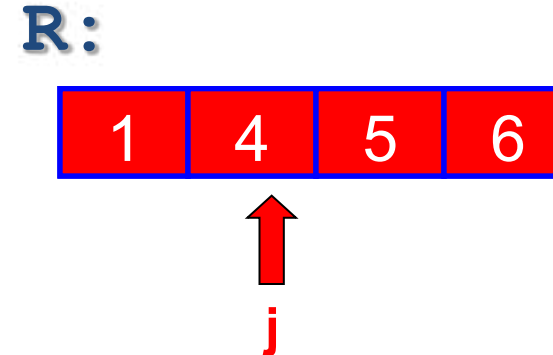
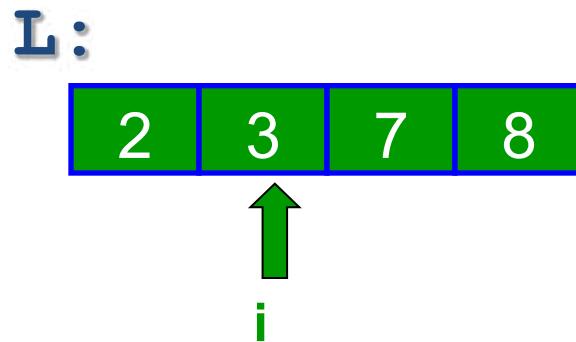
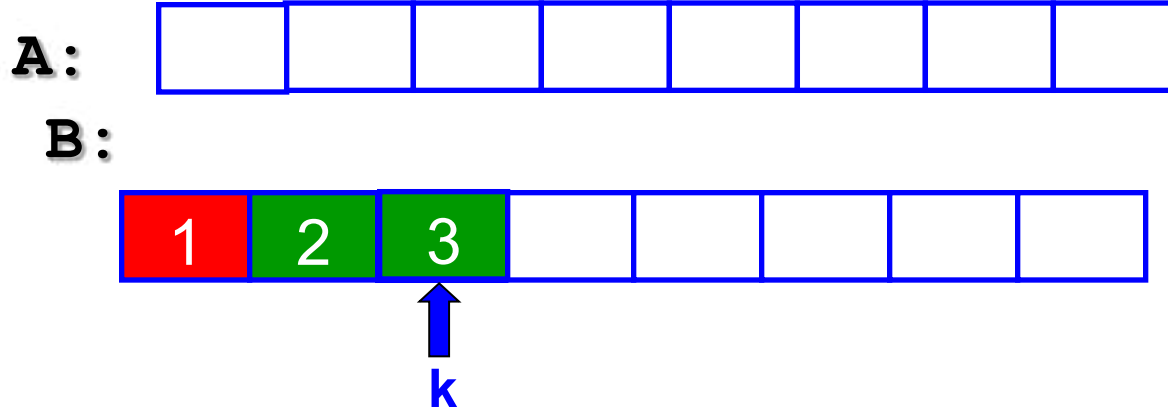
↑
i

R:

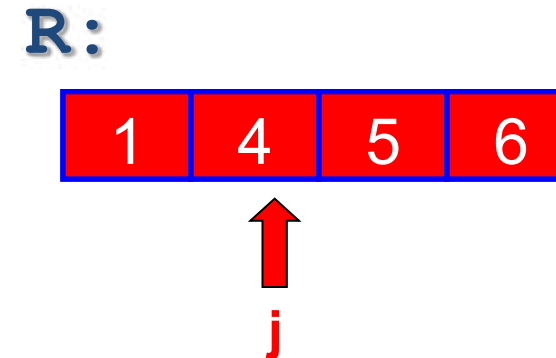
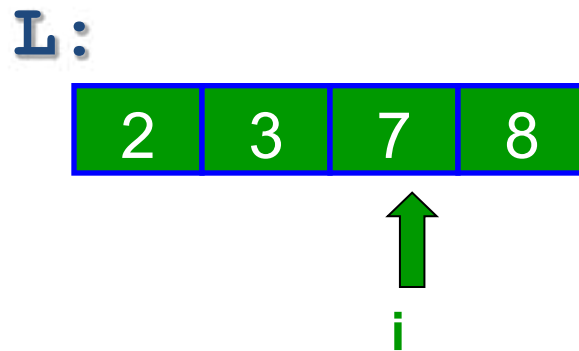
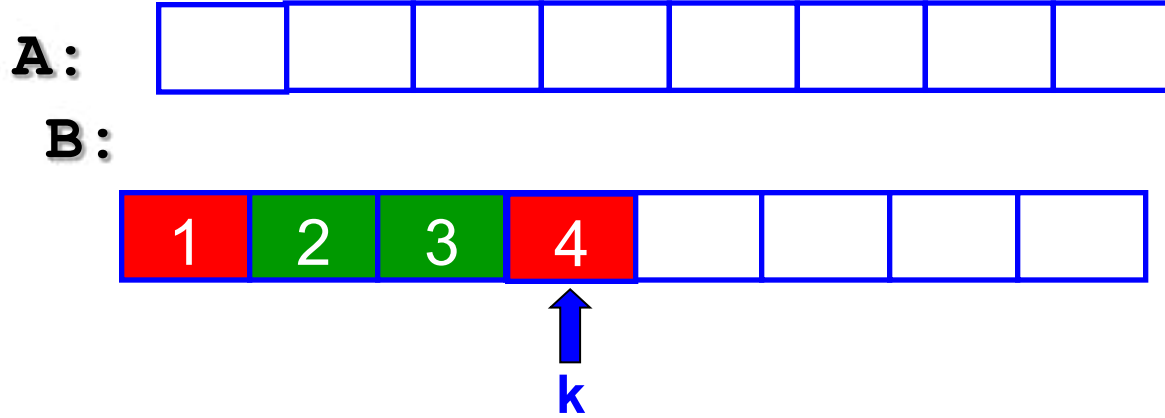
1	4	5	6
---	---	---	---

↑
j

Merge-Sort: Merge Example



Merge-Sort: Merge Example



Merge-Sort: Merge Example

A:

--	--	--	--	--	--	--	--

B:

1	2	3	4	5			
---	---	---	---	---	--	--	--

↑
k

L:

2	3	7	8
---	---	---	---

↑
i

R:

1	4	5	6
---	---	---	---

↑
j

Merge-Sort: Merge Example

A:

--	--	--	--	--	--	--	--

B:

1	2	3	4	5	6		
---	---	---	---	---	---	--	--

↑
k

L:

2	3	7	8
---	---	---	---

↑
i

R:

1	4	5	6
---	---	---	---

↑
j

Merge-Sort: Merge Example

A:

--	--	--	--	--	--	--	--

B:

1	2	3	4	5	6	7	
---	---	---	---	---	---	---	--

↑
k

L:

2	3	7	8
---	---	---	---

↑
i

R:

1	4	5	6
---	---	---	---

↑
j

Merge-Sort: Merge Example

A:

--	--	--	--	--	--	--	--

B:

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

↑
k

L:

2	3	7	8
---	---	---	---

↑
i

R:

1	4	5	6
---	---	---	---

↑
j

Merge-Sort: Merge Example

A:

--	--	--	--	--	--	--	--

B:

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

↑
k

L:

2	3	7	8
---	---	---	---

↑
i

R:

1	4	5	6
---	---	---	---

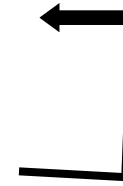
↑
j

A:

--	--	--	--	--	--	--	--

B:

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---



Merge Sort Analysis

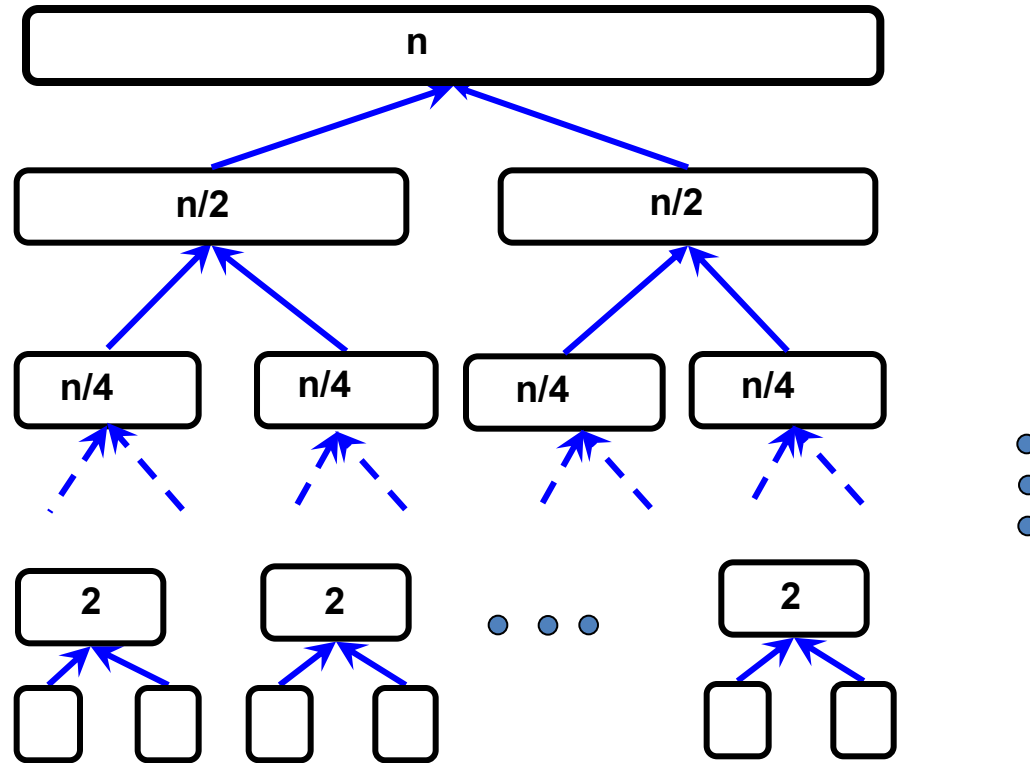
If the time for the merging operations is proportional to 'n', then the computing time for merge sort is described by the recurrence relation.

$$T(n) = \begin{cases} a & n=1, 'a' \text{ a constant} \\ 2T(n/2)+cn & n>1, 'c' \text{ a constant.} \end{cases}$$

Mergesort Analysis

- Let $T(N)$ be the running time for an array of N elements
- Mergesort divides array in half and calls itself on the two halves. After returning, it merges both halves using a temporary array
- Each recursive call takes $T(N/2)$ and merging takes cn

Best/good Case



- Total time: $O(n \log n)$

Best Case Time Complexity

When n is a power of 2, $n = 2^k$, we can solve this equation by successive substitutions:

$$\begin{aligned} T(n) &= 2(2T(n/4) + cn/2) + cn \\ &= 4T(n/4) + 2cn \\ &= 4(2T(n/8) + cn/4) + 2cn \\ &\vdots \\ &= 2^k T(1) + kcn \\ &= an + cn \log n \end{aligned}$$

It is easy to see that if $2^k < n \leq 2^{k+1}$, then $T(n) \leq T(2^{k+1})$. Therefore

$$T(n) = O(n \log n)$$

- **Divide and Conquer Strategy:** Breaks down a problem into smaller sub-problems, solves each sub-problem independently, and combines their solutions for the final result.
- **Recursive Process:** Continuously splits the input into smaller sections until the base case (often a single element) is reached.
- **Merge Sort Example:** Uses divide and conquer to split an array into two halves recursively.
- **Merging:** Combines the sorted sub-arrays into a single sorted array.

SELF-ASSESSMENT QUESTIONS

What is the average case time complexity of merge sort?

- (a) $O(n \log n)$
- (b) $O(n^2)$
- (c) $O(n^2 \log n)$
- 1. $O(n \log n^2)$

The divide and conquer approach is most suitable for problems that can be:

- (a) Broken down into non-overlapping sub-problems.
- (b) Solved in a linear fashion.
- (c) Solved without recursion.
- (d) Solved iteratively.

TERMINAL QUESTIONS

1. Explain how the divide and conquer strategy is used in merge sort. What are the main steps involved?
2. Explain how the divide and conquer approach can be applied to solving problems other than sorting. Provide at least two examples.

REFERENCES FOR FURTHER LEARNING OF THE SESSION

Reference Books :

1. Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein., 3rd, 2009, The MIT Press.
- 2 Algorithm Design Manual, Steven S. Skiena., 2nd, 2008, Springer.
- 3 Data Structures and Algorithms in Python, Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser., 2nd, 2013, Wiley.
- 4 The Art of Computer Programming, Donald E. Knuth, 3rd, 1997, Addison-Wesley Professional.

MOOCS :

1. <https://www.coursera.org/specializations/algorithms?=>
2. <https://www.coursera.org/learn/dynamic-programming-greedy-algorithms#modules>

THANK YOU

Advanced Algorithms & Data Structures



Department of CSE

ADVANCED ALGORITHMS AND DATA STRUCTURES 23CS03HF

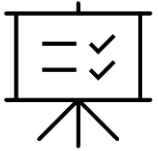
Topic:
Quick Sort

AIM OF THE SESSION



To familiarize students with the concept of Quick Sort

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

1. Demonstrate :- Quick sort algorithm.

Describe :- Quick sort time complexity and recurrence relation

LEARNING OUTCOMES



At the end of this session, you should be able to:

1. Define :- Quick sort .

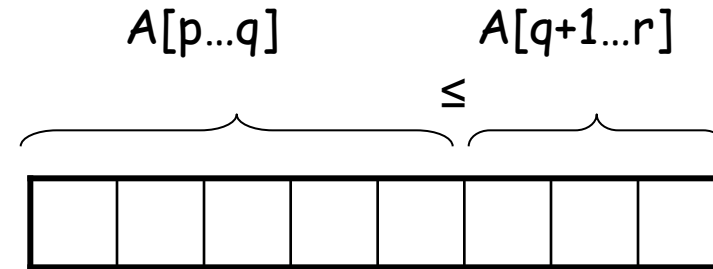
2. Describe :- Quick sort time complexity and recurrence relation

3. Summarize:- Description about the quick sort and time complexity of quick sort

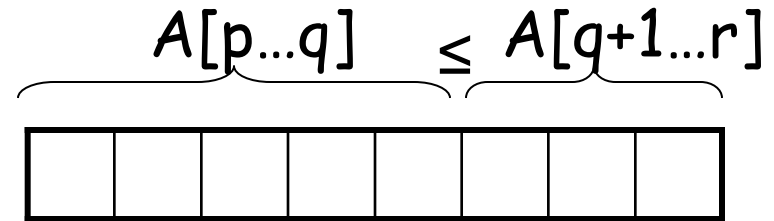
QUICK SORT

- QuickSort is a Divide and Conquer algorithm.
- It picks an element as pivot and partitions the given array around the picked pivot.
- There are many different versions of quickSort that pick pivot in different ways.
 1. Always pick first element as pivot.
 2. Always pick last element as pivot (implemented below)
 3. Pick a random element as pivot.
 4. Pick median as pivot.

- Sort an array $A[p..r]$



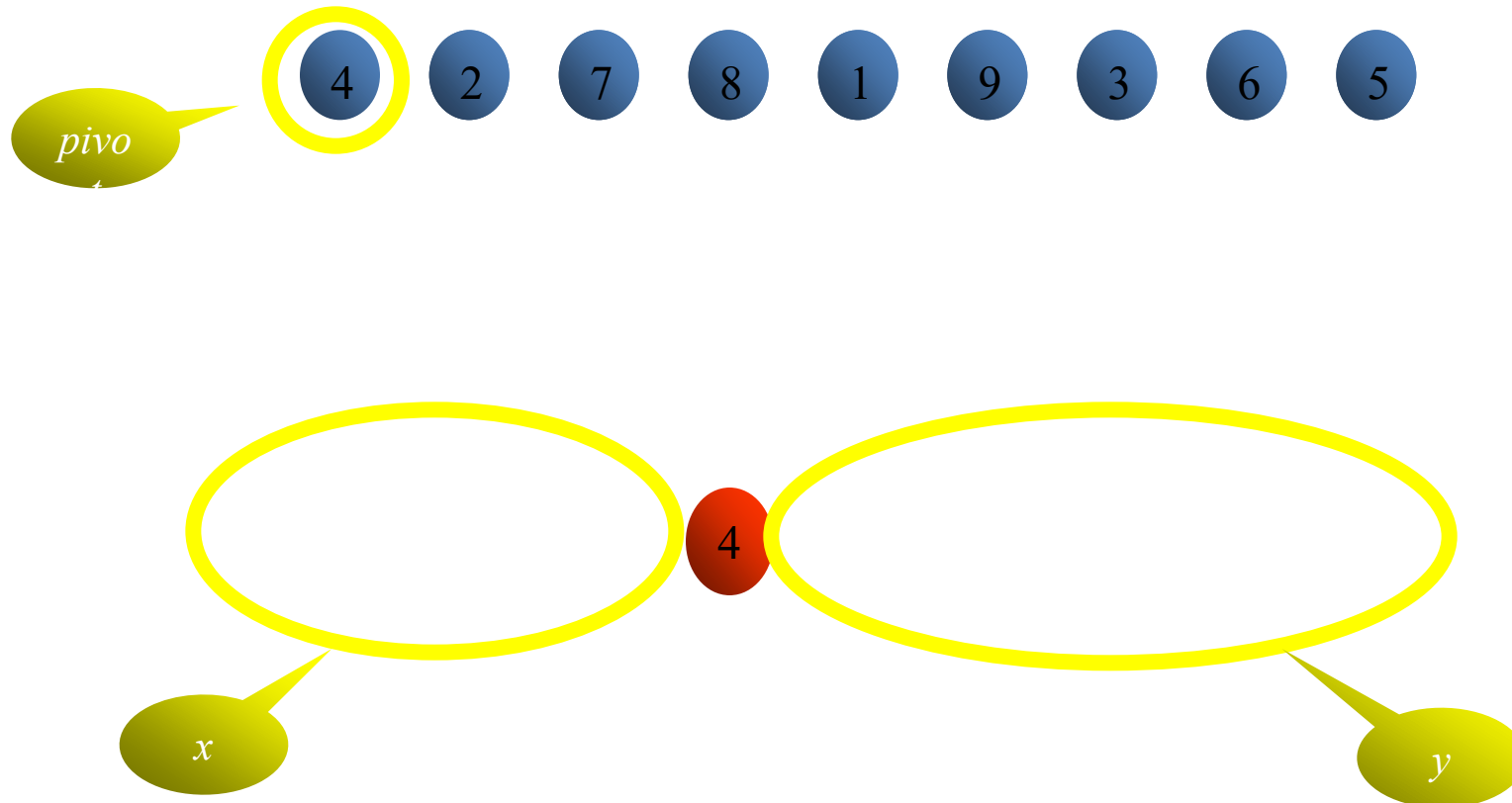
- **Divide**
 - Partition the array A into 2 subarrays $A[p..q]$ and $A[q+1..r]$, such that each element of $A[p..q]$ is smaller than or equal to each element in $A[q+1..r]$
 - Need to find index q to partition the array



- **Conquer**
 - Recursively sort $A[p..q]$ and $A[q+1..r]$ using Quicksort
- **Combine**
 - Trivial: the arrays are sorted in place
 - No additional work is required to combine them
 - The entire array is now sorted

- **Divide:**
 - Pick any element as the **pivot**, e.g, the first element
 - Partition the remaining elements into
 - FirstPart**, which contains all elements $< \text{pivot}$
 - SecondPart**, which contains all elements $> \text{pivot}$
- **Recursively sort** FirstPart and SecondPart.
- **Combine:** no work is necessary since sorting is done in place.

pivot divides *a* into two sublists *x* and *y*.



Example

Keep going from left side as long as $a[i] < \text{pivot}$ and from the right side as long as $a[j] > \text{pivot}$

<i>pivot</i> →	85	24	63	95	17	31	45
	98	<i>i</i>					<i>j</i>
	85	24	63	95	17	31	45
	98		<i>i</i>				<i>j</i>
	85	24	63	95	17	31	45
	98			<i>i</i>			<i>j</i>
	85	24	63	95	17	31	45
	98			<i>i</i>		<i>j</i>	

If $i < j$ interchange i^{th} and j^{th} elements and then
Continue the process.

85	24	63	45	17	31	95	98
			i			j	

85	24	63	45	17	31	95	98
				i		j	

85	24	63	45	17	31	95	98
					i		

85 24 63 45 17 31 95 98

j

i

85 24 63 45 17 31 95 98

j

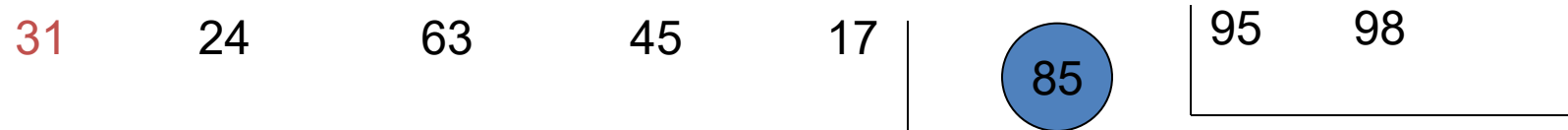
i

If $i \geq j$ interchange j^{th} and pivot elements and then divide the list into two sublists.

31 24 63 45 17 85 95 98

j

Two sublists:



Recursively sort

FirstPart and SecondPart
 QickSort(low, j-1) QickSort(j+1,high)

Quick Sort Algorithm

Algorithm QuickSort(low,high)

//Sorts the elements a[low],.....,a[high] which resides in the global array a[1:n] into //ascending order;

// a[n+1] is considered to be defined and must $\geq$ all the elements in a[1:n].

{

if(low < high) // if there are more than one element

{ // divide p into two subproblems.

j := **Partition**(low,high);

// j is the position of the partitioning element.

QuickSort(low,j-1);

QuickSort(j+1,high);

// There is no need for combining solutions.

}

}

Algorithm Partition(l,h)

```
{
    pivot:= a[l] ; i:=l; j:= h+1;
    while( i < j ) do
    {
        i++;

        while( a[ i ] < pivot ) do
            i++;
        j--;
        while( a[ j ] > pivot ) do
            j--;

        if ( i < j ) then Interchange(i,j ); // interchange ith and
        // jth elements.
    }

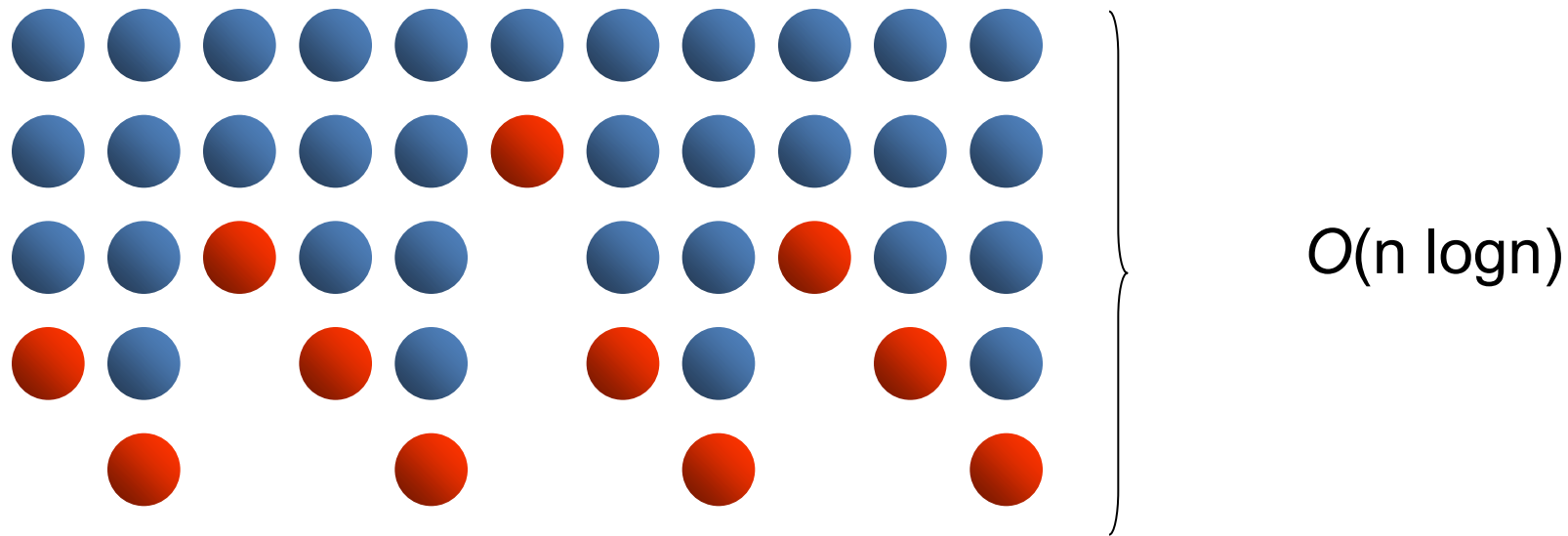
    Interchange(l, j ); return j; // interchange pivot and jth element.
}
```

Algorithm interchange (x,y)

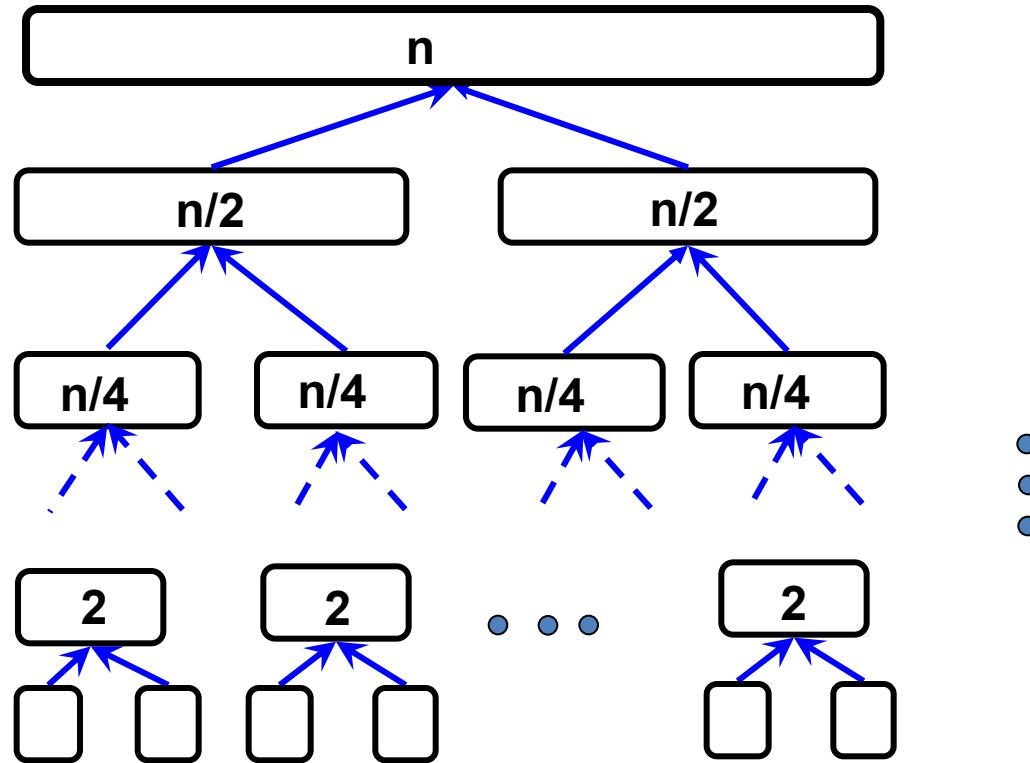
```
{  
    temp=a[x];  
    a[x]=a[y];  
    a[y]=temp;  
}
```

Time complexities of Quick sort A best/good case

- It occurs only if each partition divides the list into two equal size sublists.
- $T(n) = 2T(n/2) + cn = O(n \log n)$



Best/good Case



- **Total time:** $O(n \log n)$

Best Case Time Complexity

When n is a power of 2, $n = 2^k$, we can solve this equation by successive substitutions:

$$\begin{aligned}T(n) &= 2(2T(n/4) + cn/2) + cn \\&= 4T(n/4) + 2cn \\&= 4(2T(n/8) + cn/4) + 2cn \\&\vdots \\&= 2^k T(1) + kcn \\&= an + cn \log n\end{aligned}$$

It is easy to see that if $2^k < n \leq 2^{k+1}$, then $T(n) \leq T(2^{k+1})$. Therefore

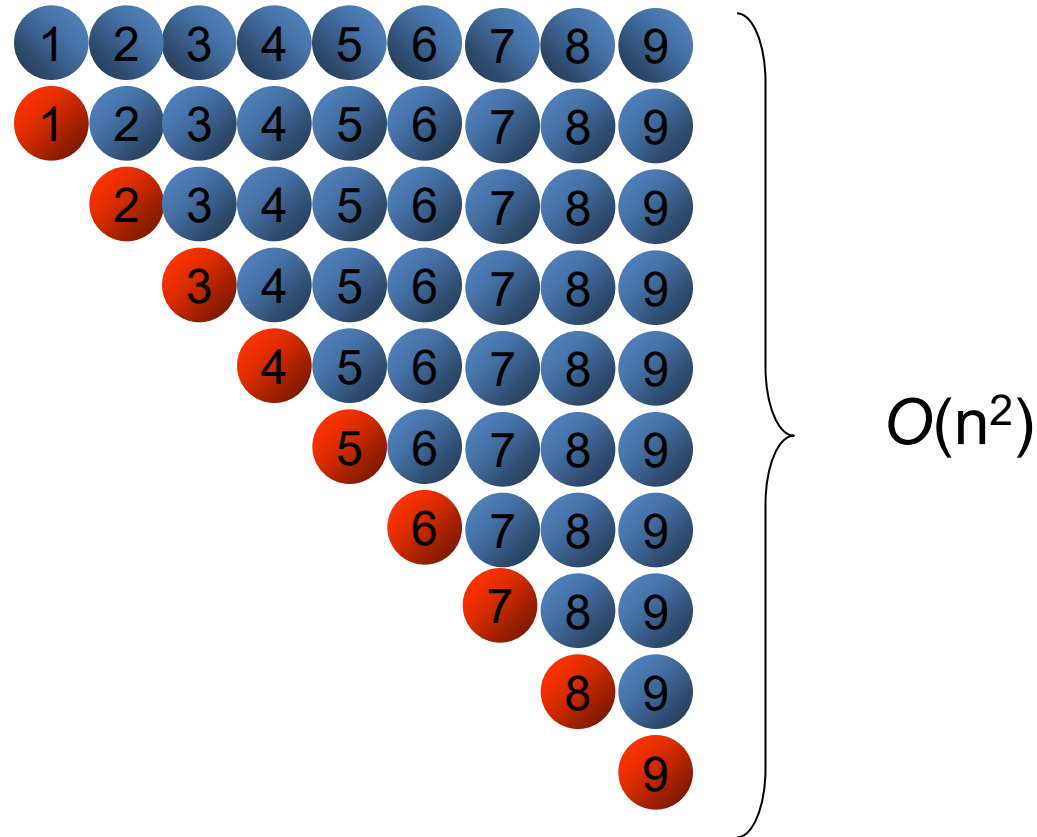
$$T(n) = O(n \log n)$$

Average case complexity

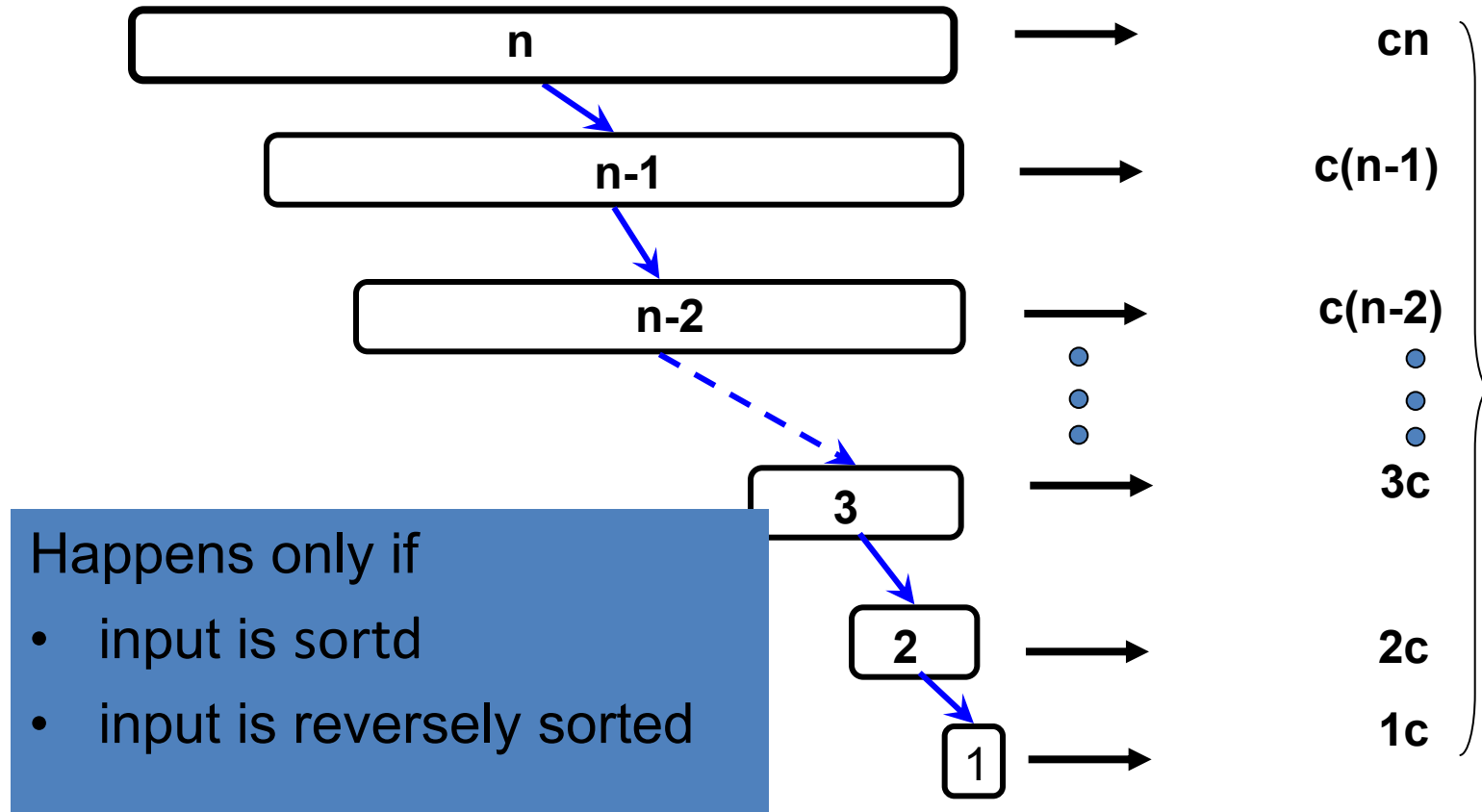
- To do average case analysis, we need to consider all possible permutation of array and calculate time taken by every permutation which doesn't look easy.
- We can get an idea of average case by considering the case when partition puts $O(n/9)$ elements in one set and $O(9n/10)$ elements in other set.
- Following is recurrence for this case.
- $T(n) = T(n/9) + T(9n/10) + (n)$
- $T(n) = O(n \log n)$

Time complexity analysis

A worst/bad case $T(n) = T(n-1) + cn = O(n^2)$



Worst/bad Case



Total time:
 $O(n^2)$

Worst-Case Analysis:

The pivot is the smallest element, all the time. Then $i = 0$, and if we ignore $T(0) = 1$, which is insignificant, the recurrence is $T(N) = T(N - 1) + cN$, $N > 1$

We telescope, using above equation repeatedly. Thus,

$$T(N - 1) = T(N - 2) + c(N - 1)$$

$$T(N - 2) = T(N - 3) + c(N - 2)$$

...

$$T(2) = T(1) + c(2)$$

Adding up all these equations yields

$$T(N) = T(1) + c((N+1)(N/2)-1) = O(N^2) \text{ as claimed}$$

earlier.

- **Pivot Selection:** Quick sort selects a pivot element from the array.
- **Partitioning:** It partitions the array into two sub-arrays, with elements less than the pivot on one side and elements greater than the pivot on the other.
- **Recursive Sorting:** The sub-arrays are recursively sorted using the same process.
- **Efficiency:** Quick sort has an average-case time complexity of $O(n \log n)$ and a worst-case time complexity of $O(n^2)$, making it efficient for large datasets.

SELF-ASSESSMENT QUESTIONS

In the quick sort algorithm, what is the purpose of the pivot element?

- (a) To act as a temporary storage
- (b) To partition the array into two sub-arrays
- (c) To find the maximum element
- (d) To merge two sorted arrays

What is the average-case time complexity of quick sort?

- (a) $O(n)$
- (b) $O(n \log n)$
- (c) $O(n^2)$
- (d) $O(\log n)$

TERMINAL QUESTIONS

1. Analyze quick sort algorithm find out recurrence relation
2. Compare and contrast quick sort with merge sort in terms of their time complexity, space complexity, and use cases.

REFERENCES FOR FURTHER LEARNING OF THE SESSION

Reference Books :

- 1 Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein., 3rd, 2009, The MIT Press.
- 2 Algorithm Design Manual, Steven S. Skiena., 2nd, 2008, Springer.
- 3 Data Structures and Algorithms in Python, Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser., 2nd, 2013, Wiley.
- 4 The Art of Computer Programming, Donald E. Knuth, 3rd, 1997, Addison-Wesley Professional.

MOOCS :

1. <https://www.coursera.org/specializations/algorithms?=>
2. <https://www.coursera.org/learn/dynamic-programming-greedy-algorithms#modules>

THANK YOU

Advanced Algorithms & Data Structures



Department of CSE

ADVANCED ALGORITHMS AND DATA STRUCTURES 23CS03HF

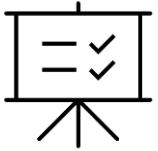
Topic: Closest Pair

AIM OF THE SESSION



To familiarize students with the concept of Closest Pair

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

1. Demonstrate :- Closest Pair.
2. Describe :- Solving of Closest pair using Divide and Conquer

LEARNING OUTCOMES

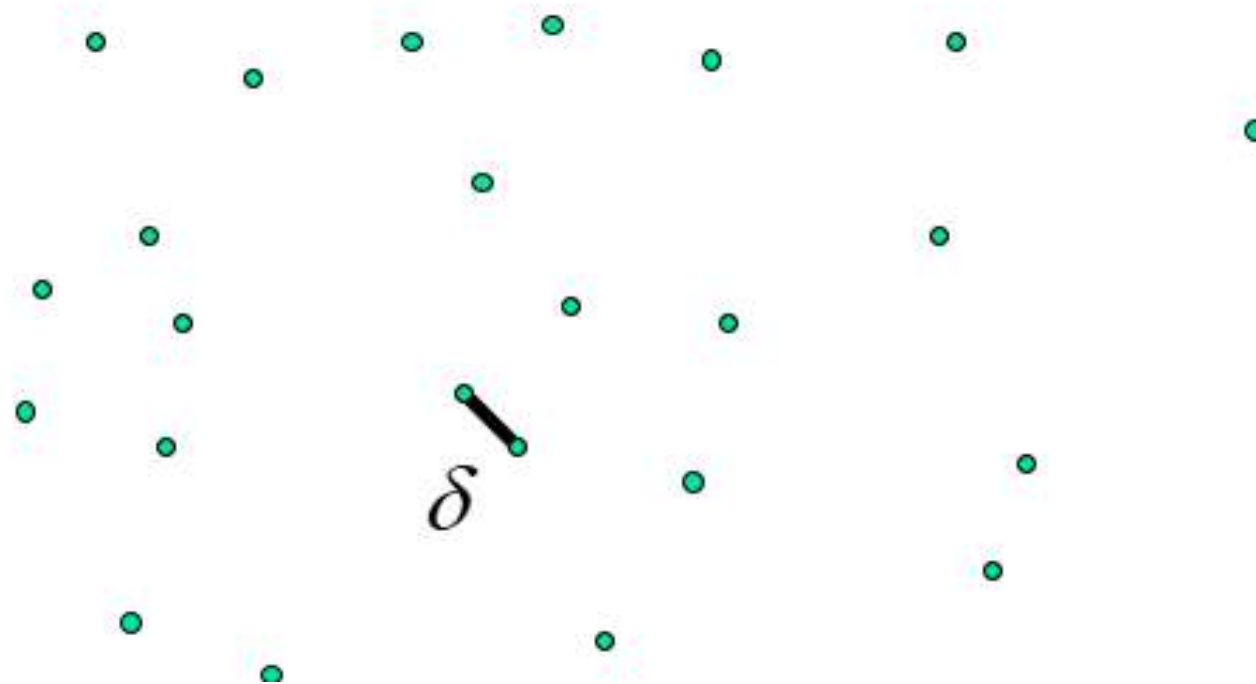


At the end of this session, you should be able to:

1. Define :- Closest Pair.
2. Describe :- Identifying Closest pair using Divide and Conquer
3. Summarize:- Description about the closest pair and time complexity of closest pair

Closest Pair

Given a set $S = \{p_1, p_2, \dots, p_n\}$ of n points in the plane find the two points of S whose distance is the smallest.



Closest Pair-Divide & Conquer

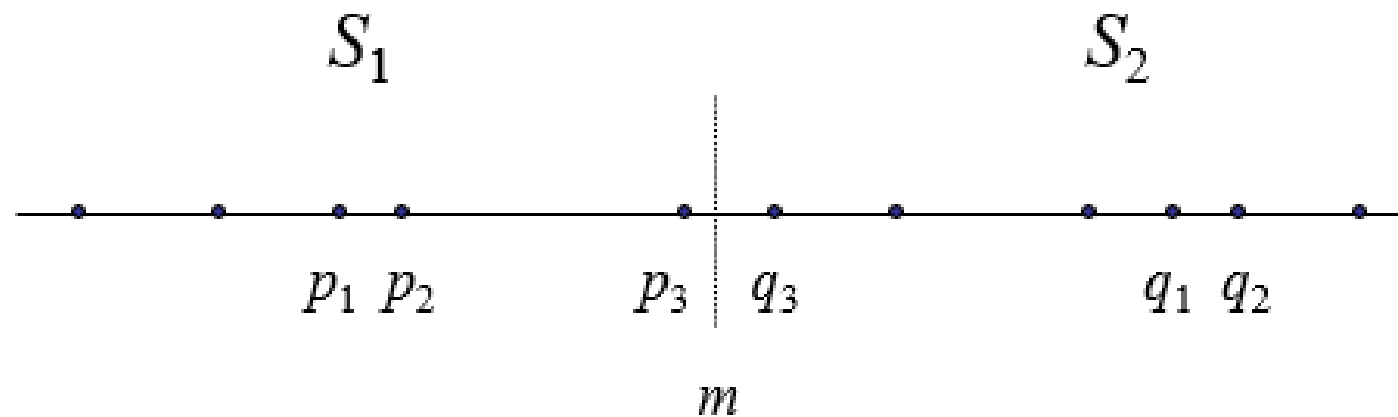
- Divide the problem into two equal-sized sub problems
- Solve those sub problems recursively
- Merge the sub problem solutions into an overall solution

Closest Pair-Divide & Conquer

- Assume that we have solutions for sub problems S_1 , S_2 .
- How can we merge in a time-efficient way?
 - The closest pair can consist of one point from S_1 and another from S_2
 - Testing all possibilities requires: $O(n/2) \cdot O(n/2) \in O(n^2)$
 - Not good enough

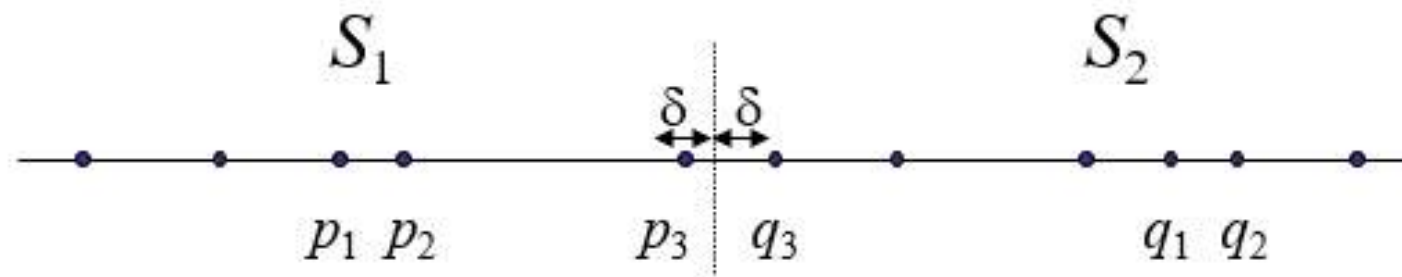
Closest Pair-Divide & Conquer dimension $d=1$

- Partition S , a set of points on a line, into two sets S_1 and S_2 at some point m such that for every point $p \in S_1$ and $q \in S_2$, $p < q$.
- Solve Closest Pair recursively on S_1 and S_2
- Let δ_1 be the smallest distance in S_1 and δ_2 in S_2
- Let δ be the smallest distance found so far: $\delta = \min(\delta_1, \delta_2)$
- What is the closest pair in S ?
 - Either $\{p_1, p_2\}$ or $\{q_1, q_2\}$ or some $\{p_3, q_3\}$ with $p_3 \in S_1$ and $q_3 \in S_2$.



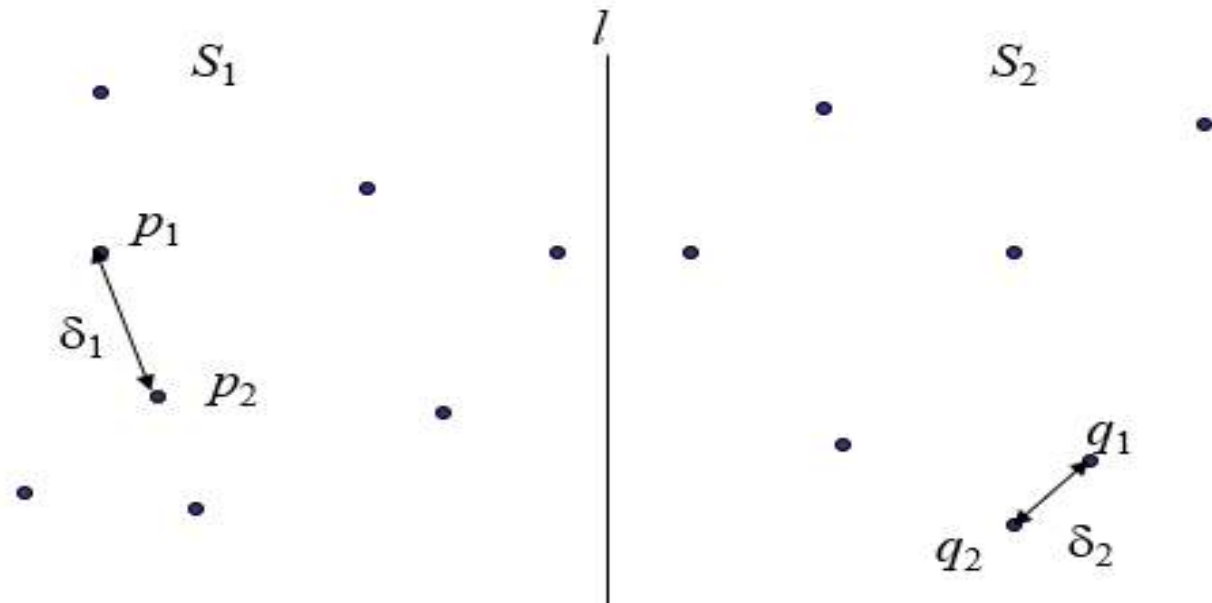
Closest Pair-Divide & Conquer dimension $d=1$

- To find a pair $\{p_3, q_3\}$, Is it necessary to check all possible pairs?
No
- How many points of S_1 or S_2 can lie within δ of m ?
 - Since δ is the distance between the closest pair in either S_1 or S_2 , there can only be at most 1 point in each side.
 - Therefore, the number of distance computations required to check for a closest pair $\{p_3, q_3\}$ with $p_3 \in S_1$ and $q_3 \in S_2$ is $O(1)$.
- Thus, the time complexity is
 - $O(n \log n)$



Closest Pair-Divide & Conquer dimension $d=2$

- Partition two dimensional set S into subsets S_1 and S_2 by a vertical line l at the median x coordinate of S .
- Solve the problem recursively on S_1 and S_2 .
- Let $\{p_1, p_2\}$ be the closest pair in S_1 and $\{q_1, q_2\}$ in S_2 .
- Let $\delta_1 = \text{distance}(p_1, p_2)$ and $\delta_2 = \text{distance}(q_1, q_2)$
- Let $\delta = \min(\delta_1, \delta_2)$



Closest Pair-Divide & Conquer dimension $d=2$

In order to merge we have to determine if exists a pair of points $\{p, q\}$ where $p \in S_1$, $q \in S_2$ and $\text{distance}(p, q) < \delta$.

If so, p and q must both be within δ of l .

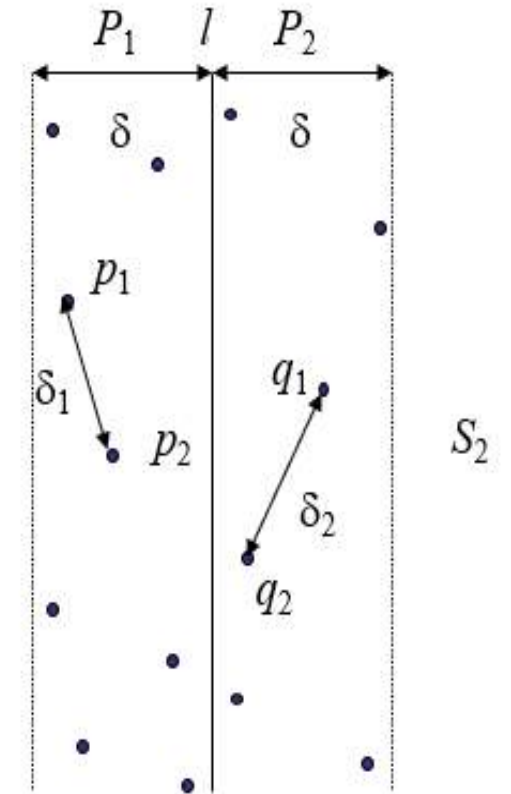
Let P_1 and P_2 be vertical regions of the plane of width δ on either side of l .

If $\{p, q\}$ exists, p must be within P_1 and q within P_2 .

For $d = 1$, there was at most one candidate point for p and one for q .

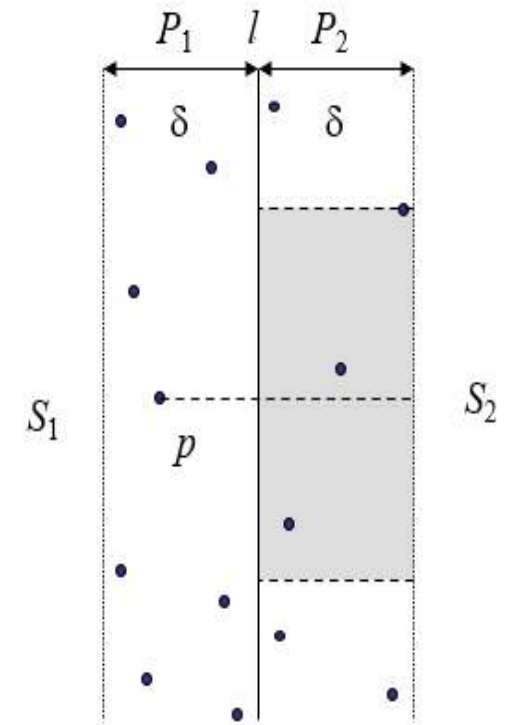
For $d = 2$, every point in S_1 and S_2 may be a candidate, as long as each is within δ of l , which implies:

$$O(n/2) \cdot O(n/2) = O(n^2)$$



Closest Pair-Divide & Conquer dimension $d=2$

- For a point p in P_1 , which portion of P_2 should be checked?
- We only need to check the points that are within δ of p .
- Thus we can limit the portion of P_2 .
- The points to consider for a point p must lie within $\delta \times 2\delta$ rectangle R .
- At most, how many points are there in rectangle R ?



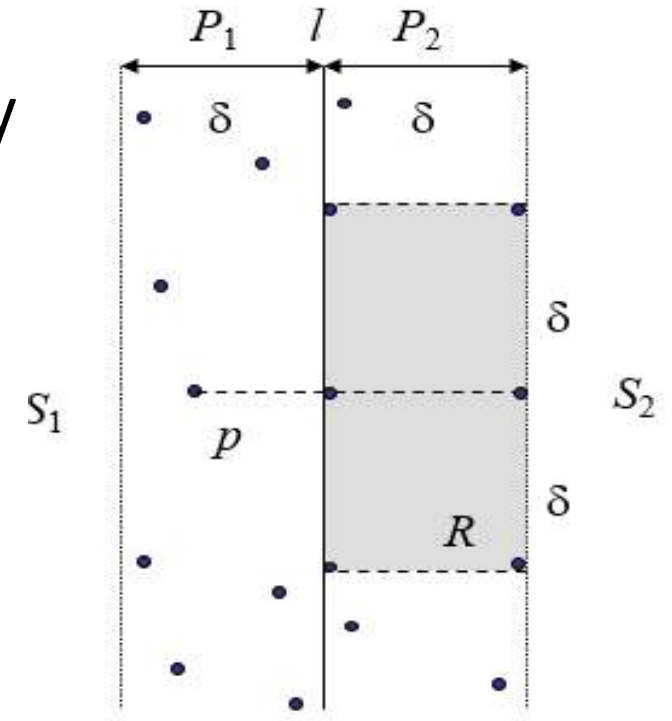
Closest Pair-Divide & Conquer dimension $d=2$

How many points are there in rectangle R ?

Since no two points can be closer than δ , there can only be at most 6 points

Therefore, $6 \cdot O(n/2) \in O(n)$

Thus, the time complexity is $O(n \log n)$



How do we know which 6 points to check?

Closest Pair-Divide & Conquer dimension $d=2$

How do we know which 6 points to check?

- Project p and all the points of S_2 within P_2 onto l .
- Only the points within δ of p in the y projection need to be considered (max of 6 points).
- After sorting the points on y coordinate we can find the points by scanning the sorted lists. Points are sorted by y coordinates only once.
- To prevent resorting in $O(n \log n)$ in each merge, two previously sorted lists are merged in $O(n)$.

Time Complexity: $O(n \log n)$

Sort Points:

Sort all points based on their x-coordinates.

• Divide:

Split the points into two halves, P_L (left) and P_R (right), based on the midpoint of the x-coordinates.

• Conquer:

Recursively find the closest pair in P_L and P_R . Compute the smallest distance (δ) from these two halves.

• Combine:

- Identify points in a vertical strip centered at the midpoint within a distance δ from the dividing line.

- Sort the points in the strip by y-coordinates and calculate distances between points within the strip.

- Update δ if a closer pair is found in the strip.

• Return the Closest Pair:

The minimum of the closest distances in P_L , P_R , and the strip is the result.

SELF-ASSESSMENT QUESTIONS

In the Divide and Conquer approach, which of the following is the first step?

- (a) Divide the points into two halves
- (b) Sort the points based on their x-coordinates
- (c) Recursively find the closest pair in both halves
- (d) Sort the points based on their y-coordinates

In the Closest Pair Problem, the Euclidean distance between two points (x_1, y_1) and (x_2, y_2) is calculated as:

- (a) $(x_1 - x_2)^2 + (y_1 - y_2)^2$
- (b) $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$
- (a) $(x_1 - x_2) + (y_1 - y_2)$
- (b) None

TERMINAL QUESTIONS

1. What is the Closest Pair Problem in computational geometry? Explain the goal and significance of solving this problem.
2. In the Divide and Conquer approach, how do you combine the solutions from the subproblems (i.e., after splitting the set of points into two halves)?

REFERENCES FOR FURTHER LEARNING OF THE SESSION

Reference Books :

1. Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein., 3rd, 2009, The MIT Press.
- 2 Algorithm Design Manual, Steven S. Skiena., 2nd, 2008, Springer.
- 3 Data Structures and Algorithms in Python, Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser., 2nd, 2013, Wiley.
- 4 The Art of Computer Programming, Donald E. Knuth, 3rd, 1997, Addison-Wesley Professiona.

MOOCS :

1. <https://www.coursera.org/specializations/algorithms?=>
2. <https://www.coursera.org/learn/dynamic-programming-greedy-algorithms#modules>

THANK YOU

Advanced Algorithms & Data Structures



Department of CSE

ADVANCED ALGORITHMS AND DATA STRUCTURES 23CS03HF

Topic:

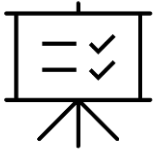
Greedy Method

AIM OF THE SESSION



To familiarize students with the concept of Greedy Method

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

1. Demonstrate :- Greedy Method.
2. Describe :- The optimization problem, Types in Greedy strategy.

LEARNING OUTCOMES



At the end of this session, you should be able to:

1. Define :- Greedy Method.
2. Describe :- The optimization problem, Types in Greedy strategy
3. Summarize:- Comparision between Divide and Conquer and Greedy Method.

Introduction to Optimization problem

Optimization problems are those for which the objective is to Maximize or Minimize some values.

For example,

- Finding the minimum number of colors needed to color a given graph.
- Finding the shortest path between two vertices in a graph

Greedy Method

- Greedy method solves problems by making the best choice that seems best at the particular moment.
- In some real-world games we need to think about the future before taking a decision and some games require a decision that is best at that moment.

Example : In chess game, every time we make a decision we need to think about the future consequences as well, where as, in Tennis or volleyball game, we make a decision which looks good at that situation.

- In computers, to solve some kind of problems, we need an algorithm that need to take a decision which looks good at that situation and there comes our greedy algorithm.
- Decisions are made based on Greedy Choice Property

Greedy Approach

- Greedy algorithm obtains an Optimal solution by making a sequence of decisions.
- Every greedy based problems will be given with a set of inputs and a set of constraints.
- Our objective is to find a solution vector which satisfies a set of constraints.
- Decisions are made one by one in some order.
- Each decision is made using a greedy-choice property or greedy criterion.

Greedy Choice Property

- Global Optimal Solutions will be made with local optimal (Greedy) choices
- In Greedy algorithm, the best choice will be selected at the moment to solve the sub problem that remains
- The choice of the greedy may depend on the previous choices made but it cannot depend on future choices or on the solutions to the sub problems

- A **feasible solution** is any subset of original input that satisfies a given set of constraints.
- A **objective function** is an input for which a feasible solution is to be obtained that either maximizes or minimizes.
- An **optimal solution** is a feasible solution which maximizes or minimizes the objective function.
- **Irrevocable** - Once the choices are made in a particular step , it can not be changed in subsequent steps.

Algorithm For Greedy Method Control Abstraction

```
Greedy(a,n)      // a[1:n] contains the n inputs
{
    solution=  $\varphi$  // Initialize solution
    for i=1 to n do
    {
        x := Select(a);
        if Feasible(solution , x) then
            solution=Union(solution , x)
    }
    return solution;
}
```

Counting Coins Problem :

This problem is to count to a desired value by choosing the least possible coins and the greedy approach forces the algorithm to pick the largest possible coin. If we are provided coins of ₹ 1, 2, 5 and 10 and we are asked to count ₹18 then the greedy procedure will be,

- 1 – Select one ₹ 10 coin, the remaining count is 8
- 2 – Then select one ₹ 5 coin, the remaining count is 3
- 3 – Then select one ₹ 2 coin, the remaining count is 1
- 4 – And finally, the selection of one ₹ 1 coins solves the problem

•Though, it seems to be working fine, for this count we need to pick only 4 coins. But if we **slightly change** the problem then the same approach may not be able to produce the same optimum result.

•For the currency system, where we have coins of 1, 7, 10 value, counting coins for value 18 will be absolutely optimum but **for count like 15**, it may **use more coins than necessary**. For example, the greedy approach will use 10 + 1 + 1 + 1 + 1 + 1, total 6 coins. Whereas the same problem could be solved by using only 3 coins (7 + 7 + 1)

•Hence, we may conclude that the greedy approach picks an immediate optimized solution and may fail where global optimization is a major concern.

Greedy Strategy

Subset Paradigm:

- The selection procedure itself based on some optimization measures

Example:

- Knapsack problem
- job sequencing with deadlines

Order Paradigm:

- The selection procedure does not depends on optimization measures. But, decisions by considering the inputs some particular order.

Example:

- Minimum Spanning tree (Prim's and Kruskal's method))

Types of Greedy Problems

- **Subset Paradigm:** To solve a problem (or possibly find the optimal/best solution), greedy approach generate subset by selecting one or more available choices.

Example:

- Knapsack problem
- Job sequencing with deadlines

- **Order Paradigm:** greedy approach generate some arrangement/order to get the best solution

Example:

- Single source shortest path problem

Three Important Activities

1. **Selection:** Selection of solution from $a[]$ and removes it
2. **Feasibility:** Feasible(solution,x) is a Boolean function to determine whether x can be included into the solution vector
3. **Optimality:** From the set of feasible solutions, the particular solution which minimizes or maximizes the given objective function

Applications of Greedy Method

- Knapsack Problem
- Job Sequencing with deadlines
- Minimum cost spanning tree
- Huffman Codes
- Single source shortest path problem

Divide and conquer	Greedy Algorithm
Divide and conquer is used to find the solution, it does not aim for the optimal solution.	A greedy algorithm is optimization technique. It tries to find an optimal solution from the set of feasible solutions
DC approach divides the problem into small sub problems, each sub problem is solved independently and solutions of the smaller problems are combined to find the solution of the large problem.	In greedy approach, the optimal solution is obtained from a set of feasible solutions.
Sub problems are independent, so DC might solve same sub problem multiple time.	Greedy algorithm does not consider the previously solved instance again, thus it avoids the re-computation.
DC approach is recursive in nature, so it is slower and inefficient.	Greedy algorithms are iterative in nature and hence faster.
Divide and conquer algorithms mostly runs in polynomial time	Greedy algorithms also run in polynomial time but takes less time than Divide and conquer
Example: Merge sort, Quick sort	Example: Knapsack problem, Job scheduling problem

- **Selection:** Make decisions based on the best available option at each step without reconsidering previous choices.
- **Locality:** Optimize locally with the hope of finding a global optimum.
- **Efficiency:** Efficient for problems where making locally optimal choices leads to an optimal solution overall, but may not guarantee the best solution in all cases.

SELF-ASSESSMENT QUESTIONS

What type of problems are well-suited for the greedy method?

- (a) Problems with complex constraints
- (b) Problems requiring global optimum solutions
- (c) Problems where local optimization leads to a feasible solution
- (d) Problems involving exhaustive search techniques

Which step is crucial in the greedy method to ensure that each chosen option contributes to a feasible solution?

- (a) Sorting choices in ascending order
- (b) Evaluating all possible subsets
- (c) Considering constraints and requirements
- (d) Using dynamic programming techniques

TERMINAL QUESTIONS

1. Describe the main ideas behind greedy algorithms.
2. List out the various types in Greedy strategy.
3. Analyze the time complexity of fractional knapsack problem

REFERENCES FOR FURTHER LEARNING OF THE SESSION

Reference Books :

1. Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein., 3rd, 2009, The MIT Press.
- 2 Algorithm Design Manual, Steven S. Skiena., 2nd, 2008, Springer.
- 3 Data Structures and Algorithms in Python, Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser., 2nd, 2013, Wiley.
- 4 The Art of Computer Programming, Donald E. Knuth, 3rd, 1997, Addison-Wesley Professiona.

MOOCS :

1. <https://www.coursera.org/specializations/algorithms?=>
2. <https://www.coursera.org/learn/dynamic-programming-greedy-algorithms#modules>

THANK YOU

Advanced Algorithms & Data Structures



Department of CSE

ADVANCED ALGORITHMS AND DATA STRUCTURES 23CS03HF

Topic:

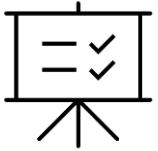
Minimum Cost Spanning Tree

AIM OF THE SESSION



To familiarize students with the concept of Spanning Tree

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

1. Demonstrate :- Minimum Cost Spanning Tree.
2. Describe :- Minimum spanning tree and its related algorithms like Prim's and Kruskal's methods

LEARNING OUTCOMES



At the end of this session, you should be able to:

1. Define :- Minimum Cost Spanning Tree.
2. Describe :- Minimum spanning tree and its related algorithms like Prim's and Kruskal's methods
3. Summarize:- How to build an MST for a given graph using Prim's and Kruskal's Algorithm.

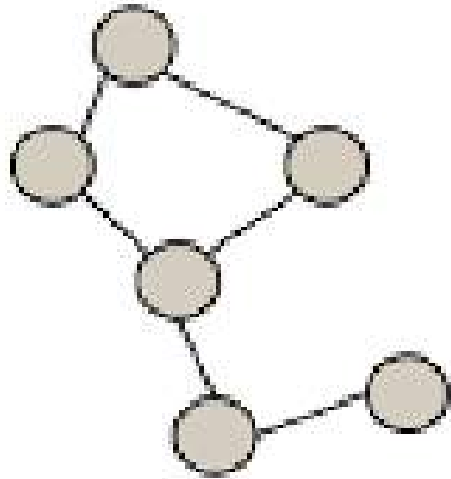
Spanning Tree

- **Graph:** Set of Vertices and edges $G: \{ V, E \}$
- A *tree* is a connected undirected graph that contains **no cycles**.
- i.e **Tree** : Tree is Graph $T: \{ V, E \}$
 - Acyclic
 - N nodes
 - N-1 edges
- Note: Every Tree is a Graph , But Every Graph is not a Tree

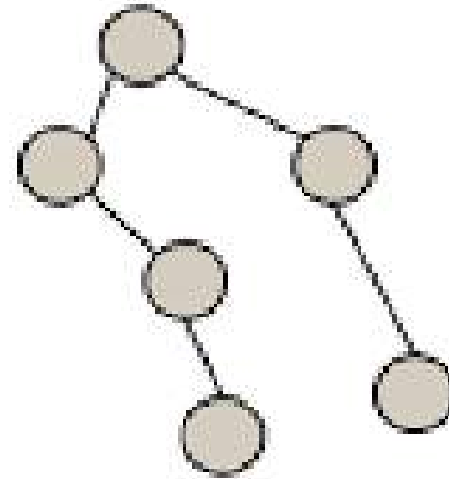
Properties of a Spanning Tree

- A connected graph G can have **more than** one spanning tree.
- All possible spanning trees of graph G , have the **same number of edges and vertices**.
- The spanning tree **does not** have any cycle (loops).
- **Removing one** edge from the spanning tree will make the graph disconnected, i.e. the spanning tree is **minimally connected**.
- **Adding one edge** to the spanning tree will create a circuit or loop, i.e. the spanning tree is **maximally acyclic**.

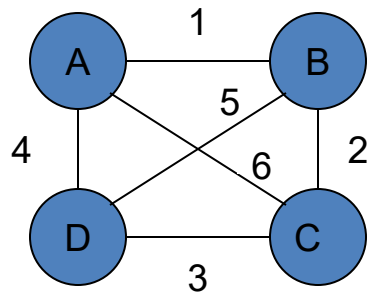
Graphs & Trees



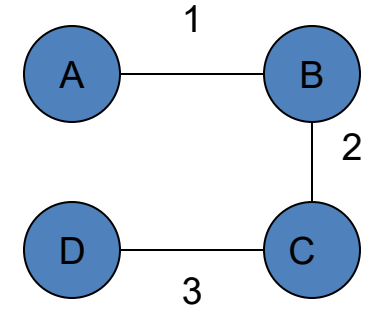
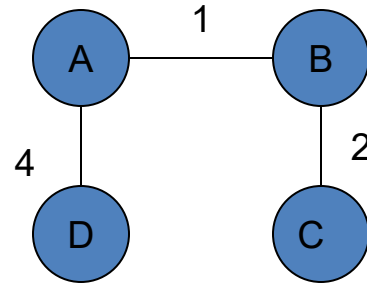
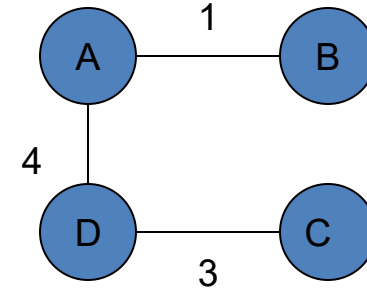
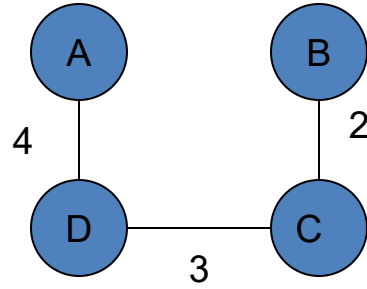
Graph



Tree



Undirected Graph



...

Some Spanning Trees

Spanning Tree : A spanning tree of G is a sub graph T that is

➤ Connected.

➤ Acyclic.

➤ Includes all of the vertices in given graph and vertices-1 edges.

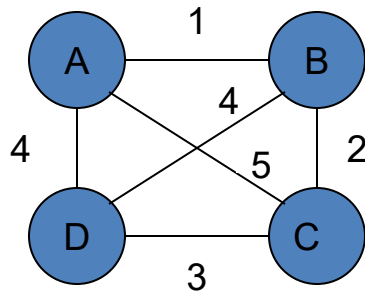
- $T \subseteq G$
 $V' = V$
- $T: \{V' = V, E' \subseteq E\}$
 $E = |V| - 1$

Spanning Tree properties:

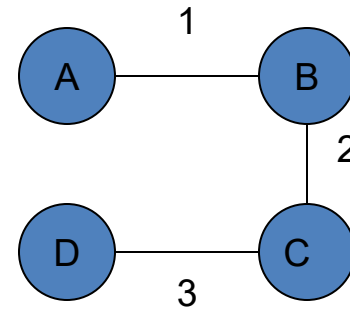
- A graph may have many spanning trees. If the Graph $G: \{V, E\}$ then the possible spanning Trees are $ECV-1$
- Removing one edge from the Spanning Tree Will make it as Disconnected graph
- Adding one edge to the Spanning Tree will create a cycle.
- If each edge has distinct weight then there will be only one & unique Minimum cost Spanning Tree.
- Disconnected graph doesn't have any Spanning Tree.

Minimum Cost Spanning Tree (MST)

- A **minimum spanning tree** is the one among all the spanning trees with the **smallest total cost** or A Spanning tree with minimum weight



Undirected Graph

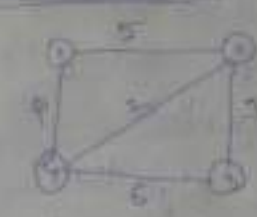


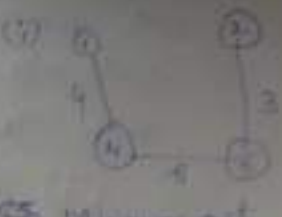
Minimum Spanning Tree

Applications of MSTs

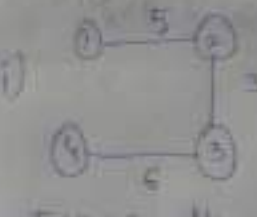
- Computer Networks
 - How to connect a set of computers using the **minimum amount of wire..**
- Civil Network Planning
- Computer Network Routing Protocol
- Cluster Analysis

Weighted graphs:

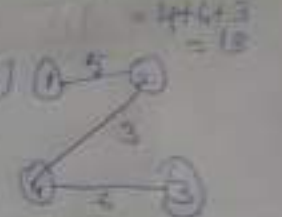


(i) 

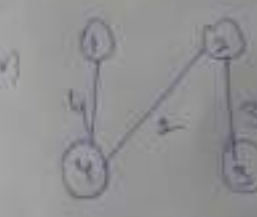
min. minimum cost
 $= 5 + 3 + 3$
 $= 11$

(ii) 

$5 + 3 + 6 = 14$

(iii) 

$5 + 3 + 6 = 14$

(iv) 

$4 + 2 + 3 = 9$

From above observation, if you have a weighted graph and get different spanning tree with varying in minimum cost
 $\Rightarrow$ For all different spanning tree which is minimum that is minimum cost spanning tree

→ Trying all possible spanning trees
with minimum cost is too lengthy.
That's why go for Greedy Method
with Two Algorithms

(i) Prim's Algorithm

(ii) Kruskal's Algorithm

Kruskal's Algorithm

- Kruskal's algorithm is a greedy algorithm in graph theory that finds a minimum spanning tree for a connected weighted graph.
- This algorithm treats every node as an independent tree and connects one with another only if it has the lowest cost compared to all other options available.
- **Steps to Kruskal's algorithm:**
 - Sort the graph edges with respect to their weights.
 - Start adding edges to the minimum spanning tree from the edge with the smallest weight until the edge of the largest weight.
 - Only add edges which don't form a cycle—edges which connect only disconnected components.

Kruskal's Algorithm:

MST KRUSKAL(G, w)

$T \leftarrow \emptyset$

cost $\leftarrow 0$

for each vertex $v \in V[G]$

do MAKE-SET(v)

sort the edges of E into nondecreasing order by weight w

for each edge $(u, v) \in E$, taken in nondecreasing order by weight

do if FIND-SET(u) $\neq$ FIND-SET(v)

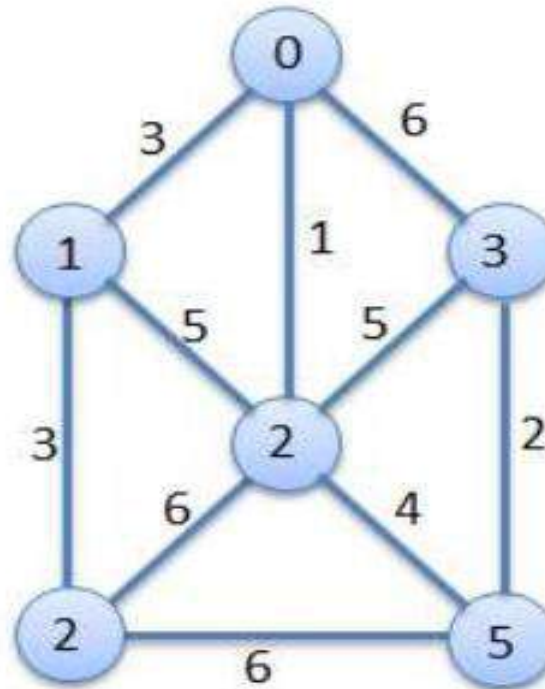
then $T.E \leftarrow T.E \cup \{(u, v)\}$

UNION(u, v)

cost $\leftarrow$ cost + cost(u, v)

return cost

Example: Find Minimum spanning tree cost using kruskal's method for the following Graph

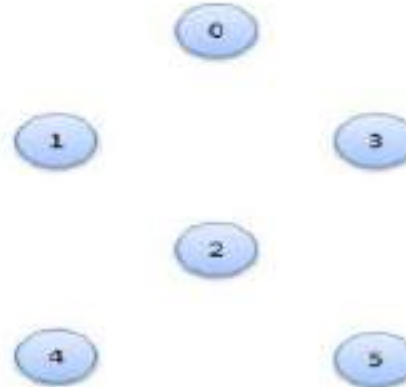


Step1. Edges are sorted in ascending order by weight.

Edge No.	Vertex Pair	Edge Weight
E1	(0,2)	1
E2	(3,5)	2
E3	(0,1)	3
E4	(1,4)	3
E5	(2,5)	4
E6	(1,2)	5
E7	(2,3)	5
E8	(0,3)	6
E9	(2,4)	6
E10	(4,5)	6

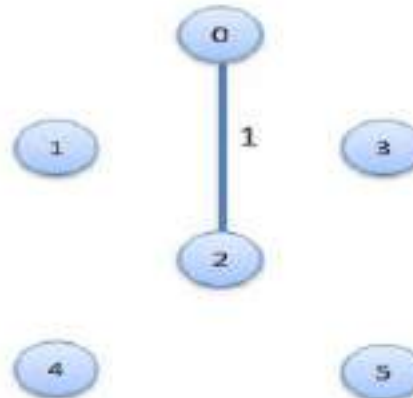
Step2. Edges are added in sequence.

$T.E = \{ \emptyset \}$ cost=0

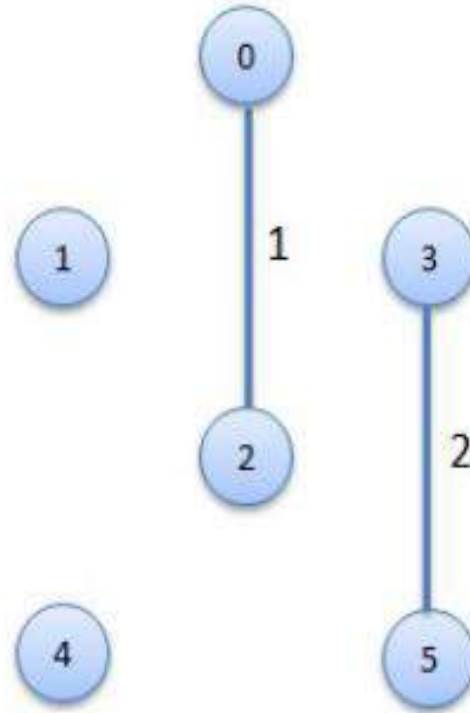


Step3: Consider the minimum edge if it's not connected add the edge and consider the cost

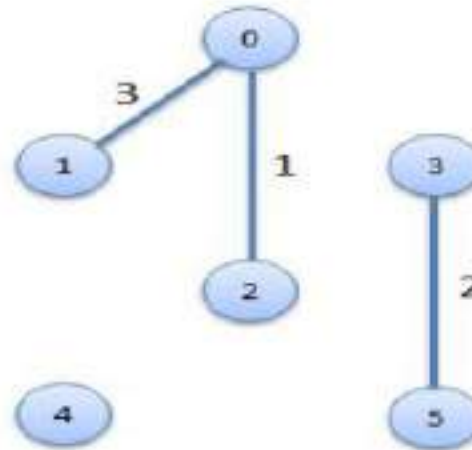
Edge E1 $T.E = \{ (0,2) \}$ cost=1



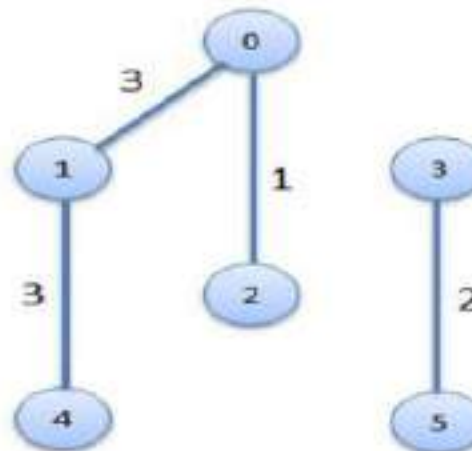
Step4: consider the Edge E2 $T.E = \{ (0,2), (3,5) \}$ $\text{cost} = 1 + 2$



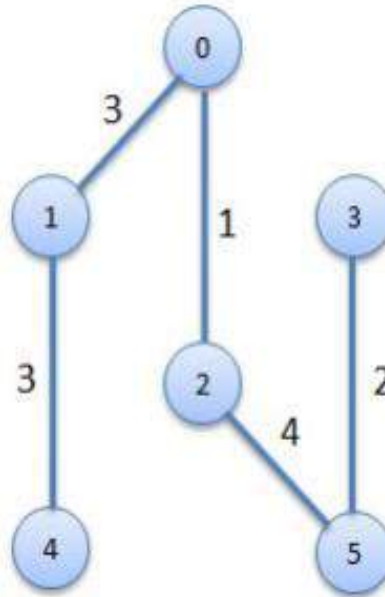
Step5: consider the Edge E3 T.E= { (0,2), (3,5), (1,0) } cost=1+2+3



Step6: consider the Edge E4 T.E= { (0,2), (3,5), (1,0), (1,4) } cost=1+2+3+3



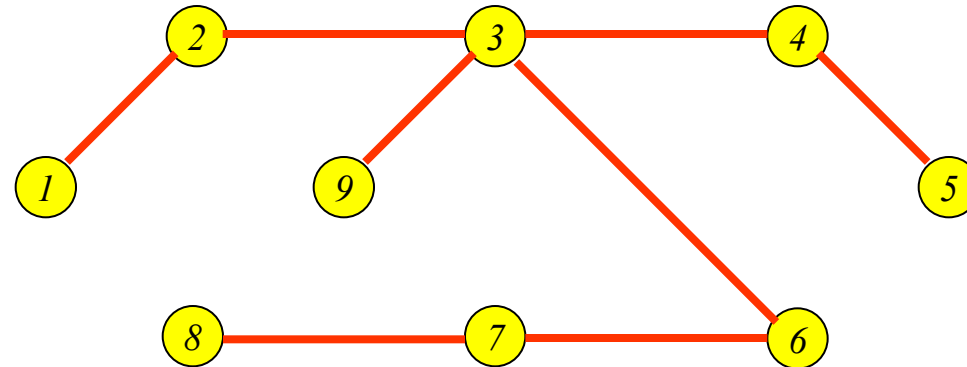
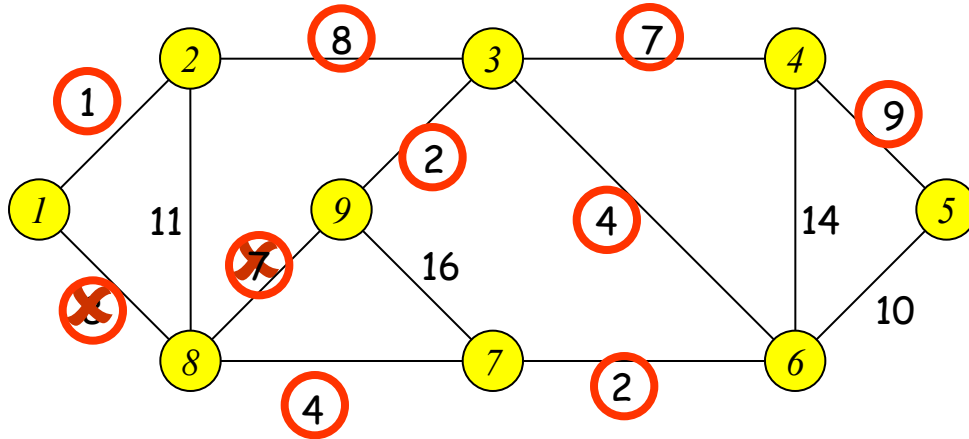
Step7: consider the Edge E5 T.E= { (0,2), (3,5), (1,0), (1,4), (2,5) } cost=1+2+3+3+4

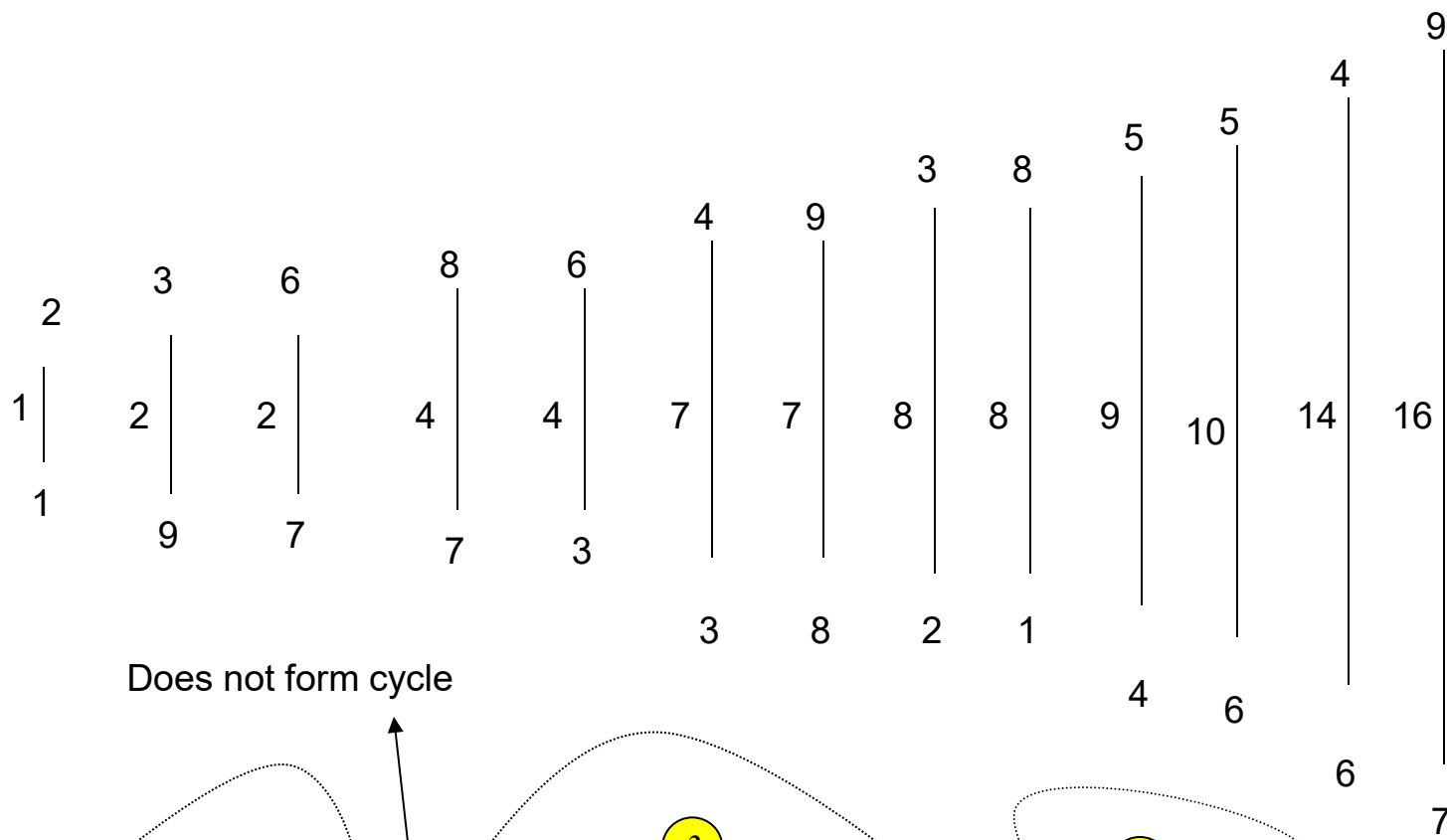


Step 8: check all the remaining edges such that it doesn't form any cycle, for all remaining if we check it will form cycle and its already connected component so the cost of the Tree is:

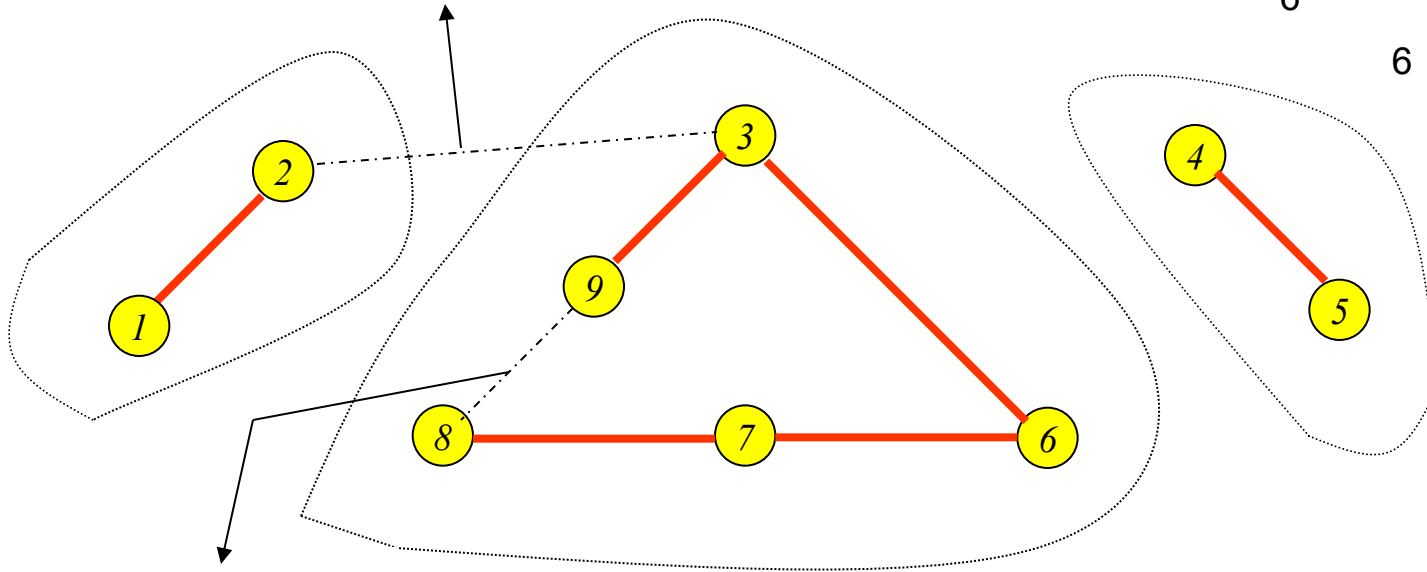
$$\text{Total Cost} = 1+2+3+3+4 = 13$$

Kruskal's Algorithm





Does not form cycle



Forms cycle

Time complexity of kruskal's algorithm

- With an efficient **Find-set** and **union** algorithms, the running time of kruskal's algorithm will be dominated by the time needed for sorting the edge costs of a given graph.
- Hence, with an efficient sorting algorithm(**merge sort**), the complexity of kruskal's algorithm is $O(E \log E)$.

MST-Prim's Algorithm

- Prim's Algorithm is another greedy algorithm used for finding the Minimum Spanning Tree (MST) of a given graph.
- The graph must be weighted, connected and undirected
- Start with minimum cost edge.
- For rest of the procedure, always select a minimum cost edge from graph make sure that already connected to the selected vertices.
- Continue this process until the tree has $n - 1$ edges.

Algorithm for prim's

```
root ← {Any arbitrary node in the G.V}
Prim(G={ V,E},root) {
  T ← ∅
  T.V ← T.V ∪ root
  Cost ← 0
  While (G.V ≠ T.V) do
  {   find an edge e: u→v  of min cost
      such that u∈ T.V and v ∉ T.V
      T ← T.E ∪ (e)
      T.V ← T.V ∪ (v)
      cost ← cost +cost(u,v)
  }
  return cost
}
```

Steps for implementing Prim's Algorithm-

Step-01: Remove all loops and parallel edges in graph if there are any in given graph then apply prim's algorithm

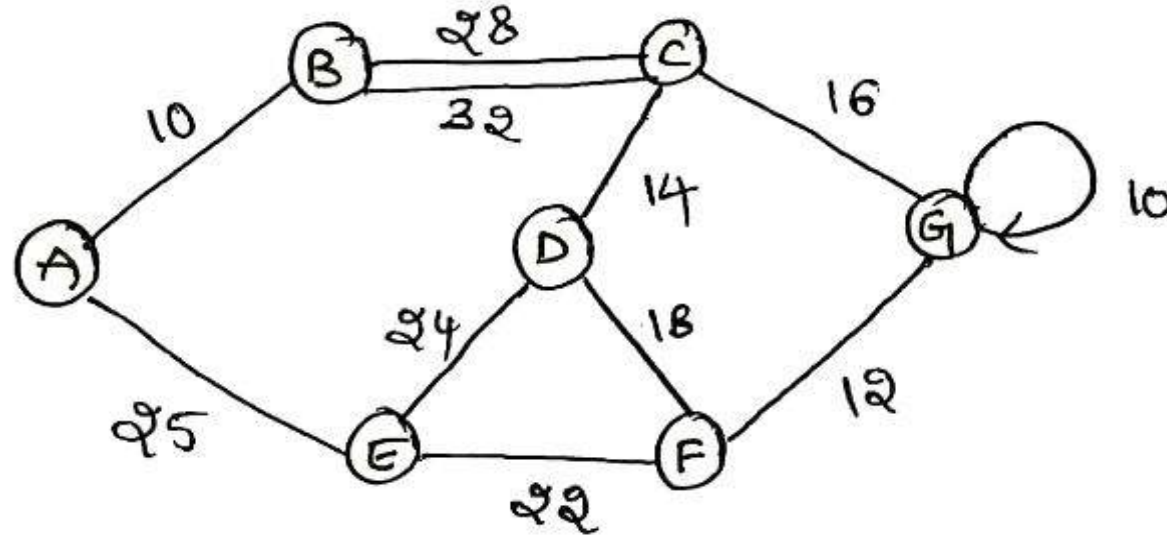
Step-02: Randomly choose any vertex. We usually select and start with a vertex that connects to the edge having least weight.

Step-03: Find all the edges that connect the tree to new vertices, then find the least weight edge among those edges and include it in the existing tree. If including that edge creates a cycle, then reject that edge and look for the next least weight edge.

Step-04: Keep repeating step-03 until all the vertices are included and Minimum Spanning Tree (MST) is obtained.

Question: Construct the minimum spanning tree

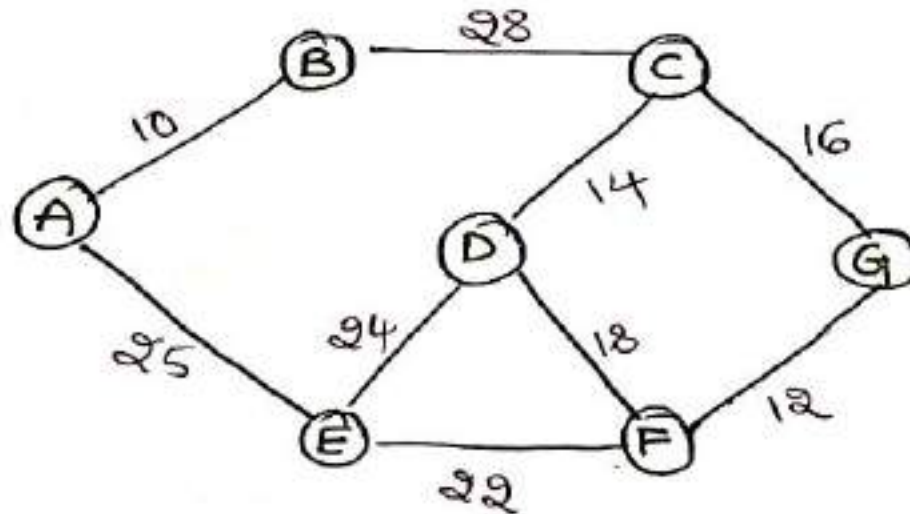
(Question: Construct the minimum spanning tree (MST) for the given graph using prims algorithm?



Solution:

Solution:

Step 1: Remove self loops and parallel edge, if any present in the graph. If any parallel edge is there consider minimum cost edge, redraw the graph.



step 2 : Select arbitrary node as root Here

step 2 : Select arbitrary node as root Here

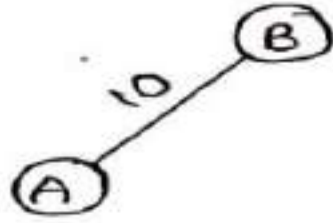
$$\text{root} \leftarrow A$$

$$T.V = \{A\} \quad \text{i.e. visited node}$$

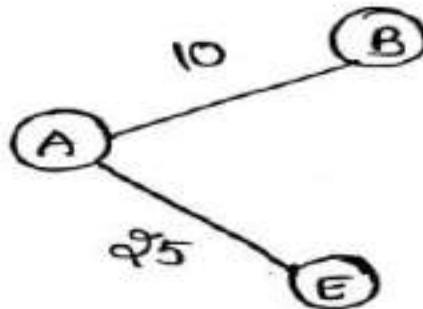
$$G.V = \{A, B, C, D, E, F, G\}$$

$$T.E = \{(A, B)\}$$

$$\text{Cost} = 10$$



STEP 3: $T.V = \{A, B, E\}$
 $G.V = \{A, B, C, D, E, F, G\}$
 $T.E = \{(A, B), (A, E)\}$
 $COST = 10 + 25$



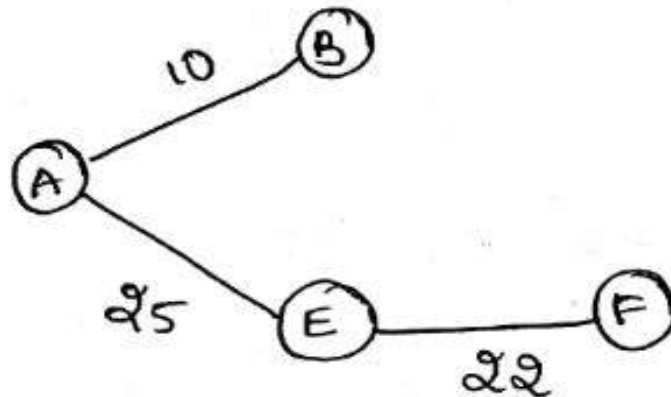
step 4:

$$T.V = \{ A, B, E, F \}$$

$$G.V = \{ A, B, C, D, E, F, G \}$$

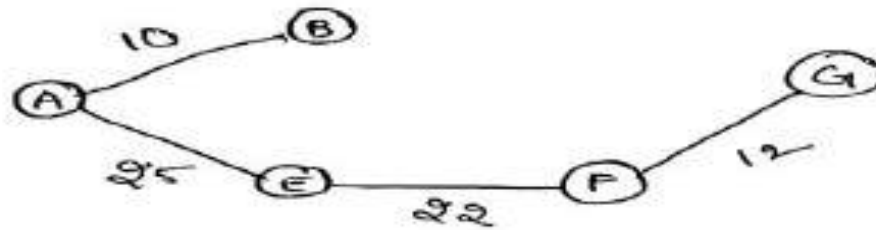
$$T.E = \{ (A, B), (A, E), (E, F) \}$$

$$\text{Cost} = 10 + 25 + 22$$



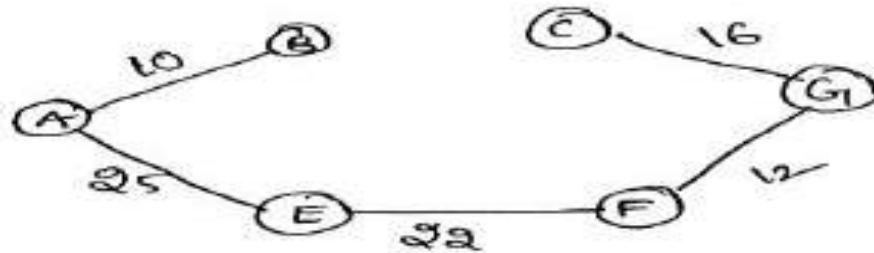
Step 5: $T.V = \{A, B, E, F, G\}$
 $G.V = \{A, B, C, D, E, F, G\}$

Step 5: $T.V = \{A, B, E, F, G\}$
 $G.V = \{A, B, C, D, E, F, G\}$
 $T-E = \{(A, B), (A, E), (E, F), (F, G)\}$
 $Cost = 10 + 25 + 22 + 12$



Step 6:

$T.V = \{A, B, E, F, G, C\}$
 $G.V = \{A, B, C, D, E, F, G\}$
 $T-E = \{(A, B), (A, E), (E, F), (F, G), (G, C)\}$
 $Cost = 10 + 25 + 22 + 12 + 16$



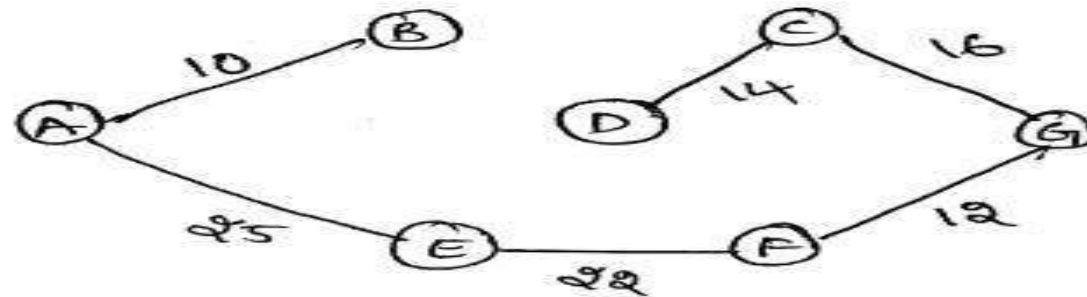
Step 7:

$$T.V = \{ A, B, E, F, G, C \}$$

$$G.V = \{ A, B, C, D, E, F, G \}$$

$$T.E = \{ (A, B), (A, E), (E, F), (F, G), (G, C), (C, D) \}$$

$$\text{Cost} = 10 + 25 + 22 + 12 + 16 + 14$$



Step 8:

All vertices covered the final cost is = $10 + 25 + 22 + 12 + 16 + 14$
 $= 99 \text{ units}$

Question-01:

Will both Prim's Algorithm and Kruskal's Algorithm always produce the same Minimum Spanning Tree (MST) for any given graph?

Question-01:

Will both Prim's Algorithm and Kruskal's Algorithm always produce the same Minimum Spanning Tree (MST) for any given graph?

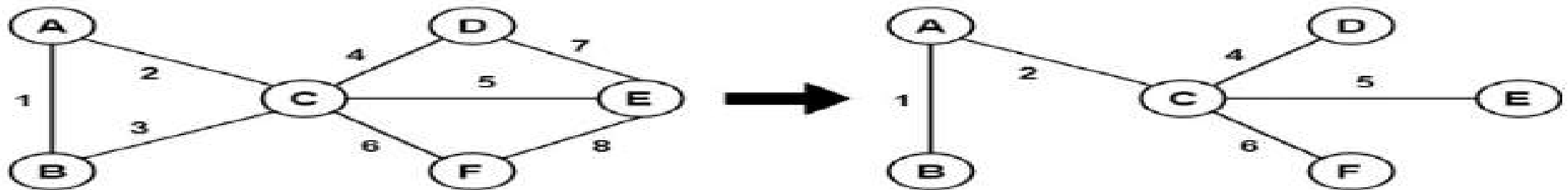
The following two cases are possible-

Case-01: When all the edge weights are distinct-

If all the edge weights are distinct, then both the algorithms are guaranteed to find the same i.e. unique MST.

Example-

Consider the following example-



Minimum Spanning Tree (MST)
(Cost = 18 units)

The application of both the algorithms on the above graph will produce the same MST as shown.

This example clearly illustrates when all the edge weights are distinct, both the algorithms always produces the same MST having the same cost as shown.

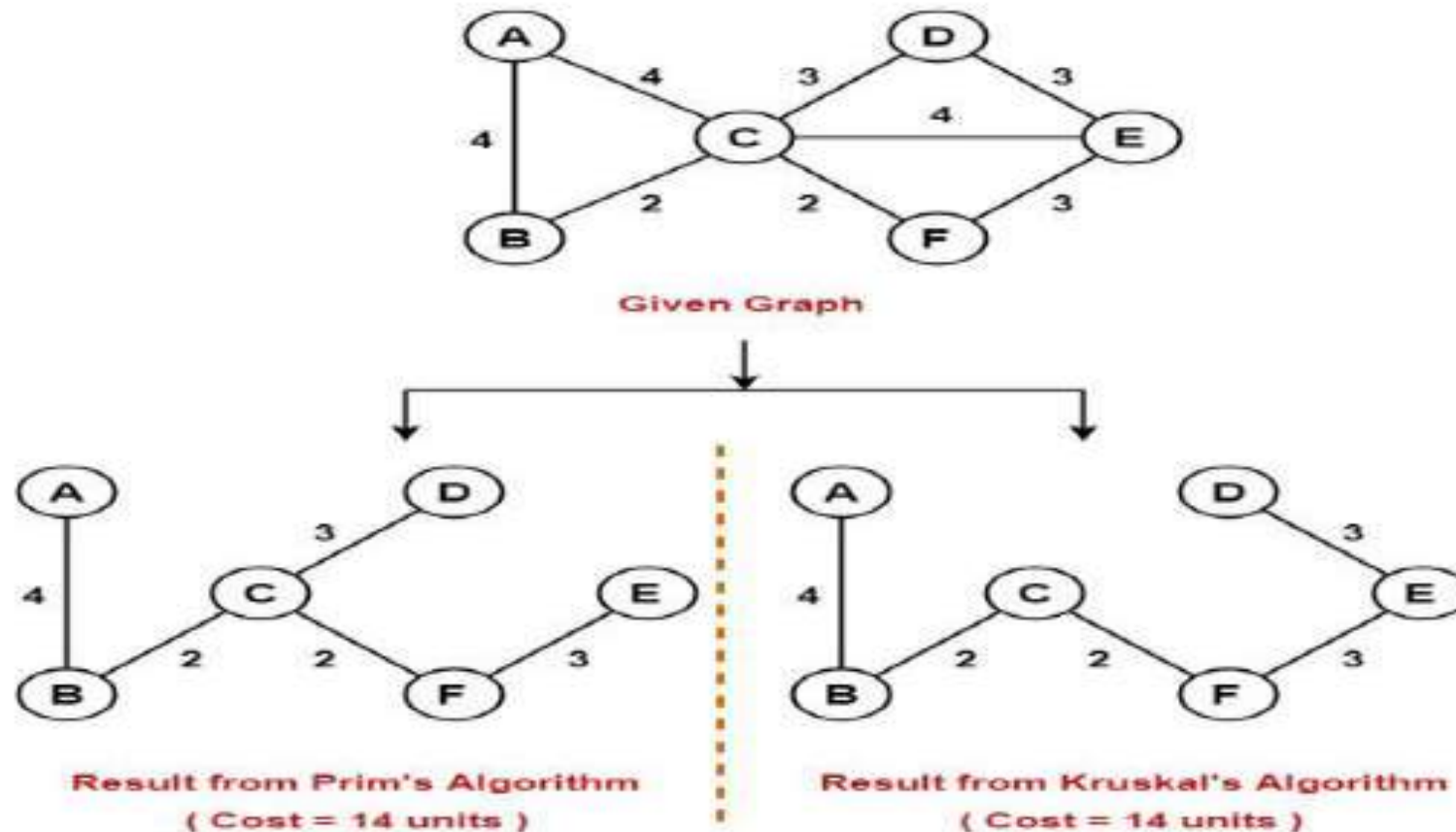
Case-02: When all the edge weights are not distinct-

Case-02: When all the edge weights are not distinct-

If all the edge weights are not distinct, then both the algorithms may not always produce the same i.e. unique MST but the cost of the MST produced would always be same in both the cases.

Example-

Consider the following example-



Differences



Kruskal's Algorithms	Prim's Algorithm
1. Select the shortest edge in a network 2. Select the next shortest edge which does not create a cycle 3. Repeat step 2 until all vertices have been connected 4. Kruskal's Begins with forest and merge into tree.	1. Select any vertex 2. Select the shortest edge connected to that vertex 3. Select the shortest edge connected to any vertex already connected 4. Repeat step 3 until all vertices have been connected 5. Prim's always stays as a tree.
<u>Complexity</u> $O(N \log N)$ comparison sort for edges.	<u>Complexity</u> $O(N \log N)$ search the least weight edge for every vertices.

SUMMARY

- A spanning tree is a subset of Graph G , which has all the vertices covered with minimum possible number of edges. Hence, a spanning tree does not have cycles and it cannot be disconnected..
- By this definition, we can draw a conclusion that every connected and undirected Graph G has at least one spanning tree. A disconnected graph does not have any spanning tree, as it cannot be spanned to all its vertices.
- Prim's and Kruskal's algorithm are used to construct the minimal spanning tree for a given graph topology.

SELF-ASSESSMENT QUESTIONS

Which data structure is commonly used to implement Prim's algorithm?

- (a) Array
- (b) Linked List
- (c) **Heap**
- (d) Stack

In Kruskal's algorithm, edges are added to the spanning tree in which order?

- (a) Decreasing order of weights.
- (b) Random order.
- (c) **Non-decreasing order of weights**
- (d) Alphabetical order.

TERMINAL QUESTIONS

1. Compare and contrast Prim's algorithm and Kruskal's algorithm for finding the MST of a weighted graph.
1. Explain the concept of cycle detection in the context of MST algorithms. Why is it important to avoid cycles when constructing a minimum spanning tree, and how do Prim's and Kruskal's algorithms ensure acyclic structures?

REFERENCES FOR FURTHER LEARNING OF THE SESSION

Reference Books :

1. Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein., 3rd, 2009, The MIT Press.
- 2 Algorithm Design Manual, Steven S. Skiena., 2nd, 2008, Springer.
- 3 Data Structures and Algorithms in Python, Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser., 2nd, 2013, Wiley.
- 4 The Art of Computer Programming, Donald E. Knuth, 3rd, 1997, Addison-Wesley Professiona.

MOOCS :

1. <https://www.coursera.org/specializations/algorithms?=>
2. <https://www.coursera.org/learn/dynamic-programming-greedy-algorithms#modules>

THANK YOU

Advanced Algorithms & Data Structures



Department of CSE

ADVANCED ALGORITHMS AND DATA STRUCTURES 23CS03HF

Topic:

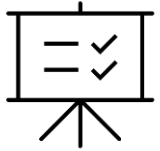
Single Source Shortest Path Problem

AIM OF THE SESSION



To familiarize students with the concept of Single Source Shortest Path Problem

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

1. Demonstrate :- Single Source Shortest Path Problem.
2. Describe :- solve single-source shortest path problem using Dijkstra's Algorithm .

LEARNING OUTCOMES



At the end of this session, you should be able to:

1. Define :- Single Source Shortest path Problem.
2. Describe :- solve single-source shortest path problem using Dijkstra's Algorithm
3. Summarize:- Finding the shortest paths from a single source vertex to all other vertices in a weighted graph.

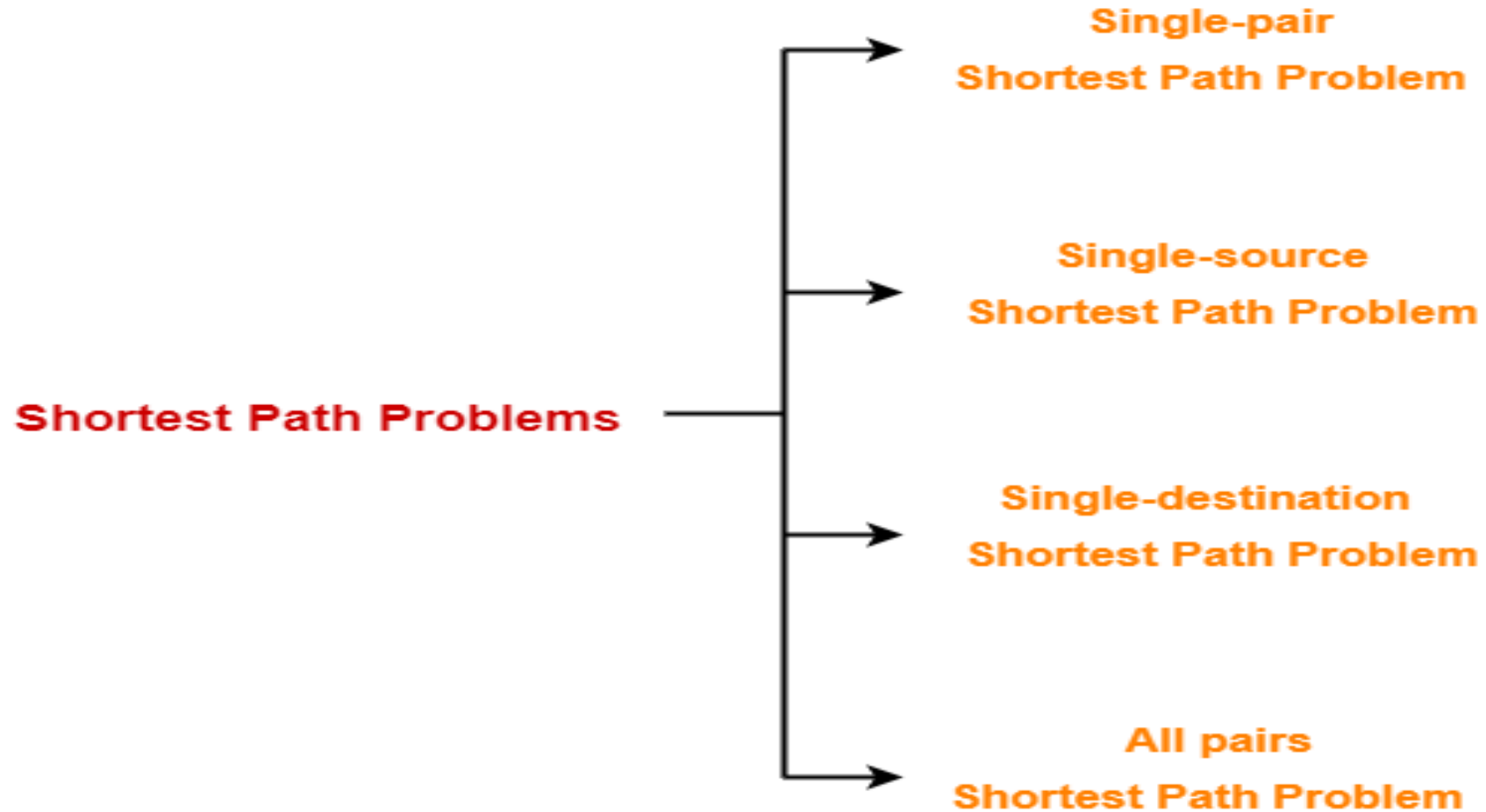
Shortest Path Problem

- Shortest path problem is a problem of finding the shortest path(s) between vertices of a given graph.
- Shortest path between two vertices is a path that has the least cost as compared to all other existing paths.

Applications

- Google Maps
- Road Networks
- Logistics Research

Types of Shortest Path Problem-



Single Source Shortest Path Problem

- It is a shortest path problem where the shortest path from a given source vertex to all other remaining vertices is computed.
- **Dijkstra's Algorithm** and Bellman Ford Algorithm are the famous algorithms used for solving single-source shortest path problem.

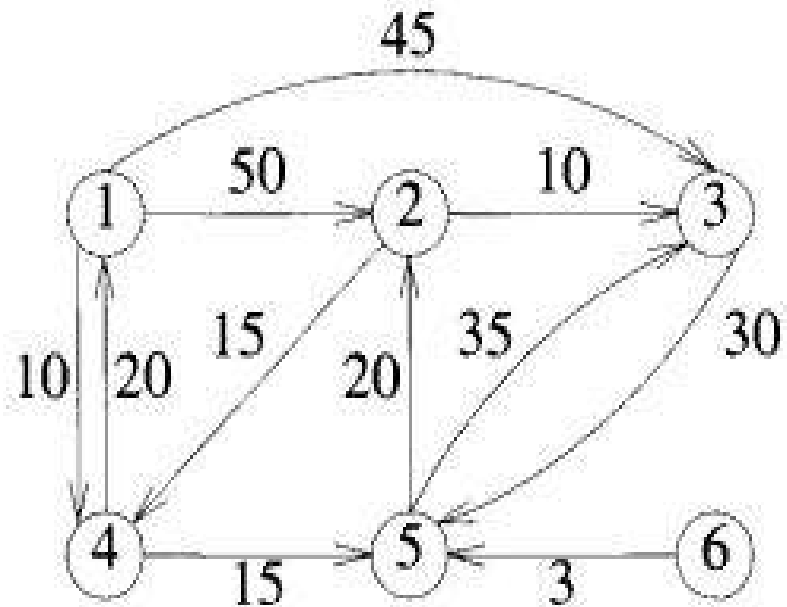
Dijkstra Algorithm

- Dijkstra Algorithm is a very famous greedy algorithm.
- It is used for solving the single source shortest path problem.
- It computes the shortest path from one particular source node to all other remaining nodes of the graph.

Conditions

It is important to note the following points regarding Dijkstra Algorithm-

- Dijkstra algorithm works only for **connected graphs**.
- Dijkstra algorithm works only for those graphs that **do not contain any negative weight edge**.
- The actual Dijkstra algorithm **does not output the shortest paths**.
- It only provides the **value or cost of the shortest paths**.
- Dijkstra algorithm works for **directed as well as undirected graphs**.



(a) Graph

<i>Path</i>	<i>Length</i>
1) 1, 4	10
2) 1, 4, 5	25
3) 1, 4, 5, 2	45
4) 1, 3	45

(b) Shortest paths from 1

Implementation

Source Vertex	Intermediate Vertex	dist[2]	dist[3]	dist[4]	dist[5]	dist[6]	Path	Distance
1	-	50	45	<u>10</u>	∞	∞	1 $\rightarrow$ 4	10
	4	50	45	-	<u>25</u>	∞	1 $\rightarrow$ 4 $\rightarrow$ 5	25
	5	<u>45</u>	45	-	-	∞	1 $\rightarrow$ 4 $\rightarrow$ 5 $\rightarrow$ 2	45
	2	-	<u>45</u>	-	-	∞	1 $\rightarrow$ 3	45
	3	-	-	-	-	∞	No path to 6	

```
1  Algorithm ShortestPaths( $v, cost, dist, n$ )
2  //  $dist[j]$ ,  $1 \leq j \leq n$ , is set to the length of the shortest
3  // path from vertex  $v$  to vertex  $j$  in a digraph  $G$  with  $n$ 
4  // vertices.  $dist[v]$  is set to zero.  $G$  is represented by its
5  // cost adjacency matrix  $cost[1 : n, 1 : n]$ .
6  {
7      for  $i := 1$  to  $n$  do
8      { // Initialize  $S$ .
9           $S[i] := \text{false}$ ;  $dist[i] := cost[v, i]$ ;
10     }
11      $S[v] := \text{true}$ ;  $dist[v] := 0.0$ ; // Put  $v$  in  $S$ .
12     for  $num := 2$  to  $n - 1$  do
13     {
14         // Determine  $n - 1$  paths from  $v$ .
15         Choose  $u$  from among those vertices not
16         in  $S$  such that  $dist[u]$  is minimum;
17          $S[u] := \text{true}$ ; // Put  $u$  in  $S$ .
18         for (each  $w$  adjacent to  $u$  with  $S[w] = \text{false}$ ) do
19             // Update distances.
20             if ( $dist[w] > dist[u] + cost[u, w]$ ) then
21                  $dist[w] := dist[u] + cost[u, w]$ ;
22     }
23 }
```

Time Complexity Analysis-

- The given graph G is represented as an adjacency matrix.
- Priority queue Q is represented as an unordered list.

Here,

- $A[i,j]$ stores the information about edge (i,j) .
- Time taken for selecting i with the smallest dist is $O(V)$.
- For each neighbor of i , time taken for updating $\text{dist}[j]$ is $O(1)$ and there will be maximum V neighbors.
- Time taken for each iteration of the loop is $O(V)$ and one vertex is deleted from Q .
- Thus, total time complexity becomes $O(V^2)$.

- **Objective:** Find the shortest paths from a single source vertex to all other vertices in a weighted graph.
- **Approach:** Initialize distances from the source to all vertices as infinity, update distances using relaxation to minimize path weights, and repeat until all shortest paths are found.
- **Algorithmic Solutions:** Algorithms like Dijkstra's (for non-negative weights) used to solve this problem efficiently.

SELF-ASSESSMENT QUESTIONS

Which of the following conditions must be true for Dijkstra's algorithm to work correctly?

- (a) The graph must be directed.
- (b) The graph must be undirected.
- (c) The graph must have non-negative edge weights.
- (d) The graph must have no cycles.

What is the main application of single source shortest path algorithms?

- (a) Finding Minimum Spanning Tree
- (b) Finding shortest paths from a single source node to all other nodes
- (c) Detecting cycles in a graph
- (d) Sorting nodes in topological order

TERMINAL QUESTIONS

1. Can you explain the basic architecture of Dijkstra's algorithm along with an example?
2. What are some real-world scenarios where Dijkstra's algorithm can be applied effectively?

REFERENCES FOR FURTHER LEARNING OF THE SESSION

Reference Books :

1. Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein., 3rd, 2009, The MIT Press.
- 2 Algorithm Design Manual, Steven S. Skiena., 2nd, 2008, Springer.
- 3 Data Structures and Algorithms in Python, Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser., 2nd, 2013, Wiley.
- 4 The Art of Computer Programming, Donald E. Knuth, 3rd, 1997, Addison-Wesley Professiona.

MOOCS :

1. <https://www.coursera.org/specializations/algorithms?=>
2. <https://www.coursera.org/learn/dynamic-programming-greedy-algorithms#modules>

THANK YOU

Advanced Algorithms & Data Structures



Department of CSE

ADVANCED ALGORITHMS AND DATA STRUCTURES 23CS03HF

Topic:

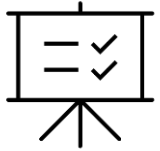
Huffman Coding

AIM OF THE SESSION



To familiarize students with the concept of Huffman Coding

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

1. Demonstrate :- Huffman Coding.
2. Describe :- Concept of Encoding and Compression of data for data transmission using Huffman coding

LEARNING OUTCOMES



At the end of this session, you should be able to:

1. Define :- Huffman Coding.
2. Describe :- Concept of Encoding and Compression of data for data transmission using Huffman coding.
3. Summarize:- Expected to know the importance of encoding and decoding using Huffman coding.

Encoding and Compression of Data

- **Compression:**

- Data Compression, shrinks down a String so that it takes up less space. This is desirable for data storage and data communication.
- Encoding means converting the String into Binary codes

- **Decompression:**

In the decompression we convert the Binary codes into the Original string.

Fixed & Variable length code

- **Fixed length code:**

If every word in the code has the same length, the code is called a fixed-length code, or a block code.

Advantage	Disadvantage
Access is fast because the computer knows where each Word starts	Using Fixed length records, the records are usually larger and therefore need more storage space and are slower to transfer

- **Variable length code:**

Variable-length codes can perform significantly better as frequent characters are given short code words, while infrequent characters get longer code words.

Advantage	Disadvantages
Variable-length codes over fixed length is short codes that can be given to characters that occur frequently.	Where a character ends and another begins, difficult to identify.

Prefix Property

- A code has the prefix property if no character code is the prefix (start of the code) for another character
- Example:

Symbol	Code
P	000
Q	11
R	01
S	001
T	10

01001101100010

R S T Q P T

000 is not a prefix of 11, 01, 001, or 10

11 is not a prefix of 000, 01, 001, or 10

Huffman Coding

- Developed by **David Huffman** in 1951
- Huffman Coding is a famous Greedy Algorithm.
- It is used for the lossless compression of data.
- It uses variable length encoding.
- It assigns variable length code to all the characters.
- The code length of a character depends on how frequently it occurs in the given text.
- The character which occurs most frequently gets the smallest code.
- The character which occurs least frequently gets the largest code.
- It is also known as **Huffman Encoding**.

Prefix Rule

1. Huffman Coding implements a rule known as a prefix rule.
2. This is to prevent the ambiguities while decoding.
3. It ensures that the code assigned to any character is not a prefix of the code assigned to any other character.

Major Steps in Huffman Coding-

There are two major steps in Huffman Coding-

1. Building a Huffman Tree from the input characters.
2. Assigning code to the characters by traversing the Huffman Tree.

Huffman Tree

The steps involved in the construction of Huffman Tree are as follows-

Step-01:

- Create a leaf node for each character of the text.
- Leaf node of a character contains the occurring frequency of that character

Step-02:

- Arrange all the nodes in increasing order of their frequency value.

Step-03:

- Considering the first two nodes having minimum frequency,
- Create a new internal node.
- The frequency of this new node is the sum of frequency of those two nodes.
- Make the first node as a left child and the other node as a right child of the newly created node.

Step-04:

- Keep repeating Step-02 and Step-03 until all the nodes form a single tree.
- The tree finally obtained is the desired Huffman Tree.

Important Formulas-

The following 2 formulas are
important to solve the problems based on Huffman Coding-

Formula-01:

$$\begin{aligned}\text{Average code length per character} &= \frac{\sum (\text{frequency}_i \times \text{code length}_i)}{\sum \text{frequency}_i} \\ &= \sum (\text{probability}_i \times \text{code length}_i)\end{aligned}$$

Formula-02:

- Total number of bits in Huffman encoded message
- = Total number of characters in the message x Average code length per character
- = $\sum (\text{frequency}_i \times \text{Code length}_i)$

PRACTICE PROBLEM BASED ON HUFFMAN CODING

Problem-

A file contains the following characters with the frequencies as shown. If Huffman Coding is used for data compression, determine-

1. Huffman Code for each character
2. Average code length
3. Length of Huffman encoded message (in bits)

Characters	Frequencies
a	10
e	15
i	12
o	3
u	4
s	13
t	1

Solution

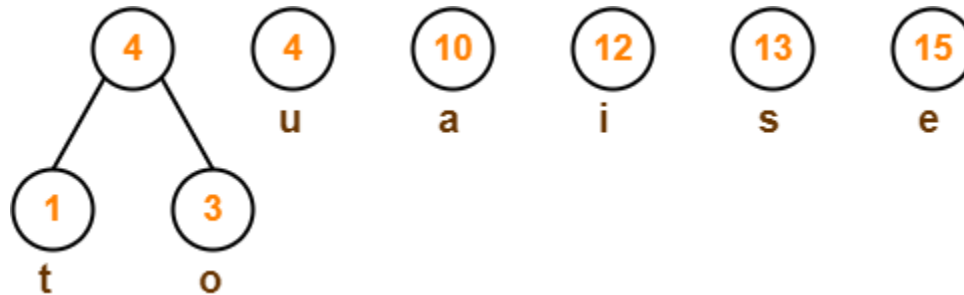
First let us construct the Huffman Tree.

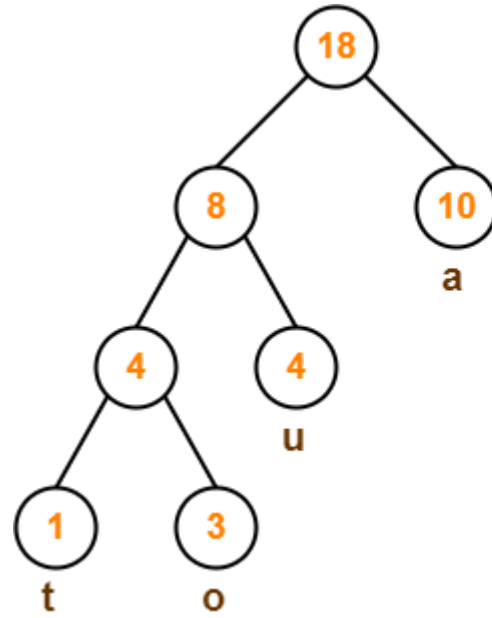
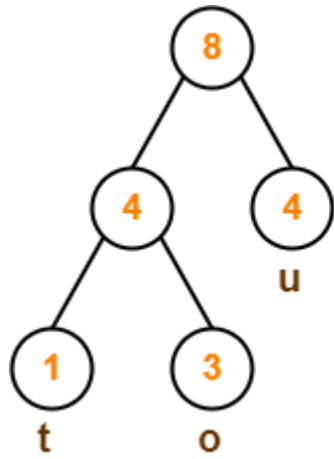
- Huffman Tree is constructed in the following steps-

Step-01:

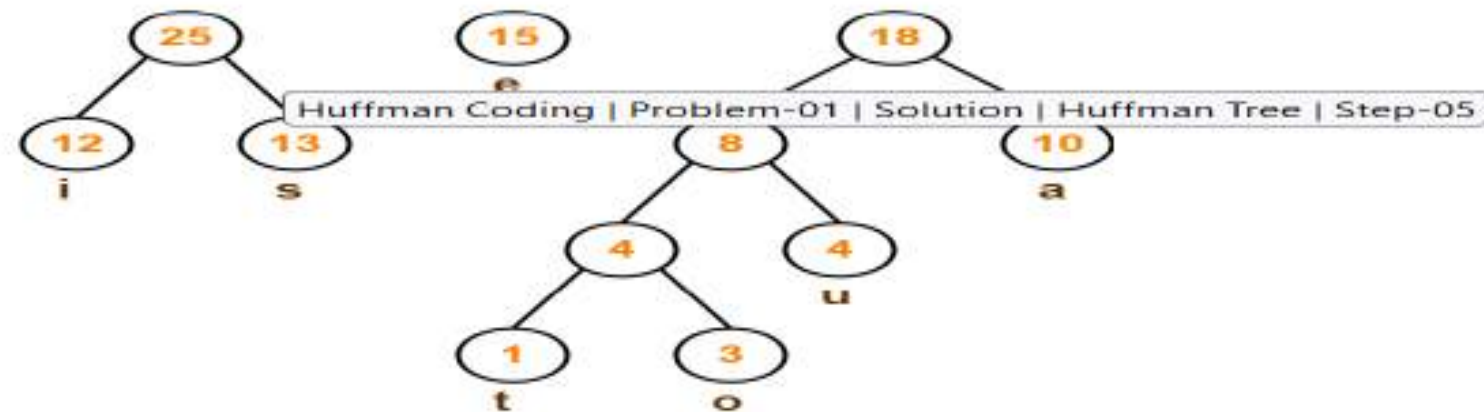
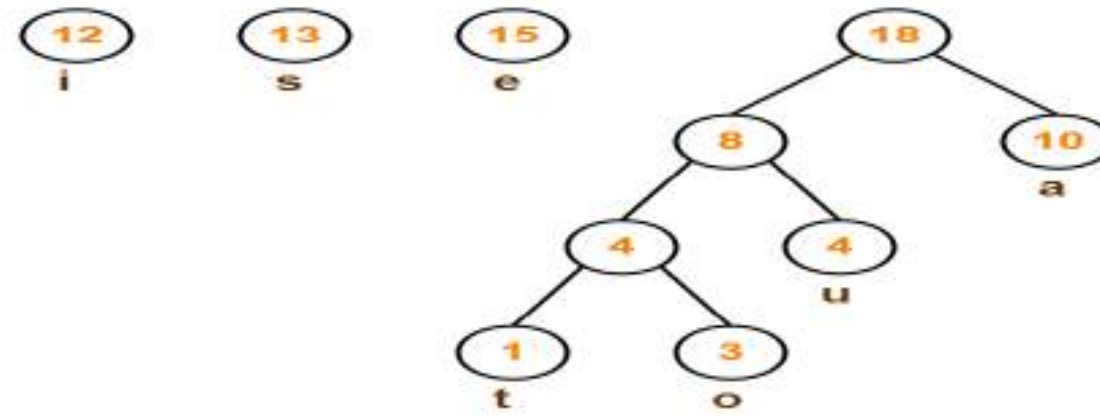


Step-02:

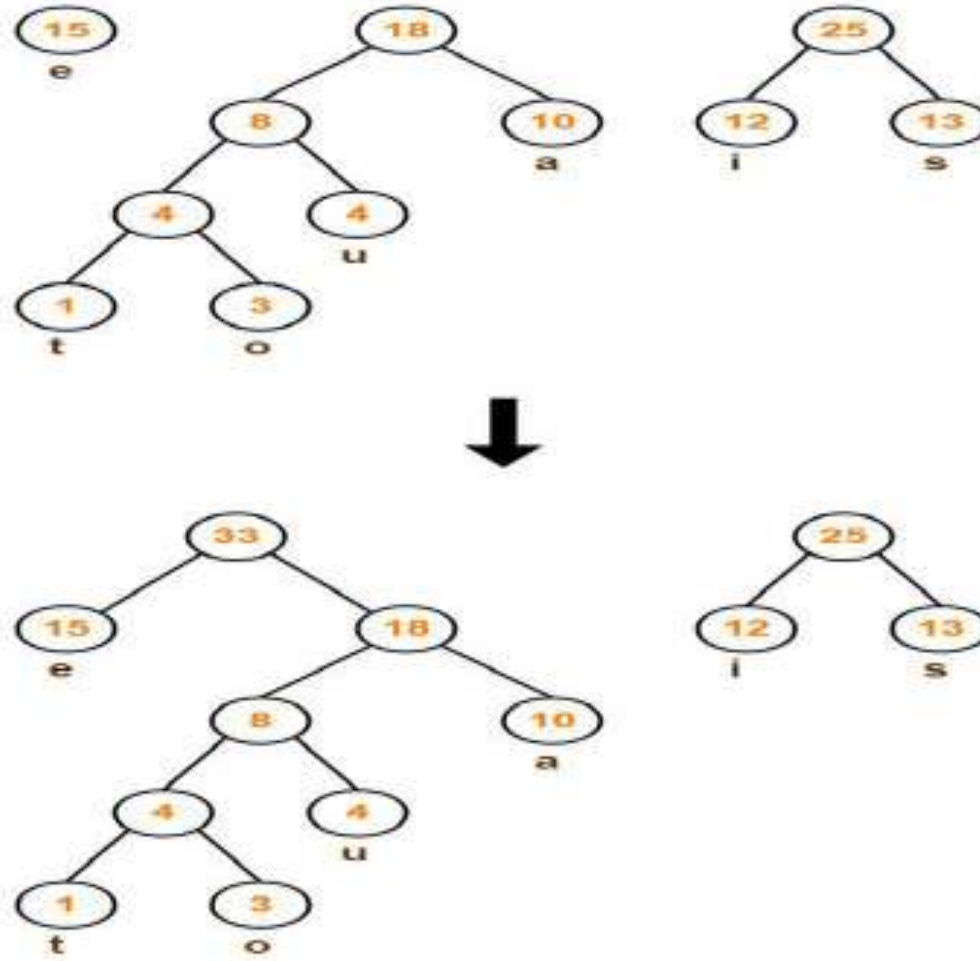




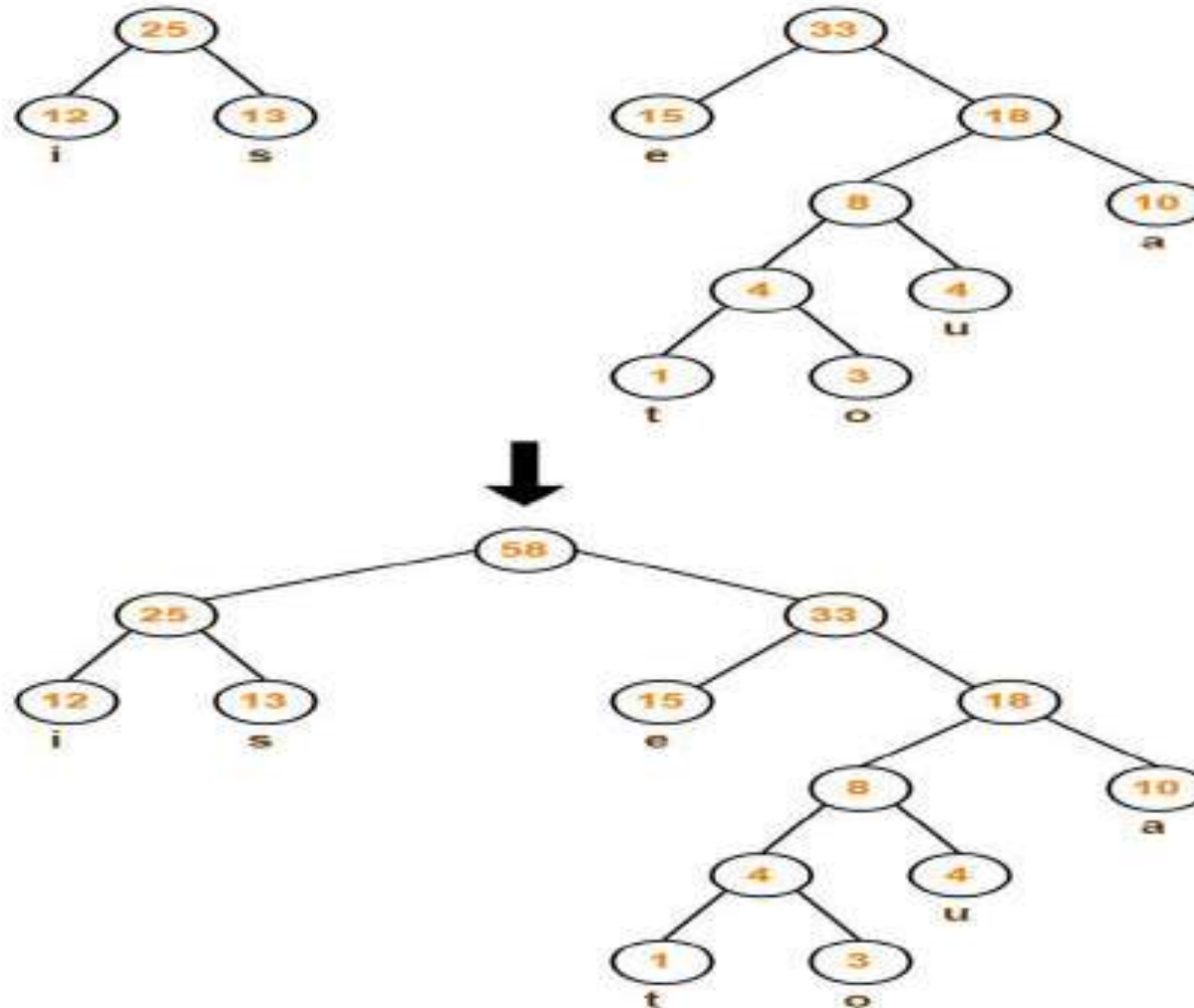
Step-05:



Step-06:



Step-07:



Huffman Tree

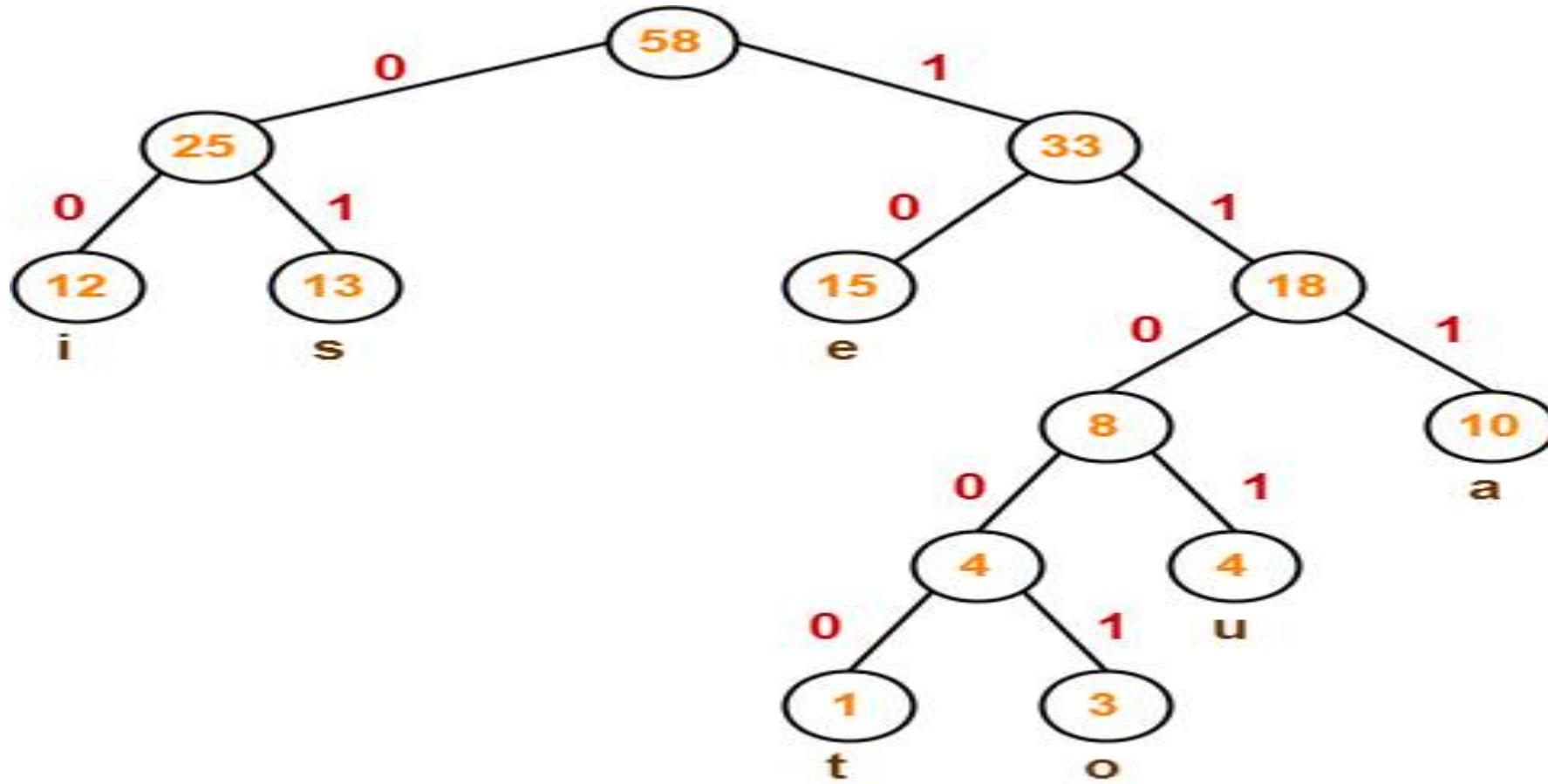
Now,

- We assign weight to all the edges of the constructed Huffman Tree.
- Let us assign weight '0' to the left edges and weight '1' to the right edges

Rule

- If you assign weight '0' to the left edges, then assign weight '1' to the right edges.
- If you assign weight '1' to the left edges, then assign weight '0' to the right edges.
- Any of the above two conventions may be followed.
- But follow the same convention at the time of decoding that is adopted at the time of encoding.

After assigning weight to all the edges, the modified Huffman Tree is-



Huffman Tree

Now, let us answer each part of the given problem one by one-

Huffman Code For Characters

To write Huffman Code for any character, traverse the Huffman Tree from root node to the leaf node of that character.

Following this rule, the Huffman Code for each character is-

- a = 111
- e = 10
- i = 00
- o = 11001
- u = 1101
- s = 01
- t = 11000

From here, we can observe-

- Characters occurring less frequently in the text are assigned the larger code.
- Characters occurring more frequently in the text are assigned the smaller code.

2. Average Code Length-

Using formula-01, we have-

Average code length

$$= \sum (\text{frequency}_i \times \text{code length}_i) / \sum (\text{frequency}_i)$$

$$= \{ (10 \times 3) + (15 \times 2) + (12 \times 2) + (3 \times 5) + (4 \times 4) + (13 \times 2) + (1 \times 5) \} / (10 + 15 + 12 + 3 + 4 + 13 + 1)$$

$$= 2.52$$

3. Length of Huffman Encoded Message

Using formula-02, we have-

Total number of bits in Huffman encoded message

= Total number of characters in the message x Average code length per character

= 58×2.52

= 146.16

$\cong 147$ bits

Applications

- Huffman coding is a technique used to compress files for transmission
- Uses statistical coding
 - more frequently used symbols have shorter code words
- Works well for text and fax transmissions
- An application that uses several data structures

Huffman coding is a compression technique that assigns variable-length codes to symbols based on their frequencies, achieving efficient data storage by minimizing average code length and enabling straightforward decoding using a constructed binary tree.

SELF-ASSESSMENT QUESTIONS

What type of tree is used in Huffman coding?

- (a) AVL Tree
- (b) Red-Black Tree
- (c) Binary Heap
- (d) Binary Tree**

Huffman coding is primarily used for

- (a) Data Encryption
- (b) Data Compression**
- (c) Data Decompression
- (d) Data Transmission

TERMINAL QUESTIONS

1. Explain about coding and de-coding
2. What is compression

REFERENCES FOR FURTHER LEARNING OF THE SESSION

Reference Books :

1. Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein., 3rd, 2009, The MIT Press.
- 2 Algorithm Design Manual, Steven S. Skiena., 2nd, 2008, Springer.
- 3 Data Structures and Algorithms in Python, Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser., 2nd, 2013, Wiley.
- 4 The Art of Computer Programming, Donald E. Knuth, 3rd, 1997, Addison-Wesley Professiona.

MOOCS :

1. <https://www.coursera.org/specializations/algorithms?=>
2. <https://www.coursera.org/learn/dynamic-programming-greedy-algorithms#modules>

THANK YOU

Advanced Algorithms & Data Structures



Department of CSE

ADVANCED ALGORITHMS AND DATA STRUCTURES 23CS03HF

Topic:

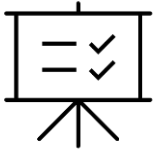
Dynamic Programming

AIM OF THE SESSION



To familiarize students with the concept of Dynamic Programming.

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

1. Demonstrate :- Dynamic programming and its elements.
2. Describe :- Steps that the dynamic programming follows.

LEARNING OUTCOMES



At the end of this session, you should be able to:

1. Define :- Dynamic programming and its elements.
2. Describe :- Principles of optimality
3. Summarize:- top-down and bottom-up approaches of dynamic programming.

Dynamic Programming

- **Dynamic programming is a technique that breaks the problems into sub-problems, and saves the result for future purposes so that we do not need to compute the result again.**
- **The main use of dynamic programming is to solve optimization problems. Here, optimization problems mean that when we are trying to find out the minimum or the maximum solution of a problem.**
- **The dynamic programming guarantees to find the optimal solution of a problem if the solution exists.**

Elements of Dynamic Programming

DP is used to solve problems with the following characteristics:

- **Simple subproblems**

- We should be able to break the original problem to smaller subproblems that have the same structure

- **Optimal substructure of the problems**

- The optimal solution to the problem contains within optimal solutions to its subproblems.

- **Overlapping sub-problems**

- It is mainly used where the solution of one sub-problem is needed repeatedly. The computed solutions are stored in a table, so that these don't have to be re-computed. Hence, this technique is needed where overlapping sub-problem exists.

Principle of optimality

- The dynamic Programming works on a principle of optimality.
- The principle of optimality states that each sub-sequence must also be optimal in an optimal sequence of decisions or choices.

Example

Consider an example of the Fibonacci series. The following series is the Fibonacci series:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ,...

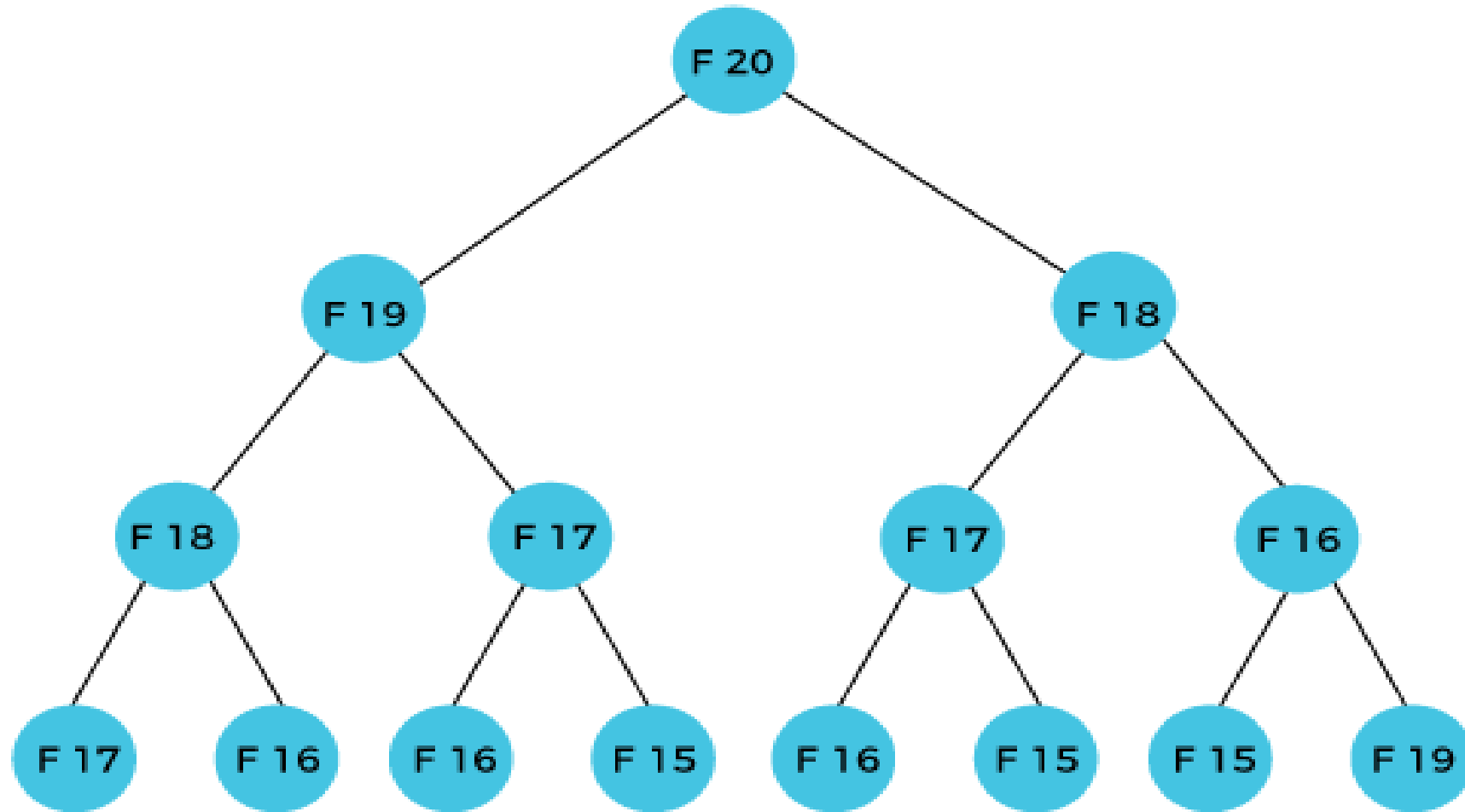
The numbers in the above series are not randomly calculated.

Mathematically, we could write each of the terms using the below formula:

$$F(n) = F(n-1) + F(n-2),$$

How can we calculate $F(20)$?

The $F(20)$ term will be calculated using the n th formula of the Fibonacci series. The below figure shows how $F(20)$ is calculated.



Approach of work

The following are the steps that the dynamic programming follows:

- **It breaks down the complex problem into simpler subproblems.**
- **It finds the optimal solution to these sub-problems.**
- **It stores the results of subproblems (memorization). The process of storing the results of subproblems is known as memorization.**
- **It reuses them so that the same sub-problem is calculated more than once.**
- **Finally, calculate the result of the complex problem.**

Approach of DP

There are two approaches to dynamic programming:

- 1. Top-down approach**
- 2. Bottom-up approach**

Top-Down approach

The top-down approach follows the memorization technique, while the bottom-up approach follows the tabulation method. Here memorization is equal to the sum of recursion and caching. Recursion means calling the function itself, while caching means storing the intermediate results.

Advantages

- **It is very easy to understand and implement.**
- **It solves the subproblems only when it is required.**
- **It is easy to debug.**

Disadvantages

- **It uses the recursion technique that occupies more memory in the call stack.**
Sometimes when the recursion is too deep, the stack overflow condition will occur.
- **It occupies more memory degrades the overall performance**

```
int fib(int n)
{
    if(n<0)
        error;
    if(n==0)
        return 0;
    if(n==1)
        return 1;
    sum = fib(n-1) + fib(n-2);
}
```

In the above code, we have used the recursive approach to find out the Fibonacci series. When the value of 'n' increases, the function calls will also increase, and computations will also increase. In this case, the time complexity increases exponentially, and it becomes 2^n .

One solution to this problem is to use the dynamic programming approach. Rather than generating the recursive tree again and again, we can reuse the previously calculated value. If we use the dynamic programming approach, then the time complexity would be $O(n)$.

Bottom-Up approach

- The bottom-up approach is also one of the techniques which can be used to implement dynamic programming.
- It uses the tabulation technique to implement the dynamic programming approach. It solves the same kind of problems but it removes the recursion.
- In this tabulation technique, we solve the problems and store the results in a matrix.
- The bottom-up is the approach used to avoid the recursion, thus saving the memory space.
- In the bottom-up approach, we start from the base case to find the answer for the end. As we know, the base cases in the Fibonacci series are 0 and 1. Since the bottom approach starts from the base cases, so we will start from 0 and 1.

Key points

- We solve all the smaller sub-problems that will be needed to solve the larger sub-problems then move to the larger problems using smaller sub-problems.
- We use a for loop to iterate over the sub-problems.
- The bottom-up approach is also known as the tabulation or table-filling method.

Let's understand through an example.

Suppose we have an array that has 0 and 1 values at $a[0]$ and $a[1]$ positions, respectively shown as below:

0	1
$a[0]$	$a[1]$

Since the bottom-up approach starts from the lower values, so the values at $a[0]$ and $a[1]$ are added to find the value of $a[2]$ shown as below:

0	1	1
$a[0]$	$a[1]$	$a[2]$

The value of $a[3]$ will be calculated by adding $a[1]$ and $a[2]$, and it becomes 2 shown as below:

0	1	1	2
$a[0]$	$a[1]$	$a[2]$	$a[3]$

The value of $a[4]$ will be calculated by adding $a[2]$ and $a[3]$, and it becomes 3 shown as below:

0	1	1	2	3
A[0]	A[1]	A[2]	A[3]	A[4]

The value of $a[5]$ will be calculated by adding the values of $a[4]$ and $a[3]$, and it becomes 5 shown as below:

0	1	1	2	3	5
A[0]	A[1]	A[2]	A[3]	A[4]	A[5]

The code for implementing the Fibonacci series using the bottom-up approach is given below:

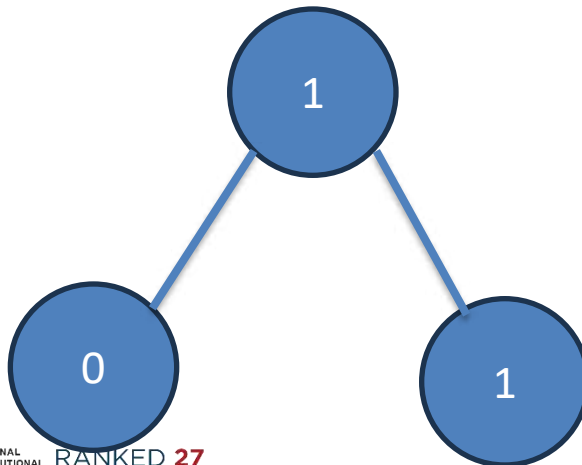
```
int fib(int n)
{
    int A[];
    A[0] = 0, A[1] = 1;
    for( i=2; i<=n; i++)
    {
        A[i] = A[i-1] + A[i-2]
    }
    return A[n];
}
```

In the above code, base cases are 0 and 1 and then we have used for loop to find other values of Fibonacci series.

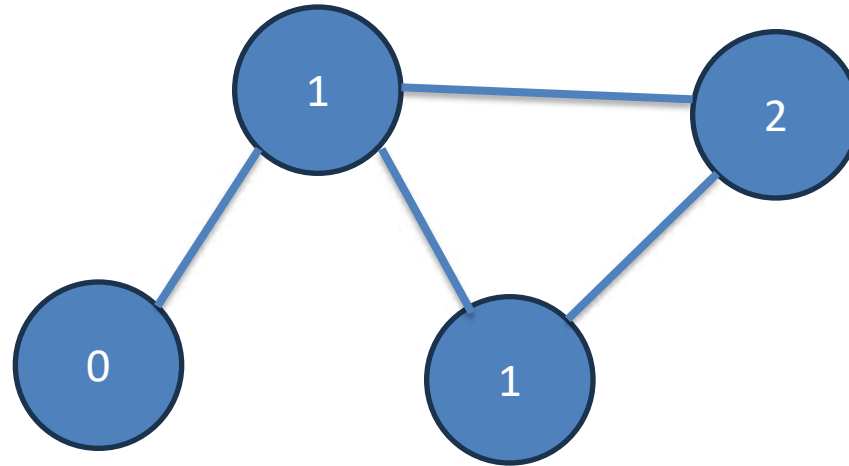
Let's understand through the diagrammatic representation.
Initially, the first two values, i.e., 0 and 1 can be represented as:



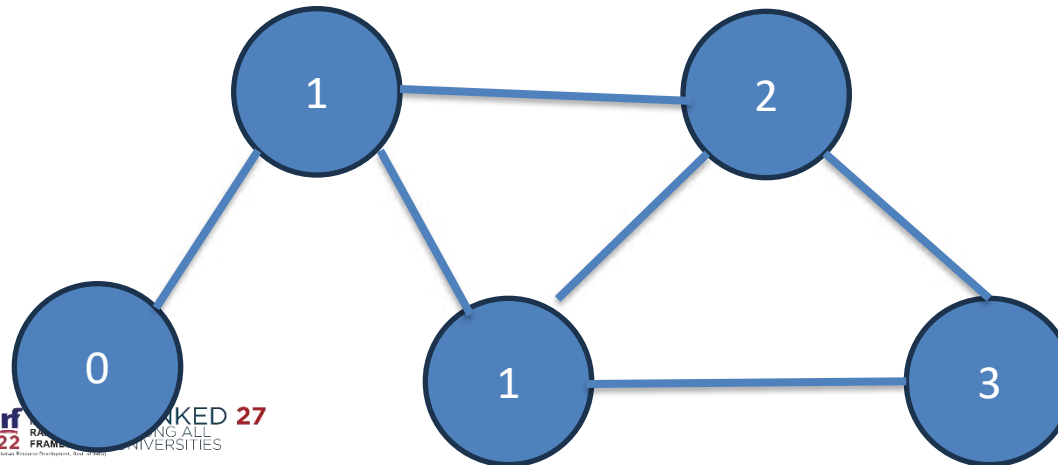
When $i=2$ then the values 0 and 1 are added shown as below:



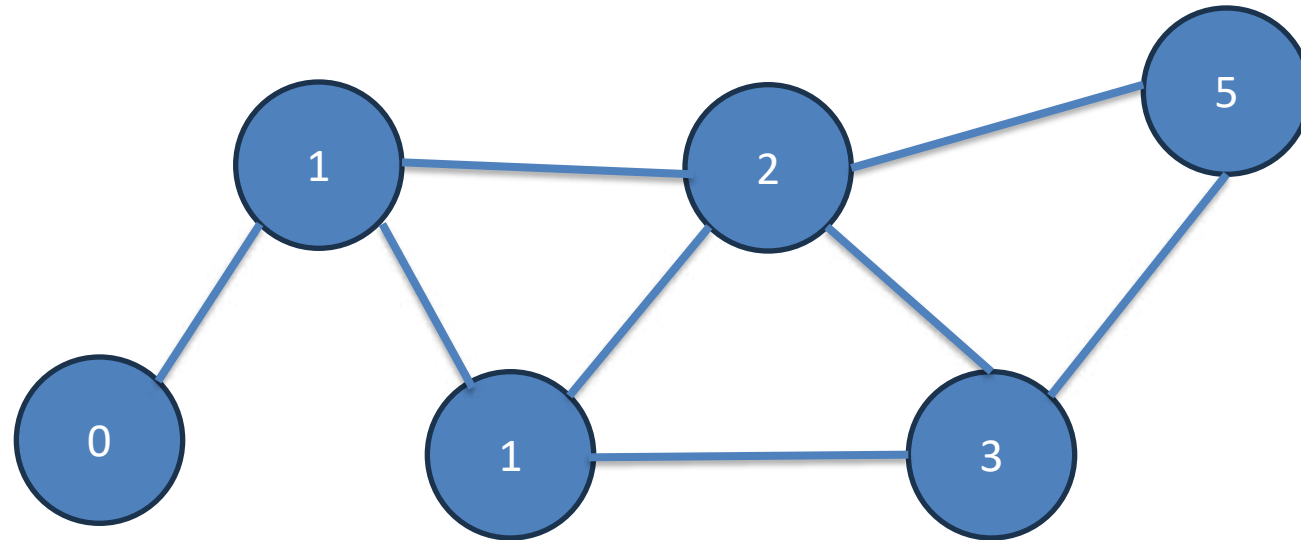
When $i=3$ then the values of 1 and 1 are added shown as below:



When $i=4$ then the values 2 and 1 are added shown as below:



When $i=5$ then the values of 3 and 2 are added shown as below:



In the above case, we are starting from the bottom and reaching to the top.

Divide & Conquer

1. Partitions a problem into independent smaller sub-problems
2. Doesn't store solutions of sub-problems. (Identical sub-problems may arise results in the same computations being performed repeatedly)
3. Top-down algorithms: which logically progresses from the initial instance down to the smallest sub-instances via intermediate sub-instances.

Dynamic Programming

1. Partitions a problem into overlapping sub-problems
2. Stores solutions of sub-problems: thus avoids calculations of same quantity twice
3. Bottom-up algorithms: in which the smallest sub-problems are explicitly solved first and the results of these are used to construct solutions to progressively larger sub-instances

DP vs Greedy method

Dynamic Programming

1. Dynamic Programming is used to obtain the optimal solution.
2. In Dynamic Programming, we choose at each step, but the choice may depend on the solution to sub-problems.
3. It is guaranteed that Dynamic Programming will generate an optimal solution using the Principle of Optimality.
4. Example: 0/1 Knapsack

Greedy Method

1. Greedy Method is also used to get the optimal solution.
2. In a greedy Algorithm, we make whatever choice seems best at the moment and then solve the sub-problems arising after the choice is made.
3. In the Greedy Method, there is no such guarantee of getting Optimal Solution
4. Example: Fractional Knapsack

Examples

- 0/1 Knapsack Method
- Traveling sales person Problem
- Optimal Binary search tree
- Matrix chain multiplication
- Longest Common Sequence

- **Dynamic programming is a technique that breaks the problems into sub-problems, and saves the result for future purposes so that we do not need to compute the result again.**
- **The main use of dynamic programming is to solve optimization problems. Here, optimization problems mean that when we are trying to find out the minimum or the maximum solution of a problem.**
- **The dynamic programming guarantees to find the optimal solution of a problem if the solution exists.**

SELF-ASSESSMENT QUESTIONS

1. Which of the following is/are property/properties of a dynamic programming problem?

- (a) Optimal substructure
- (b) Overlapping subproblems
- (c) Greedy approach
- (d) Both optimal substructure and overlapping subproblems

2. If a problem can be solved by combining optimal solutions to non-overlapping problems, the strategy is called

- (a) Dynamic programming
- (b) Greedy
- (c) Divide and conquer
- (d) Recursion

TERMINAL QUESTIONS

- 1. What is the dynamic programming question?**
- 2. Which problem is solved by dynamic programming?**
- 3. What are the two methods of dynamic programming?**
- 4. What are the applications of dynamic programming?**
- 5. What are the characteristics of dynamic programming?**

REFERENCES FOR FURTHER LEARNING OF THE SESSION

Reference Books :

1. Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein., 3rd, 2009, The MIT Press.
- 2 Algorithm Design Manual, Steven S. Skiena., 2nd, 2008, Springer.
- 3 Data Structures and Algorithms in Python, Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser., 2nd, 2013, Wiley.
- 4 The Art of Computer Programming, Donald E. Knuth, 3rd, 1997, Addison-Wesley Professional.

MOOCS :

1. <https://www.coursera.org/specializations/algorithms?=>
2. <https://www.coursera.org/learn/dynamic-programming-greedy-algorithms#modules>

THANK YOU

Advanced Algorithms & Data Structures



Department of CSE

ADVANCED ALGORITHMS AND DATA STRUCTURES 23CS03HF

Topic:

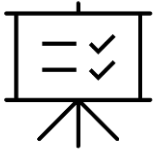
Matrix Chain Multiplication

AIM OF THE SESSION



To familiarize students with the concept of Matrix Chain Multiplication

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

1. Demonstrate :- Matrix Chain Multiplication
2. Describe :- Solving of Matrix Chain Multiplication using Dynamic Programming

LEARNING OUTCOMES



At the end of this session, you should be able to:

1. Define :- Matrix Chain Multiplication
2. Describe :- Solving of Matrix Chain Multiplication using Dynamic Programming
3. Summarize:-Procedure of Matrix Chain Multiplication

Matrix Chain Multiplication

- Given some matrices to multiply, determine the **best** order to multiply them so you minimize the number of single element multiplications.
i.e. Determine the way the matrices are parenthesized.
- First off, it should be noted that matrix multiplication is associative, but not commutative. But since it is associative, we always have:
- $((AB)(CD)) = (A(B(CD)))$, or any other grouping as long as the matrices are in the same consecutive order.
- BUT NOT: $((AB)(CD)) = ((BA)(DC))$

- It may appear that the amount of work done won't change if you change the parenthesization of the expression, but we can prove that is not the case!
- Let us use the following example:
Let A be a 2×10 matrix
Let B be a 10×50 matrix
Let C be a 50×20 matrix
- But FIRST, let's review some matrix multiplication rules...

- Let's get back to our example: We will show that the way we group matrices when multiplying A, B, C *matters*:
Let A be a 2x10 matrix
Let B be a 10x50 matrix
Let C be a 50x20 matrix
- Consider computing **A(BC)**:
multiplications for (BC) = $10 \times 50 \times 20 = 10000$, creating a 10x20 answer matrix
multiplications for A(BC) = $2 \times 10 \times 20 = 400$
Total multiplications = $10000 + 400 = 10400$.
- Consider computing **(AB)C**:
multiplications for (AB) = $2 \times 10 \times 50 = 1000$, creating a 2x50 answer matrix
multiplications for (AB)C = $2 \times 50 \times 20 = 2000$,
Total multiplications = $1000 + 2000 = 3000$

- Thus, our **goal** today is:
- Given a chain of matrices to multiply, determine the fewest number of multiplications necessary to compute the product.

- Formal Definition of the problem:
 - Let $A = A_1 \bullet A_2 \bullet \dots A_n$
 - Let $M_{i,j}$ denote the minimal number of multiplications necessary to find the product:
 $A_i \bullet A_{i+1} \bullet \dots A_j$.
 - And let $p_{i-1} \times p_i$ denote the dimensions of matrix A_i .
- We must attempt to determine the minimal number of multiplications necessary($m_{1,n}$) to find A ,
 - assuming that we simply do each single matrix multiplication in the standard method.

The key to solving this problem is noticing the ***sub-problem optimality condition***:

If a particular parenthesization of the whole product is optimal, then any sub-parenthesization in that product is optimal as well.

Say What?

If $(A (B ((CD) (EF))))$ is optimal

Then $(B ((CD) (EF)))$ is optimal as well

Proof on the next slide...

- Assume that we are calculating ABCDEF and that the following parenthesization is optimal:
 - (A (B ((CD) (EF))))
 - Then it is necessarily the case that
 - (B ((CD) (EF)))
 - is the optimal parenthesization of BCDEF.
- Why is this?
 - Because if it wasn't, and say (((BC) (DE)) F) was better, then it would also follow that
 - (A (((BC) (DE)) F)) was better than
 - (A (B ((CD) (EF)))),

- Our final multiplication will ALWAYS be of the form
 $(A_1 \bullet A_2 \bullet \dots A_k) \bullet (A_{k+1} \bullet A_{k+2} \bullet \dots A_n)$
- In essence, there is exactly one value of k for which we should "split" our work into two separate cases so that we get an optimal result.
 - Here is a list of the cases to choose from:
 - $(A_1) \bullet (A_2 \bullet A_3 \bullet \dots A_n)$
 - $(A_1 \bullet A_2) \bullet (A_3 \bullet A_4 \bullet \dots A_n)$
 - $(A_1 \bullet A_2 \bullet A_3) \bullet (A_4 \bullet A_5 \bullet \dots A_n)$
 - ...
 - $(A_1 \bullet A_2 \bullet \dots A_{n-2}) \bullet (A_{n-1} \bullet A_n)$
 - $(A_1 \bullet A_2 \bullet \dots A_{n-1}) \bullet (A_n)$
- Basically, count the number of multiplications in each of these choices and **pick the minimum**.
 - One other point to notice is that you have to account for the minimum number of multiplications in each of the two products.

Consider the case multiplying these 4 matrices:

A: 2×4

B: 4×2

C: 2×3

D: 3×1

1. (A)(BCD) - This is a 2×4 multiplied by a 4×1 ,
so $2 \times 4 \times 1 = 8$ multiplications, plus whatever work it will take to multiply (BCD).
2. (AB)(CD) - This is a 2×2 multiplied by a 2×1 ,
so $2 \times 2 \times 1 = 4$ multiplications, plus whatever work it will take to multiply (AB) and (CD).
3. (ABC)(D) - This is a 2×3 multiplied by a 3×1 ,
so $2 \times 3 \times 1 = 6$ multiplications, plus whatever work it will take to multiply (ABC).

- **Recursive formula:**

$M_{i,j} = \min \text{ value of } M_{i,k} + M_{k+1,j} + p_{i-1}p_kp_j, \text{ over all valid values of } k.$

- Now let's turn this recursive formula into a dynamic programming solution
 - Which sub-problems are necessary to solve first?
 - Clearly it's necessary to solve the smaller problems before the larger ones.
 - In particular, we need to know $m_{i,i+1}$, the number of multiplications to multiply any adjacent pair of matrices before we move onto larger tasks.
 - Similarly, the next task we want to solve is finding all the values of the form $m_{i,i+2}$, then $m_{i,i+3}$, etc.

MATRIX-CHAIN-ORDER(p)

```
1   $n \leftarrow \text{length}[p] - 1$ 
2  for  $i \leftarrow 1$  to  $n$ 
3      do  $m[i, i] \leftarrow 0$ 
4  for  $l \leftarrow 2$  to  $n$             $\triangleright l$  is the chain length.
5      do for  $i \leftarrow 1$  to  $n - l + 1$ 
6          do  $j \leftarrow i + l - 1$ 
7               $m[i, j] \leftarrow \infty$ 
8              for  $k \leftarrow i$  to  $j - 1$ 
9                  do  $q \leftarrow m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j$ 
10                     if  $q < m[i, j]$ 
11                         then  $m[i, j] \leftarrow q$ 
12                              $s[i, j] \leftarrow k$ 
13  return  $m$  and  $s$ 
```

Basically, we're checking different places to "split" our matrices by checking different values of k and seeing if they improve our current minimum value.

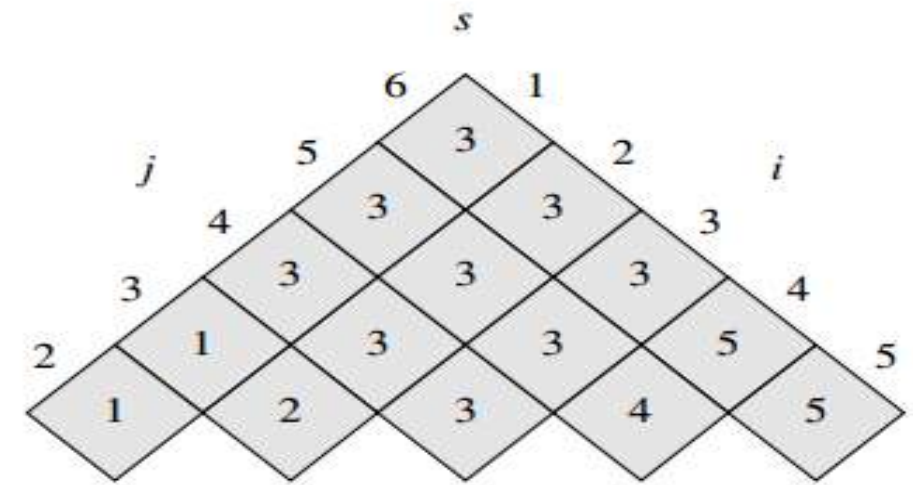
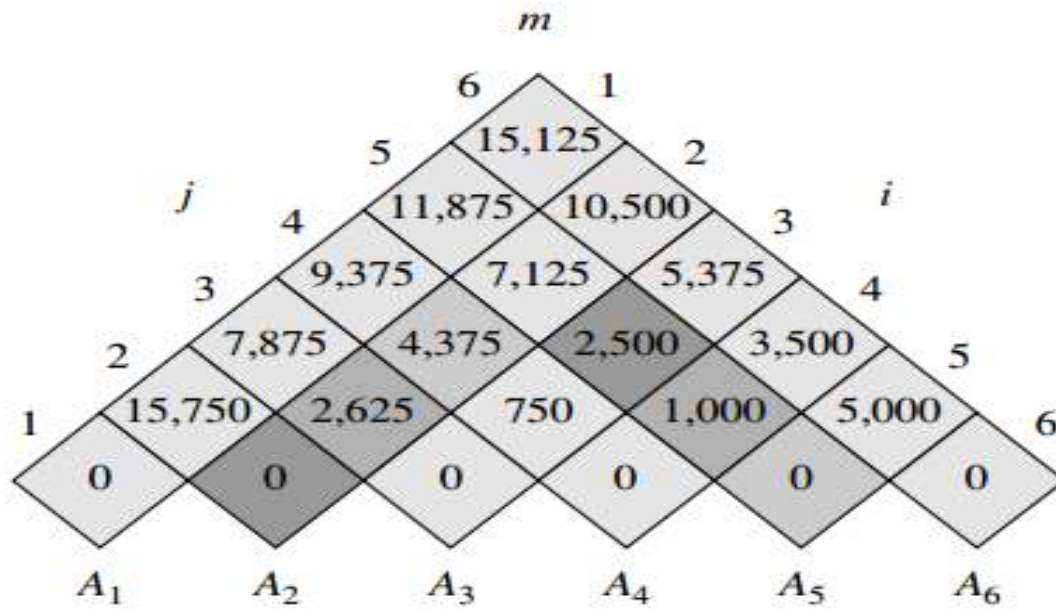


Figure 15.3 The *m* and *s* tables computed by MATRIX-CHAIN-ORDER for $n = 6$ and the following matrix dimensions:

matrix	dimension
A_1	30×35
A_2	35×15
A_3	15×5
A_4	5×10
A_5	10×20
A_6	20×25

$$m[2, 5] = \min \begin{cases} m[2, 2] + m[3, 5] + p_1 p_2 p_5 = 0 + 2500 + 35 \cdot 15 \cdot 20 = 13000, \\ m[2, 3] + m[4, 5] + p_1 p_3 p_5 = 2625 + 1000 + 35 \cdot 5 \cdot 20 = 7125, \\ m[2, 4] + m[5, 5] + p_1 p_4 p_5 = 4375 + 0 + 35 \cdot 10 \cdot 20 = 11375 \end{cases}$$

$$= 7125.$$

- . Matrix Chain Multiplication is a classic optimization problem in dynamic programming. The goal is to determine the most efficient way to multiply a sequence of matrices by minimizing the total number of scalar multiplications.
- Compute the cost for chains of increasing lengths.
 - Use a table to store intermediate results for subproblems to avoid recomputation.
 - Retrieve the optimal order by backtracking through the computed table.

SELF-ASSESSMENT QUESTIONS

Which of the following is true about Matrix Chain Multiplication?

- A. Matrix multiplication is commutative.
- B. The order of parenthesization does not affect the number of operations.
- C. The problem is solved using divide and conquer.
- D. **The problem is solved using dynamic programming.**

Which of the following statements about Matrix Chain Multiplication is correct?

- A. The sequence of matrices is rearranged to achieve the minimum cost.
- B. **The problem does not change the sequence of matrices but optimizes the parenthesization.**
- C. The algorithm computes the determinant of the resulting matrix.
- D. The problem assumes that all matrices have square dimensions.

TERMINAL QUESTIONS

1. Solve the Matrix Chain Multiplication problem for the sequence of matrices with dimensions: 10×20 , 20×30 , 30×40 . Show all steps in detail.
2. Explain the time and space complexity of the Matrix Chain Multiplication algorithm.

REFERENCES FOR FURTHER LEARNING OF THE SESSION

Reference Books :

1. Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein., 3rd, 2009, The MIT Press.
- 2 Algorithm Design Manual, Steven S. Skiena., 2nd, 2008, Springer.
- 3 Data Structures and Algorithms in Python, Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser., 2nd, 2013, Wiley.
- 4 The Art of Computer Programming, Donald E. Knuth, 3rd, 1997, Addison-Wesley Professiona.

MOOCS :

1. <https://www.coursera.org/specializations/algorithms?=>
2. <https://www.coursera.org/learn/dynamic-programming-greedy-algorithms#modules>

THANK YOU

Advanced Algorithms & Data Structures



Department of CSE

ADVANCED ALGORITHMS AND DATA STRUCTURES 23CS03HF

Topic:

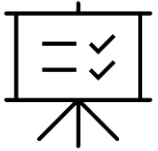
Longest Common Subsequence

AIM OF THE SESSION



To familiarize students with the concept of Longest Common Subsequence.

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

1. Demonstrate :- Longest Common Subsequence.
2. Describe :- Use of Dynamic programming to find longest common subsequence.

LEARNING OUTCOMES



At the end of this session, you should be able to:

1. Define :- Longest Common Subsequence.
2. Describe :- Learn to formulate the recurrence relation for the LCS problem..
3. Summarize:- LCS problem identifies the longest sequence common to two strings in the same relative order using dynamic programming.

Longest Common Subsequence

Application: comparison of two DNA strings

Ex: $X = \{A B C B D A B\}$, $Y = \{B D C A B A\}$

Longest Common Subsequence:

$X = A \text{ **B** } \text{ **C** } \text{ **B** } D \text{ **A** } B$

$Y = \text{ **B** } D \text{ **C** } A \text{ **B** } \text{ **A** }$

Brute force algorithm would compare each subsequence of X with the symbols in Y

- if $|X| = m$, $|Y| = n$, then there are 2^m subsequences of X ; we must compare each with Y (n comparisons)
- So the running time of the brute-force algorithm is $O(n 2^m)$
- Notice that the LCS problem has *optimal substructure*: solutions of subproblems are parts of the final solution.
- Subproblems: “find LCS of pairs of *prefixes* of X and Y ”

- First we'll find the length of LCS. Later we'll modify the algorithm to find LCS itself.
- Define X_i, Y_j to be the prefixes of X and Y of length i and j respectively
- Define $c[i,j]$ to be the length of LCS of X_i and Y_j
- Then the length of LCS of X and Y will be $c[m,n]$

$$c[i, j] = \begin{cases} c[i-1, j-1] + 1 & \text{if } x[i] = y[j], \\ \max(c[i, j-1], c[i-1, j]) & \text{otherwise} \end{cases}$$

$$c[i, j] = \begin{cases} c[i-1, j-1] + 1 & \text{if } x[i] = y[j], \\ \max(c[i, j-1], c[i-1, j]) & \text{otherwise} \end{cases}$$

- We start with $i = j = 0$ (empty substrings of x and y)
- Since X_0 and Y_0 are empty strings, their LCS is always empty (i.e. $c[0,0] = 0$)
- LCS of empty string and any other string is empty, so for every i and j:
 $c[0, j] = c[i, 0] = 0$

$$c[i, j] = \begin{cases} c[i-1, j-1] + 1 & \text{if } x[i] = y[j], \\ \max(c[i, j-1], c[i-1, j]) & \text{otherwise} \end{cases}$$

- When we calculate $c[i, j]$, we consider two cases:
- **First case:** $x[i]=y[j]$: one more symbol in strings X and Y matches, so the length of LCS X_i and Y_j equals to the length of LCS of smaller strings X_{i-1} and Y_{j-1} , plus 1

$$c[i, j] = \begin{cases} c[i-1, j-1] + 1 & \text{if } x[i] = y[j], \\ \max(c[i, j-1], c[i-1, j]) & \text{otherwise} \end{cases}$$

- **Second case: $x[i] \neq y[j]$**
- As symbols don't match, our solution is not improved, and the length of $\text{LCS}(X_i, Y_j)$ is the same as before (i.e. maximum of $\text{LCS}(X_i, Y_{j-1})$ and $\text{LCS}(X_{i-1}, Y_j)$)

Why not just take the length of $\text{LCS}(X_{i-1}, Y_{j-1})$?

LCS Length Algorithm

LCS-Length(X, Y)

1. $m = \text{length}(X)$ // get the # of symbols in X
2. $n = \text{length}(Y)$ // get the # of symbols in Y
3. for $i = 1$ to m $c[i,0] = 0$ // special case: Y_0
4. for $j = 1$ to n $c[0,j] = 0$ // special case: X_0
5. for $i = 1$ to m // for all X_i
6. for $j = 1$ to n // for all Y_j
7. if ($X_i == Y_j$)
8. $c[i,j] = c[i-1,j-1] + 1$
9. else $c[i,j] = \max(c[i-1,j], c[i,j-1])$
10. return $c[m,n]$ // return LCS length for X and Y

LCS Example

We'll see how LCS algorithm works on the following example:

- $X = \text{ABCB}$
- $Y = \text{BDCAB}$

What is the Longest Common Subsequence of X and Y?

$\text{LCS}(X, Y) = \text{BCB}$

$X = A \quad \mathbf{B} \quad \quad \mathbf{C} \quad \quad \mathbf{B}$

$Y = \quad \mathbf{B} \quad \mathbf{D} \quad \mathbf{C} \quad \mathbf{A} \quad \mathbf{B}$

LCS Example (0)

ABCB

BDCAB

		j	0	1	2	3	4	5
			Y _j	B	D	C	A	B
i								
0	X _i							
1	A							
2	B							
3	C							
4	B							

$X = ABCB; \quad m = |X| = 4$

$Y = BDCAB; \quad n = |Y| = 5$

Allocate array $c[5,4]$

LCS Example (1)

ABCB

BDCAB

		j	0	1	2	3	4	5
			Y _j	B	D	C	A	B
i	X _i							
0			0	0	0	0	0	0
1	A		0					
2	B		0					
3	C		0					
4	B		0					

for i = 1 to m $c[i,0] = 0$

for j = 1 to n $c[0,j] = 0$

LCS Example (2)

A B C B

B D C A B

i \ j	0	1	2	3	4	5
	Y _j	B	D	C	A	B
0	X _i	0	0	0	0	0
1	A	0	0			
2	B	0				
3	C	0				
4	B	0				

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (3)

A B C B

B D C A B

j	0	1	2	3	4	5
	Y _j	B	D	C	A	B
i	X _i					
0		0	0	0	0	0
1	A	0	0	0	0	
2	B	0				
3	C	0				
4	B	0				

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (4)

A B C B

B D C A B

		j	0	1	2	3	4	5
			Y _j	B	D	C	A	B
i	X _i	0	0	0	0	0	0	0
1	A	0	0	0	0	0	1	
2	B	0						
3	C	0						
4	B	0						

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (5)

A B C B

B D C A B

i \ j	0	1	2	3	4	5
	Y _j	B	D	C	A	B
0	X _i	0	0	0	0	0
1	A	0	0	0	1	1
2	B	0				
3	C	0				
4	B	0				

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (6)

ABCB
BDCAB

		j	0	1	2	3	4	5
			Y _j	B	D	C	A	B
i	0	X _i	0	0	0	0	0	0
1	A	0	0	0	0	0	1	1
2	B	0	1					
3	C	0						
4	B	0						

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (7)

ABCB
BD CAB

j	0	1	2	3	4	5
i	Y _j	B	D	C	A	B
0	X _i	0	0	0	0	0
1	A	0	0	0	0	1
2	B	0	1	1	1	
3	C	0				
4	B	0				

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (8)

ABCB

BDCAB

		j	0	1	2	3	4	5
			Yj	B	D	C	A	B
i	Xi	0	0	0	0	0	0	0
1	A	0	0	0	0	0	1	1
2	B	0	1	1	1	1	1	2
3	C	0						
4	B	0						

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (10)

ABCB

BD CAB

j	0	1	2	3	4	5
i	Y _j	B	D	C	A	B
0	X _i	0	0	0	0	0
1	A	0	0	0	1	1
2	B	0	1	1	1	2
3	C	0	1	1		
4	B	0				

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (11)

ABCB

BDCAB

i \ j	0	1	2	3	4	5
	Y _j	B	D	C	A	B
0	X _i	0	0	0	0	0
1	A	0	0	0	1	1
2	B	0	1	1	1	2
3	C	0	1	1	2	
4	B	0				

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (12)

ABCB

BDCAB

j	0	1	2	3	4	5
i	Yj	B	D	C	A	B
0	Xi	0	0	0	0	0
1	A	0	0	0	1	1
2	B	0	1	1	1	2
3	C	0	1	1	2	2
4	B	0				

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (13)

ABCB

BDCAB

j	0	1	2	3	4	5
Yj		B	D	C	A	B
Xi						
0	0	0	0	0	0	0
1	A	0	0	0	1	1
2	B	0	1	1	1	2
3	C	0	1	1	2	2
4	B	0	1			

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (14)

ABCB

BD CAB

i \ j	0	1	2	3	4	5
	Y _j	B	D	C	A	B
0	X _i	0	0	0	0	0
1	A	0	0	0	0	1
2	B	0	1	1	1	2
3	C	0	1	1	2	2
4	B	0	1	1	2	2

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Example (15)

ABCB

BDCAB

i \ j	0	1	2	3	4	5
	Y _j	B	D	C	A	B
0	X _i	0	0	0	0	0
1	A	0	0	0	1	1
2	B	0	1	1	1	2
3	C	0	1	1	2	2
4	B	0	1	1	2	3

if ($X_i == Y_j$)
 $c[i,j] = c[i-1,j-1] + 1$
 else $c[i,j] = \max(c[i-1,j], c[i,j-1])$

LCS Algorithm Running Time

- LCS algorithm calculates the values of each entry of the array $c[m,n]$
- So what is the running time?

$O(m*n)$

since each $c[i,j]$ is calculated in constant time, and there are $m*n$ elements in the array

How to find actual LCS

- So far, we have just found the *length* of LCS, but not LCS itself.
- We want to modify this algorithm to make it output Longest Common Subsequence of X and Y

Each $c[i,j]$ depends on $c[i-1,j]$ and $c[i,j-1]$
 or $c[i-1, j-1]$

For each $c[i,j]$ we can say how it was acquired:

2	2	
2	3	

For example, here
 $c[i,j] = c[i-1,j-1] + 1 = 2+1=3$

How to find actual LCS - continued

- Remember that

$$c[i, j] = \begin{cases} c[i-1, j-1] + 1 & \text{if } x[i] = y[j], \\ \max(c[i, j-1], c[i-1, j]) & \text{otherwise} \end{cases}$$

So we can start from $c[m, n]$ and go backwards

Look first to see if 2nd case above was true

If not, then $c[i, j] = c[i-1, j-1] + 1$, so remember $x[i]$ (because $x[i]$ is a part of LCS)

When $i=0$ or $j=0$ (i.e. we reached the beginning), output remembered letters in reverse order

Algorithm to find actual LCS

- Here's a recursive algorithm to do this:

```
LCS_print(x, m, n, c) {  
    if (c[m][n] == c[m-1][n]) // go up?  
        LCS_print(x, m-1, n, c);  
    else if (c[m][n] == c[m][n-1]) // go left?  
        LCS_print(x, m, n-1, c);  
    else { // it was a match!  
        LCS_print(x, m-1, n-1, c);  
        print(x[m]); // print after recursive call  
    }  
}
```


Finding LCS

		j	0	1	2	3	4	5
i		Yj	B	D	C	A	B	
	Xi							
0	A	0	0	0	0	0	0	
1	B	0	0	0	0	1	1	
2	C	0	1	1	1	1	2	
3	D	0	1	1	2	2	2	
4	E	0	1	1	2	2	3	

Finding LCS (2)

		j	0	1	2	3	4	5
i	Xi	Yj		B	D	C	A	B
0			0	0	0	0	0	0
1	A		0	0	0	0	1	1
2	B		0	1	1	1	1	2
3	C		0	1	1	2	2	2
4	B		0	1	1	2	2	3

LCS (reversed order): **B C B**

LCS (straight order): **B C B**

(this string turned out to be a palindrome)

The **Longest Common Subsequence (LCS)** problem involves finding the longest sequence of characters that appear in the same relative order in two given strings, without needing to be contiguous. LCS is widely used in **bioinformatics, text comparison, and data analysis.**

SELF-ASSESSMENT QUESTIONS

1. What is the length of the LCS for X="AGGTAB"X = "AGGTAB"X="AGGTAB" and Y="GTXAYB"Y = "GTXAYB"Y="GTXAYB"?

A. 2

B. 4

C. 5

D. 6

2. In the context of LCS, a subsequence is defined as:

A. A contiguous subset of characters

B. A sequence derived by deleting characters without rearranging others

C. A rearrangement of characters to form another string

D. A subsequence of maximum length

TERMINAL QUESTIONS

1. Given two strings $X = \text{"ABCB DAB"}$ and $Y = \text{"BDCAB"}$, describe the steps involved in constructing the LCS table and backtracking to find the LCS.
2. Describe the dynamic programming approach to solve the LCS problem. How do you construct the DP table, and what is the recurrence relation used?

REFERENCES FOR FURTHER LEARNING OF THE SESSION

Reference Books :

1. Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein., 3rd, 2009, The MIT Press.
- 2 Algorithm Design Manual, Steven S. Skiena., 2nd, 2008, Springer.
- 3 Data Structures and Algorithms in Python, Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser., 2nd, 2013, Wiley.
- 4 The Art of Computer Programming, Donald E. Knuth, 3rd, 1997, Addison-Wesley Professiona.

MOOCS :

1. <https://www.coursera.org/specializations/algorithms?=>
2. <https://www.coursera.org/learn/dynamic-programming-greedy-algorithms#modules>

THANK YOU

Advanced Algorithms & Data Structures



Department of CSE

ADVANCED ALGORITHMS AND DATA STRUCTURES 23CS03HF

Topic:

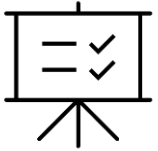
Dynamic Programming – 0/1 Knapsack Problem

AIM OF THE SESSION



To familiarize students with the concept of Knapsack problem.

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

1. Demonstrate :- Knapsack problem.
2. Describe :- Bounding function.

LEARNING OUTCOMES



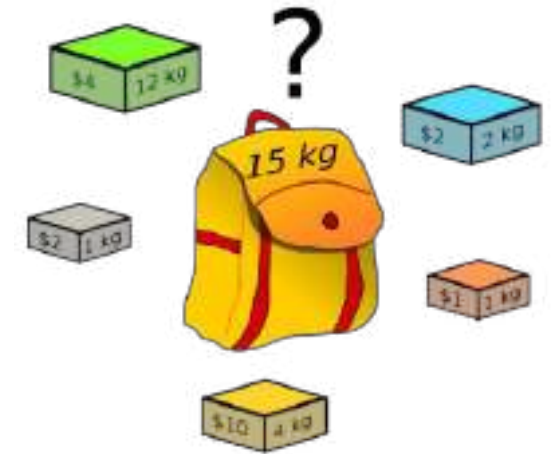
At the end of this session, you should be able to:

1. Define :- Knapsack problem.
2. Describe :- Bounding function.
3. Summarize:- Backtracking solution to the 0/1 knapsack problem.

Knapsack Problem

You are given the following-

- A knapsack - with limited weight capacity.
- items - having some weight and value.



The problem states-

Which items should be placed into the knapsack such that-

- The value or profit obtained by putting the items into the knapsack is maximum.
- And the weight limit of the knapsack does not exceed.



Knapsack Problem variants

Variants:

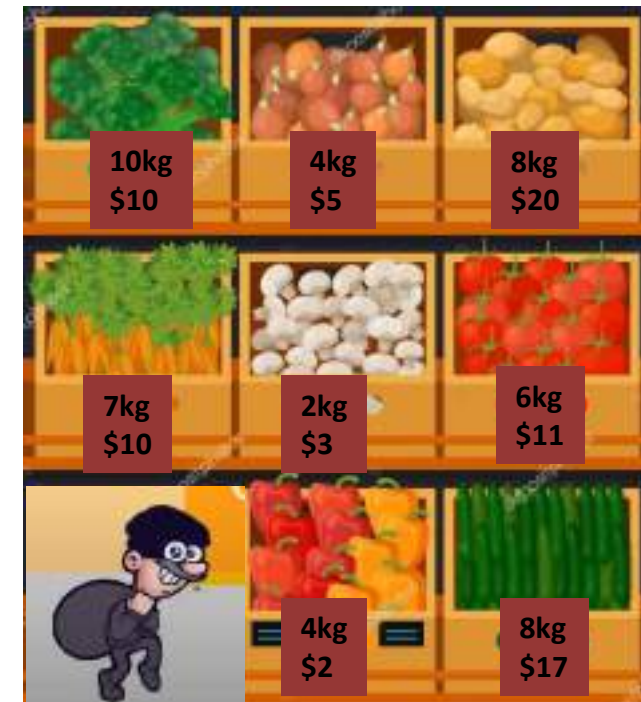
– 0/1 Knapsack.

- not allowed to break items. We either take the whole item or don't take it.



Fractional Knapsack

- can break items for maximizing the total value of knapsack



0/1 Knapsack Problem

In 0/1 Knapsack Problem,

- As the name suggests, items are indivisible here.
- We can not take the fraction of any item.
- We have to either take an item completely or leave it completely.

Example:

Consider the knapsack instance $n = 3$, $(w_1, w_2, w_3) = (2, 3, 4)$, $(p_1, p_2, p_3) = (1, 2, 5)$ and $m = 6$.

Probability of Chosen Items $(x_i) = [\{0, 0, 0\}, \{0, 0, 1\}, \dots, \{1, 1, 1\}]$

No. of Possible Solutions $(2^n) = 2^3 = 8$.

The problem is to find the Best Optimal Solution among the 8 for the 0/1 Knapsack.

0/1 Knapsack Problem-solution: Set Method

Initially $S^0 = \{(0,0)\}$

$$S_1^i = \{ (P, W) / (P - p_{i+1}, W - w_{i+1}) \in S^i \}$$

S^{i+1} can be computed by merging from S^i and S_1^i

- **Purging or dominance rule:** if S^{i+1} contains two pairs (p_j, w_j) and (p_k, w_k) with the property that $p_j \leq p_k$ and $w_j \geq w_k$ then the pair (p_j, w_j) can be discarded.
- When generating S^i , we can also purge all pairs (p, w) with $w > m$ as these pairs determine the value of $f_n(x)$ only for $x > m$.
- The optimal solution $f_n(m)$ is given by the *highest profit pair*.

**Example 1: Consider the knapsack instance $n = 3$,
 $(w_1, w_2, w_3) = (2, 3, 4)$, $(p_1, p_2, p_3) = (1, 2, 5)$, and $m = 6$.**

Initially

$$S^0 = \{(0, 0)\}$$

Include 1st object

$$S_1^0 = \{(0+1, 0+2)\} = \{(1, 2)\}$$

Next Stage can be obtained S^{0+1} (S^1) can be computed by merging from S^0 and S_1^0

$$S^1 = \{(0, 0), (1, 2)\}$$

Include 2nd object

$$S_1^1 = \{(0+2, 0+3), (1+2, 2+3)\} = \{(2, 3), (3, 5)\}$$

Next Stage can be obtained S^{1+1} (S^2) can be computed by merging from S^1 and S_1^1

$$S^2 = \{(0, 0), (1, 2), (2, 3), (3, 5)\}$$

Include 3rd object

$$\begin{aligned} S_1^2 &= \{(0+5, 0+4), (1+5, 2+4), (2+5, 3+4), (3+5, 5+4)\} \\ &= \{(5, 4), (6, 6), (7, 7), (8, 9)\} \end{aligned}$$

Next Stage can be obtained S^{2+1} (S^3) can be computed by merging from S^2 and S_1^2

$$S^3 = \{(0,0)(1,2)(2,3)(3,5) (5,4)(6,6)(7,7)(8,9)\}$$

Apply Purging rule

Pairs(3,5) (7,7)(8,9) will be discarded

Therefore ,

$$S^3 = \{(0,0)(1,2)(2,3)(5,4)(6,6)\}$$

To find the included items:

(6,6) is in S^3 So $x_3 = 1$

$$P - p_3 = 6 - 5 = 1 \text{ \& } W - w_3 = 6 - 4 = 2$$

(1,2) is in S^1 So $x_1 = 1$

Therefore $X = (1,0,1)$

0/1 Knapsack Algorithm

Algorithm DKP(p,w,n,m)

{

$S^0 := \{(0,0)\};$

for i := 1 to n-1 do

{

$S^{i-1} = \{(P,W) \mid (P-p_i, W-w_i) \in S^{i-1} \text{ and } W \leq m\};$

$S^i = \text{MergePurge}(S^{i-1}, S_1^{i-1});$

}

$(P_x, W_x) = \text{last pair in } S^{n-1} ;$

$(P_y, W_y) = (P^1 + p_n, W^1 + w_n)$ where W^1 is the largest W in any pair in S^{n-1} such that

$W + w_n \leq m;$

```
// Trace back for  $x_n, x_{n-1}, \dots, x_1$ .  
if ( $P_x > P_y$ ) then  
     $x_n = 0$ ;  
else  $x_n = 1$ ;  
TraceBackFor( $x_{n-1}, \dots, x_1$ );  
}
```

Complexity Analysis:

The complexity of the algorithm is $O(nW)$ where, n is the number of items and W is the capacity of knapsack.

- **Example 2** Solve Knapsack instance $M = 8$, and $n = 4$. Let P_i and W_i are as shown below.

i	P_i	W_i
1	1	2
2	2	3
3	5	4
4	6	5

Solution : Let us build the sequence of decision S^0, S^1, S^2 .

$$S^0 = \{(0, 0)\} \text{ initially}$$

$$S_1^0 = \{(1, 2)\}$$

That means while building S_1^0 we select the next i^{th} pair. For S_1^0 we have selected first (P, W) pair which is $(1, 2)$.

Now

$$S^1 = \{\text{Merge } S^0 \text{ and } S_1^0\}$$

$$= \{(0, 0), (1, 2)\}$$

$$S_1^1 = \{\text{Select next } (P, W) \text{ pair and add it with } S^1\}$$

$$= \{(2, 3), (2+0, 3+0), (2+1, 3+2)\}$$

$$S_1^1 = \{(2, 3), (3, 5)\} \quad \because \text{Repetition of } (2, 3) \text{ is avoided.}$$

$$S^2 = \{\text{Merge candidates from } S^1 \text{ and } S_1^1\}$$

$$= \{(0, 0), (1, 2), (2, 3), (3, 5)\}$$

$$\therefore S_1^2 = \{\text{Select next } (P, W) \text{ pair and add it with } S^2\}$$

$$= \{(5, 4), (6, 6), (7, 7), (8, 9)\}$$

Now $S^3 = \{\text{Merge candidates from } S^2 \text{ and } S_1^2\}$

$$S^3 = \{(0, 0), (1, 2), (2, 3), (5, 4), (6, 6), (7, 7), (8, 9)\}$$

Note that the pair (3, 5) is purged from S^3 . This is because, let us assume $(P_j, W_j) = (3, 5)$ and $(P_k, W_k) = (5, 4)$. Here $P_j \leq P_k$ and $W_j > W_k$ is true hence we will eliminate pair (P_j, W_j) i.e. (3, 5) from S^3 .

$$S_1^3 = \{(6, 5), (7, 7), (8, 8), (11, 9), (12, 11), (13, 12), (14, 14)\}$$

$$S^4 = \{(0, 0), (1, 2), (2, 3), (5, 4), (6, 6), (7, 7), (8, 9), \\ (6, 5), (8, 8), (11, 9), (12, 11), (13, 12), (14, 14)\}$$

Now we are interested in $M = 8$. We get pair (8, 8) in S^4 . Hence we will set $x_4 = 1$.
 Now to select next object $(P - P_4)$ and $(W - W_4)$.

i.e. $(8 - 6)$ and $(8 - 5)$.

i.e. (2, 3)

Pair $(2, 3) \in S^2$. Hence set $x_2 = 1$. So we get the final solution as (0, 1, 0, 1).

Example 3:

Consider the knapsack instance $n = 3$, $m=50$, $(w_1, w_2, w_3) = (10,20,30)$ & $(p_1, p_2, p_3) = (60, 100, 120)$.

The 0/1 Knapsack problem can be defined as follows: We have (N) items, each with a weight (w_i) and a value (v_i) . We also have a bag with a capacity (W). The goal is to select items to place in the bag such that the total value is maximized, without exceeding the bag's capacity.

SELF-ASSESSMENT QUESTIONS

1. What is the main objective of the 01 Knapsack problem?

- A. To maximize the total weight of items in the knapsack
- B. To minimize the total value of items in the knapsack
- C. To maximize the total value of items in the knapsack without exceeding the weight capacity
- D. To minimize the total weight of items in the knapsack

In the 0/1 Knapsack problem, if (n) is the number of items and (W) is the maximum weight capacity, what is the time complexity of the dynamic programming solution?

- A. ($O(n + W)$)
- B. ($O(nW)$)
- C. ($O(n^2)$)
- D. ($O(n^3)$)

TERMINAL QUESTIONS

1. What is the dynamic 0/1 knapsack problem?
2. Can we solve knapsack problem using dynamic programming?
3. What is the formula for the 0/1 knapsack problem?

REFERENCES FOR FURTHER LEARNING OF THE SESSION

Reference Books :

1. Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein., 3rd, 2009, The MIT Press.
- 2 Algorithm Design Manual, Steven S. Skiena., 2nd, 2008, Springer.
- 3 Data Structures and Algorithms in Python, Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser., 2nd, 2013, Wiley.
- 4 The Art of Computer Programming, Donald E. Knuth, 3rd, 1997, Addison-Wesley Professiona.

MOOCS :

1. <https://www.coursera.org/specializations/algorithms?=>
2. <https://www.coursera.org/learn/dynamic-programming-greedy-algorithms#modules>

THANK YOU

